

CS3520
Computer Organisation and Architecture I
Section 5 - Processor Design and Implementation

Mr K.O, Setetemela
BJO 308
Department of Mathematics & Computer Science
National University of Lesotho
w: 59105441 | e: khobatha.setetemela@gmail.com

Lecture Outcomes

- Understand different classes of computers and their characteristic requirements
- Understand basic computer design principles and conventions
- Specify CPU requirements using an instruction set architecture
- Design a datapath given a simplified instruction set architecture for both single-cycle implementation and pipelined implementation
- Understand pipeline hazards and how they can be resolved
- Design CPU control for both single-cycle implementation and pipelined implementation

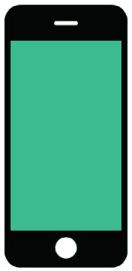
Lecture Outline

- Introduction
  - Classes of computer systems
  - 8 great computer design principles
  - Logic design conventions
- Building a datapath
  - Stage by stage: Fetch, decode, execute, access memory and write back
  - Unified datapath
- Simple implementation scheme
  - ALU control
  - Main control unit design and implementation strategies
- Pipelined Implementation
  - Why single implementation not used today
  - Overview of pipelining
  - Instruction set design for pipelining
  - Pipeline hazards
  - Pipelined datapath and control

INTRODUCTION:

Applications: Classes of Computers

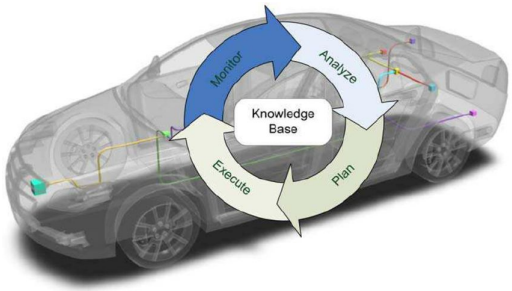
Feature	Personal mobile device (PMD)	Desktop	Server	Clusters/warehouse-scale computer	Embedded
Price of system	$100–$1000	$300–$2500	$5000–$10,000,000	$100,000–$200,000,000	$10–$100,000
Price of micro-processor	$10–$100	$50–$500	$200–$2000	$50–$250	$0.01–$100
Critical system design issues	Cost, energy, media performance, responsiveness	Price-performance, energy, graphics performance	Throughput, availability, scalability, energy	Price-performance, throughput, energy proportionality	Price, energy, application-specific performance











Eight Great Computer Design Principles

- **Design for Moore's law**
  - Moore's law states that integrated circuit resources double every 18 – 24 months
  - Computer architects must design with the future resources of ICs as predicted by Moore's law in mind
- **Use abstraction to simplify design**
  - Abstraction helps reduce design time and increase productivity
  - Use abstraction to represent the design clearly and simply at different levels of representation
- **Make the common case fast**
  - Making the common case fast tend to to enhance performance better than optimising the rare case
  - The common case is often simpler than the rare case and hence easier to optimise
- **Performance via parallelism**
  - Performing operations in parallel can result in performance enhancements
  - However, potential parallel performance enhancement is limited by Amdahl's law

Eight Great Computer Design Principles

- **Performance via pipelining**
  - Pipelining is a particular form of parallelism
- **Performance via prediction**
  - In some cases it can be faster on average to guess and start working rather than wait until you know for sure
  - Assuming that the mechanism to recover from a misprediction is not too expensive and your prediction is relatively accurate
- **Hierarchy of memories**
  - Programmer want memory to be fast, large and cheap, as memory speed often shapes performance, capacity limits the size of problems that can be solved, and the cost of memory forms majority of computer cost
  - Address through a hierarchy of memories, with the fastest, smallest and most expensive memory per bit at the top of the hierarchy and the slowest, largest and cheapest per bit at the bottom
  - Cache gives the programmer the illusion that main memory is nearly as fast as the top of the hierarchy and nearly as big and cheap as the bottom of the hierarchy
- **Dependability via redundancy**
  - Computers not only need to be fast, they must be dependable
  - Any physical device can fail, therefore we make systems dependable by including redundant components that can take over during failure and to help detect failures

Moore's Law

- **Definition:** Moore's Law states that the number of transistors on a microchip doubles approximately every two years, leading to exponential growth in computing power.
- **Key Implications:**
  - Performance Improvement: Faster processors and more powerful devices.
  - Cost Reduction: Cheaper production of transistors over time.
  - Miniaturization: Devices become smaller and more efficient.
- **Historical Trend:**
  - Formulated by Gordon Moore (1965), co-founder of Intel.
  - Initially observed for the first few decades of chip manufacturing.
- **Limitations:**
  - Physical Limits: Miniaturization nearing atomic scales.
  - R&D Challenges: Increasing heat dissipation and power consumption.
  - Shift to Parallelism: Modern focus on multi-core architectures and quantum computing.
- **Current Status:** Moore's Law is slowing down, but new innovations (e.g., chip stacking, AI accelerators) aim to continue performance growth.

Amdahl's Law

- **Definition:** Amdahl's Law states that the maximum speedup of a task using multiple processors is limited by the portion of the task that cannot be parallelized.
- **Formula:** $S(N) = \frac{1}{(1-P)+\frac{P}{N}}$
  - **S(N):** Speedup with N processors
  - **P:** Fraction of the program that is parallelizable
  - **1 - P:** Fraction that must remain serial
- **Key Insights:**
  - Linear speedup is only possible if the entire task is parallelizable.
  - Diminishing returns occur as N increases, due to the serial portion.
  - **Optimization Focus:** Minimize the serial portion (1 - P) for better performance gains.
- **Example:**
  - If 30% of a task is serial (P = 0.7), adding more processors won't achieve more than ~3.3x speedup, no matter how many processors are used.

Which One is Faster?







What is Performance?

- Several different definitions of performance
  - **Desktop computing:** a faster program is the one that gets the job done first? That is, shorter response time.
  - **Mobile computing:** a better performing mobile device is the one that lets me enjoy long battery life?
  - **Cluster computing:** the faster server is the one that gets most jobs completed in a day? That is, higher throughput and bandwidth.
- Therefore, in most cases different benchmark kernel applications and performance metrics are needed to benchmark different computing platforms
  - Generally, response time used for personal computing platforms such as mobile devices
  - Throughput and bandwidth used for large scale computing platforms such as servers
  - Both response time and throughput used for high-performance embedded computing

Performance Measurement: User View

- **Time is generally the most basic measure of computer performance**
  - The computer that performs the same amount of work in the shortest time is the fastest
- **Program execution time measurement**
  - Wall clock time / response time/ elapsed time
  - Includes disk accesses, memory accesses, I/O operations, OS overhead etc
- **CPU time**
  - Time spent by the CPU processing a task
  - Excludes I/O overhead, disk accesses, memory accesses
  - CPU time = User CPU time + System CPU time

Performance Measurement: Designer View

- Computer users and designers often consider performance using different metrics
  - Users concerned about response time, throughput
  - Designers concerned about clock speed, dynamic power, word length etc
- By relating these different metrics, we could determine the effect of a change in the computer design aspect on performance as experienced by the user

Program CPU Execution Time (sec) = No. of CPU Clock Cycles for Program x Clock Cycle Time

= Program instructions count x Average Clock Time per Instruction (CPI)

= Program instructions count x CPI x Clock Cycle Time (sec)

Exercise: Improving Performance

Our favorite program runs in 10 seconds on computer A, which has a 2GHz clock. We are trying to help a computer designer build a computer B, which will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program.

Q:What clock rate should we tell the designer to target?

Performance Factors

- CPU performance factors
  - Instruction count
    - Determined by ISA and compiler
  - CPI and Cycle time
    - Determined by CPU hardware

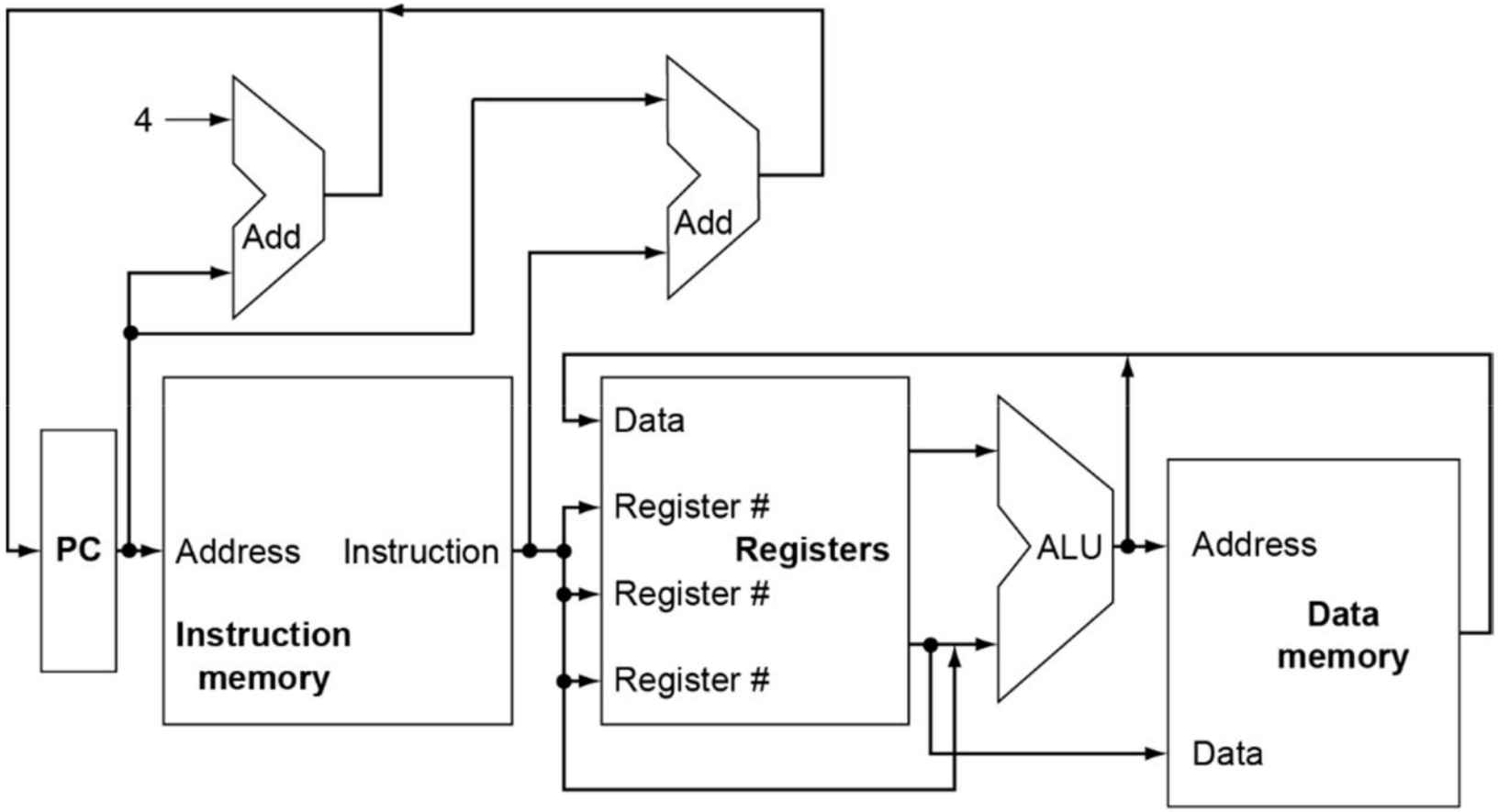
RISC-V Implementations Overview

- We will examine two RISC-V implementations
  - A simplified version
  - A more realistic pipelined version
- Simple subset, shows most aspects
  - Memory reference: ld, sd
  - Arithmetic/logical: add, sub, and, or
  - Control transfer: beq

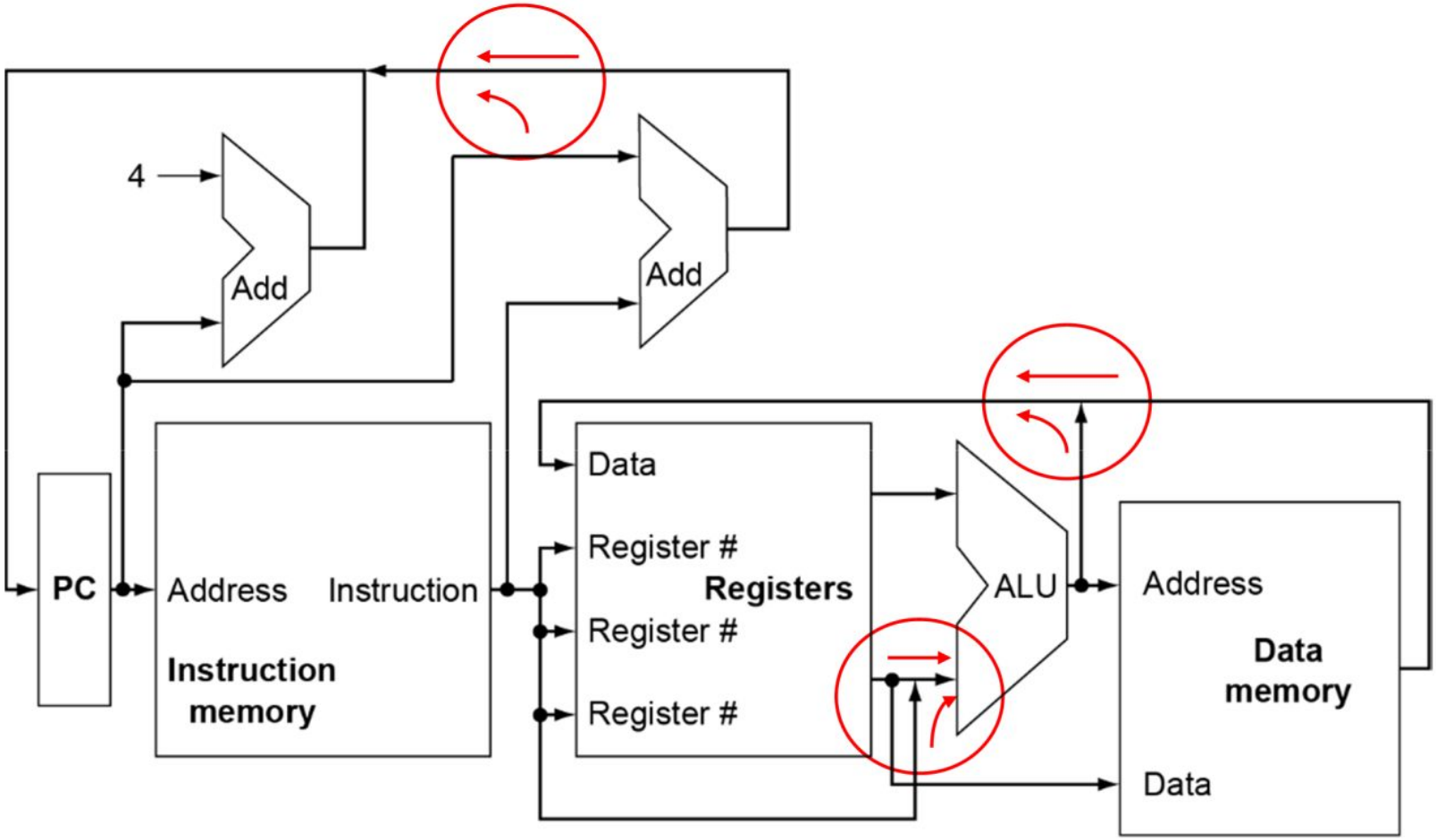
Instruction Execution

- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch comparison
  - Access data memory for load/store
  - PC target address or PC + 4

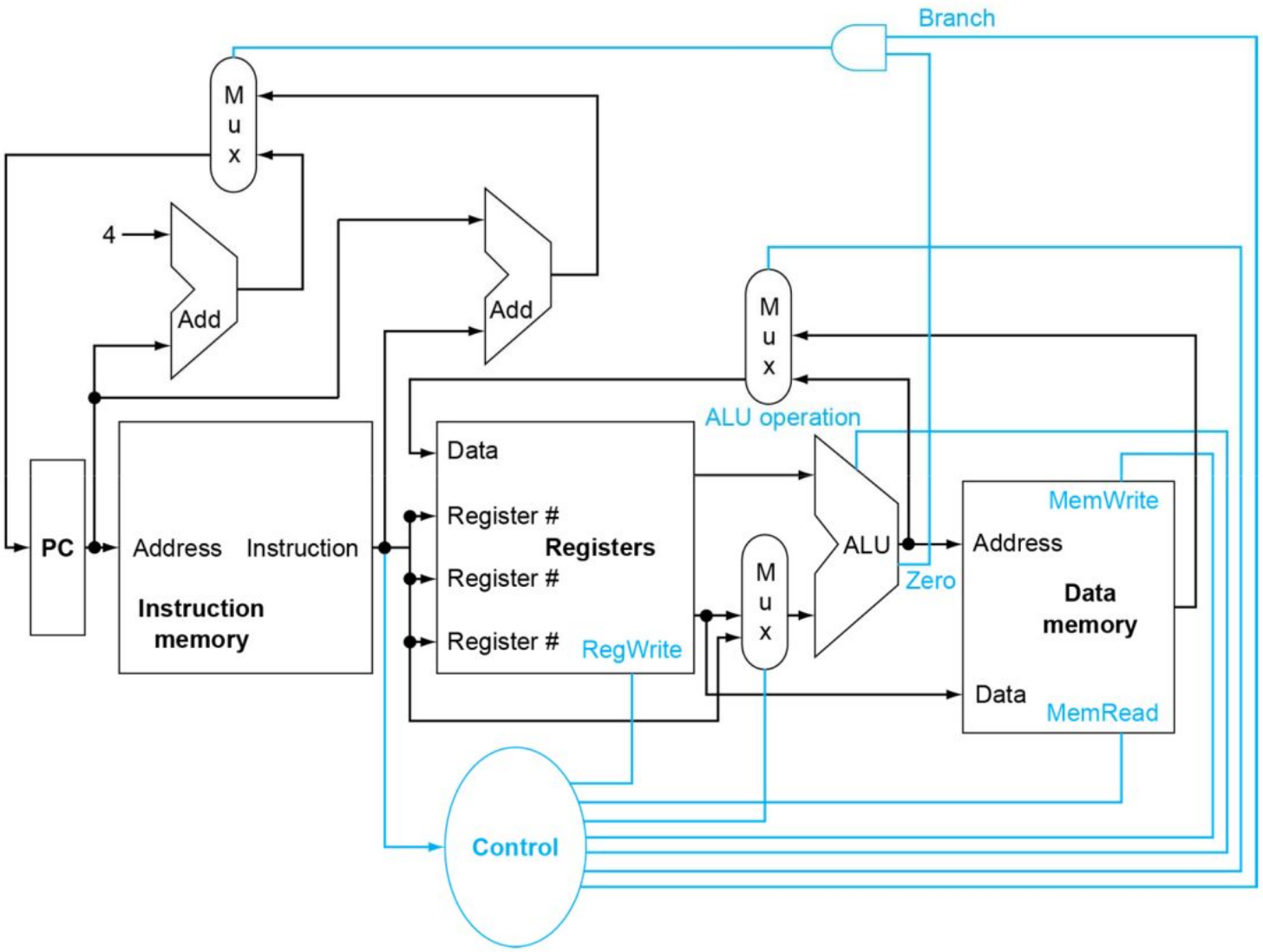
CPU Overview



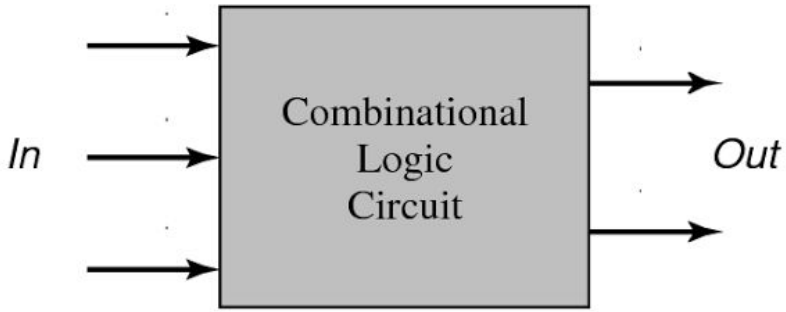
Multiplexers



Control

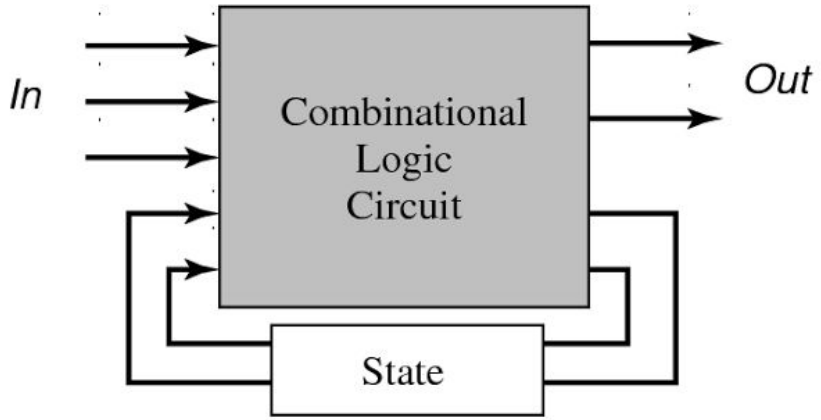

Branch
M
u
x
4
Add
Add
M
u
x
ALU operation
PC
Address
Instruction
Instruction
memory
Data
Register #
Registers
Register #
Register #
RegWrite
M
u
x
ALU
Zero
MemWrite
Address
Data
memory
Data
MemRead
Control

Logic Design Conventions: Processor Build from Combinational and Sequential Circuits



(a) Combinational

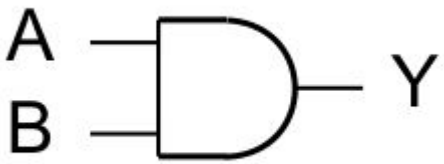
output = *f(in)*

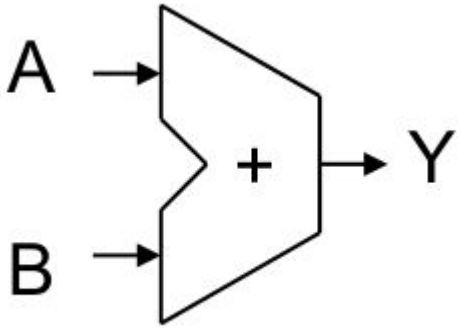


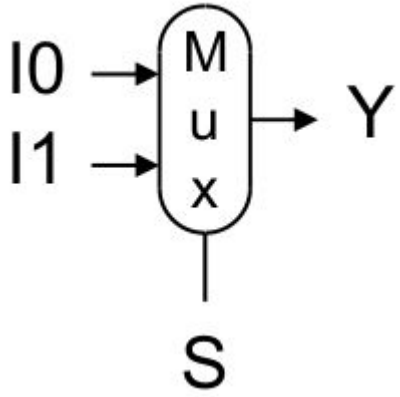
(b) Sequential

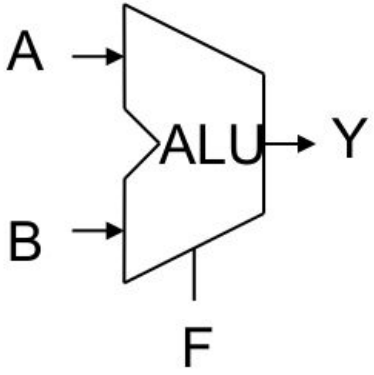
output = *f(in, previousIn)*

Logic Design Conventions: Combinational Elements



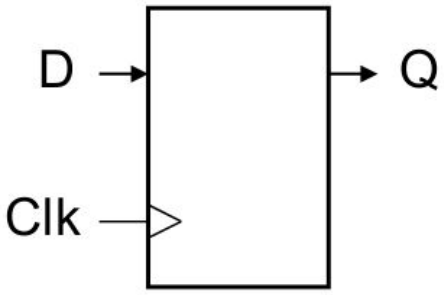


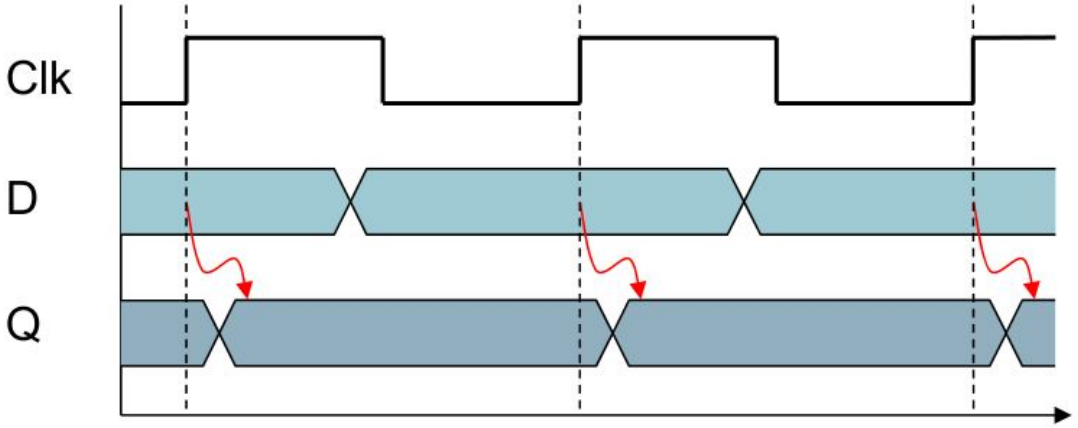




Logic Design Conventions: Sequential Elements

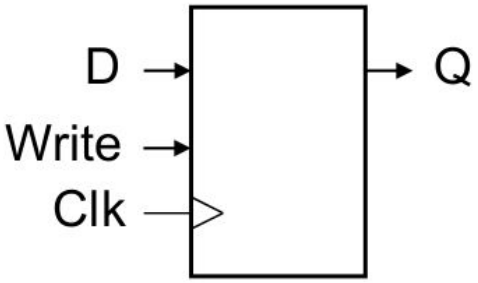
- Register: stores data in a circuit
  - Uses a clock signal to determine when to update the stored value
  - **Edge-triggered:** update when Clk changes from 0 to 1

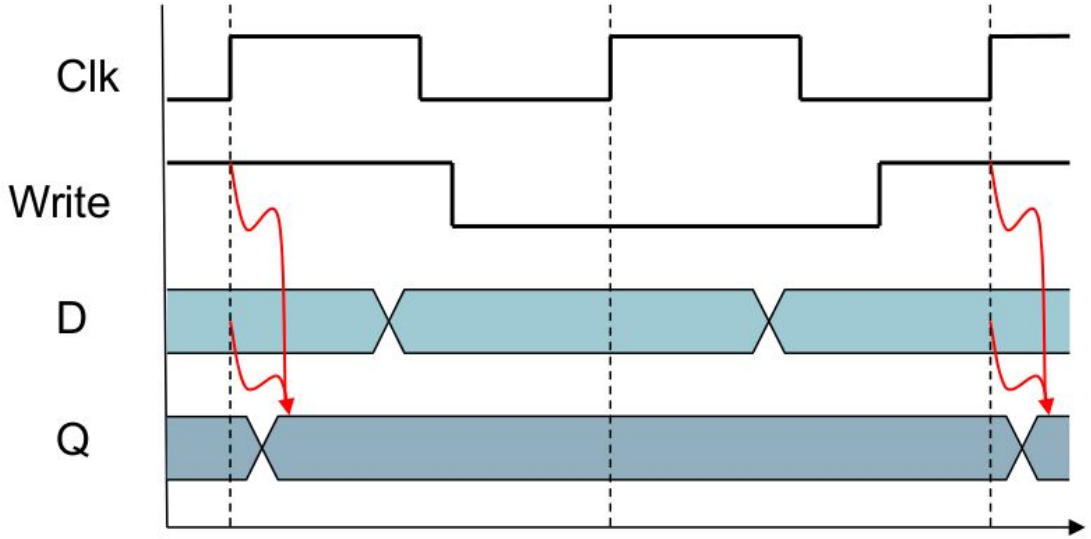




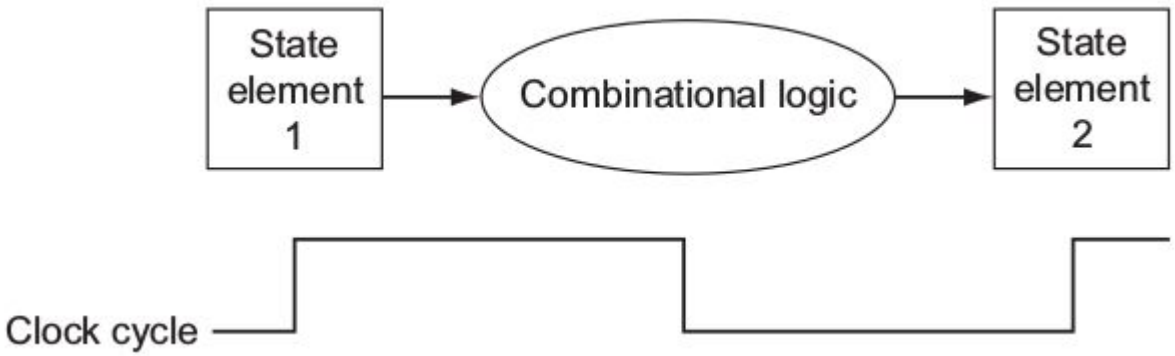
Logic Design Conventions: Sequential Elements

- Register with write control
  - Only updates on clock edge when write control input is 1
  - Used when stored value is required later



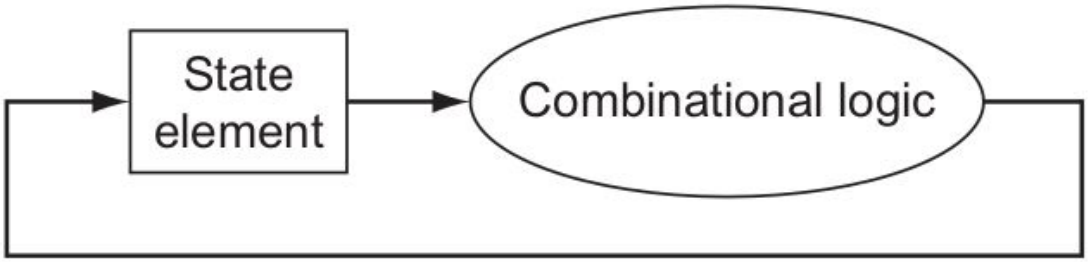


Logic Design Conventions: Clocking Methodology



- Combinational logic transforms data during clock cycles
  - Between clock edges
  - Input from state elements, output to state element
  - **Longest delay determines clock period**

Logic Design Conventions: Edge-triggering and Write and Read



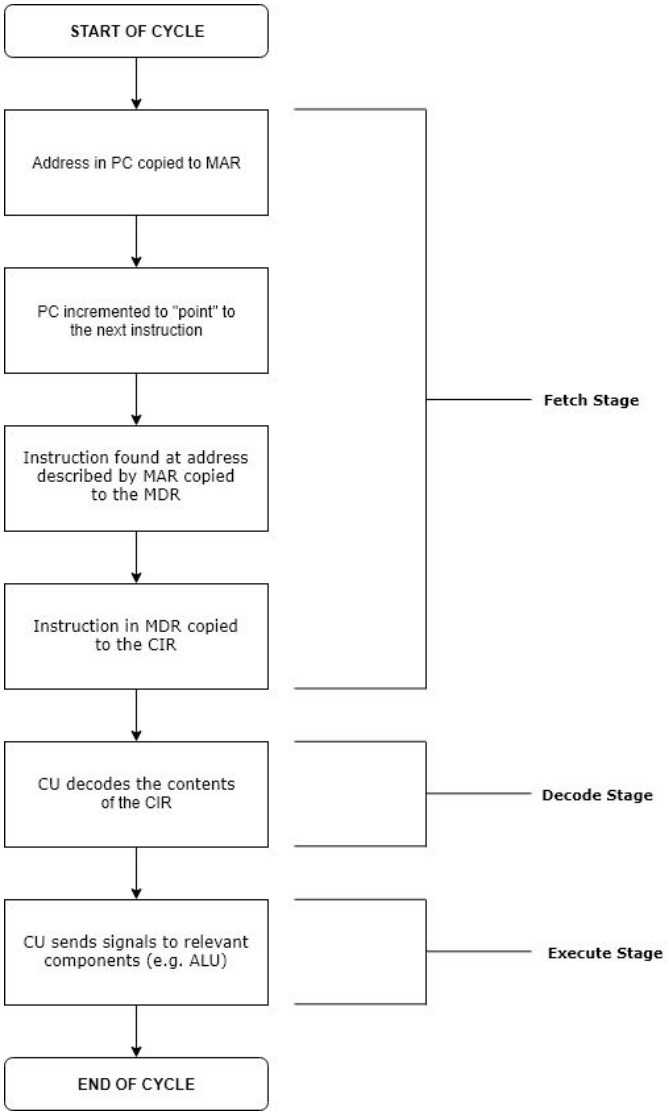
- Edge-triggering allows a **state element to be read and written in the same clock cycle**
- However, the **clock cycle must be long enough** so that the input values are stable when the active clock occurs

PROCESSOR DESIGN:
Instruction Set Architecture

ISA: RISC-V Example

Criteria	Detail	Examples / comments
(1) Classification	• load-store • RISC (Von Neumann)	• CPU activities: fetch, decode, execute
(2) Register set	• 32 integer GPRs x0–x31; x0 is hard-wired 0; separate PC; optional 32 FP regs f0–f31 with F/D extensions; CSRs for control/status	• ABI names: x1=ra, x2=sp, x8=fp/s0, x10–x17=a0–a7 (args/returns), x5–x7/x28–x31 temps, x18–x27 saved.
(3) Data sizes and types	• 8-bit bytes, 16-bit halfwords, 32-bit words, 64-bit doublewords; integers in RV32/64; IEEE-754 single (F) and double (D) precision FP when enabled	Base ISAs: RV32I, RV64I. Vector (V) and other extensions optional.
(4) Addressing modes	• Immediate, register, and base+displacement (load/store)	• Memory is byte-addressed. Default little-endian (most implementations); some environments support bi-endian. Natural alignment recommended; many cores also handle misaligned accesses.
(5) Instruction format	• Fixed 32-bit instructions (primary opcode = 7 bits); formats: R, I, S, B, U, J; optional 16-bit compressed (C)	• egular fields ease decoding/pipelining. C extension improves code density.
(6) Operations	• Loads/stores, integer ALU, shifts, branches/jumps, system/CSR; floating-point ops with F/D; multiply/divide with M	• Loads: `lb/lh/lw/ld`, stores: `sb/sh/sw/sd`. ALU:`add/sub/and/or/xor/slt/sltu/slli/srli/srai`. Branches:`beq/bne/blt/bge/bltu/bgeu`. Jumps: `jal/jalr`. FP: `flw/fld`, `fadd.s/d`, etc.

Von Neumann Machine Cycle

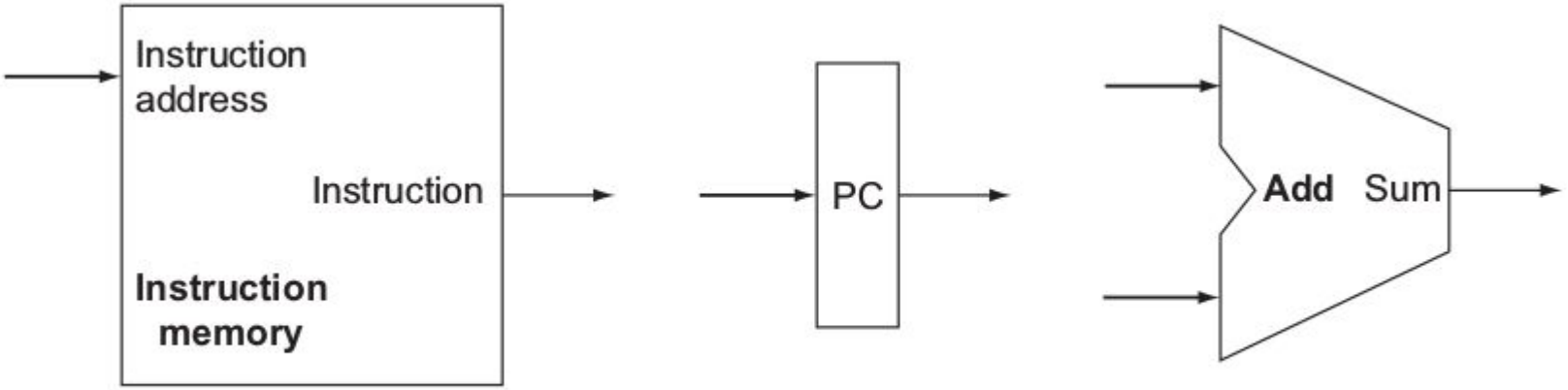


PROCESSOR ARCHITECTURE:
Building a datapath

Overview

- Datapath
  - Elements that process data and addresses
    - Registers, ALUs, mux's, memories, …
- We will build a RISC-V datapath incrementally
    - Refining the overview design

Instruction Fetch: Required Elements (All)



a. Instruction memory b. Program counter c. Adder

To execute any instruction, we must start by **fetching the instruction from memory** at a given starting address. To prepare to execute the next instruction after the current one, we must also **point to the next instruction address.**

To realise this, we need the following components:

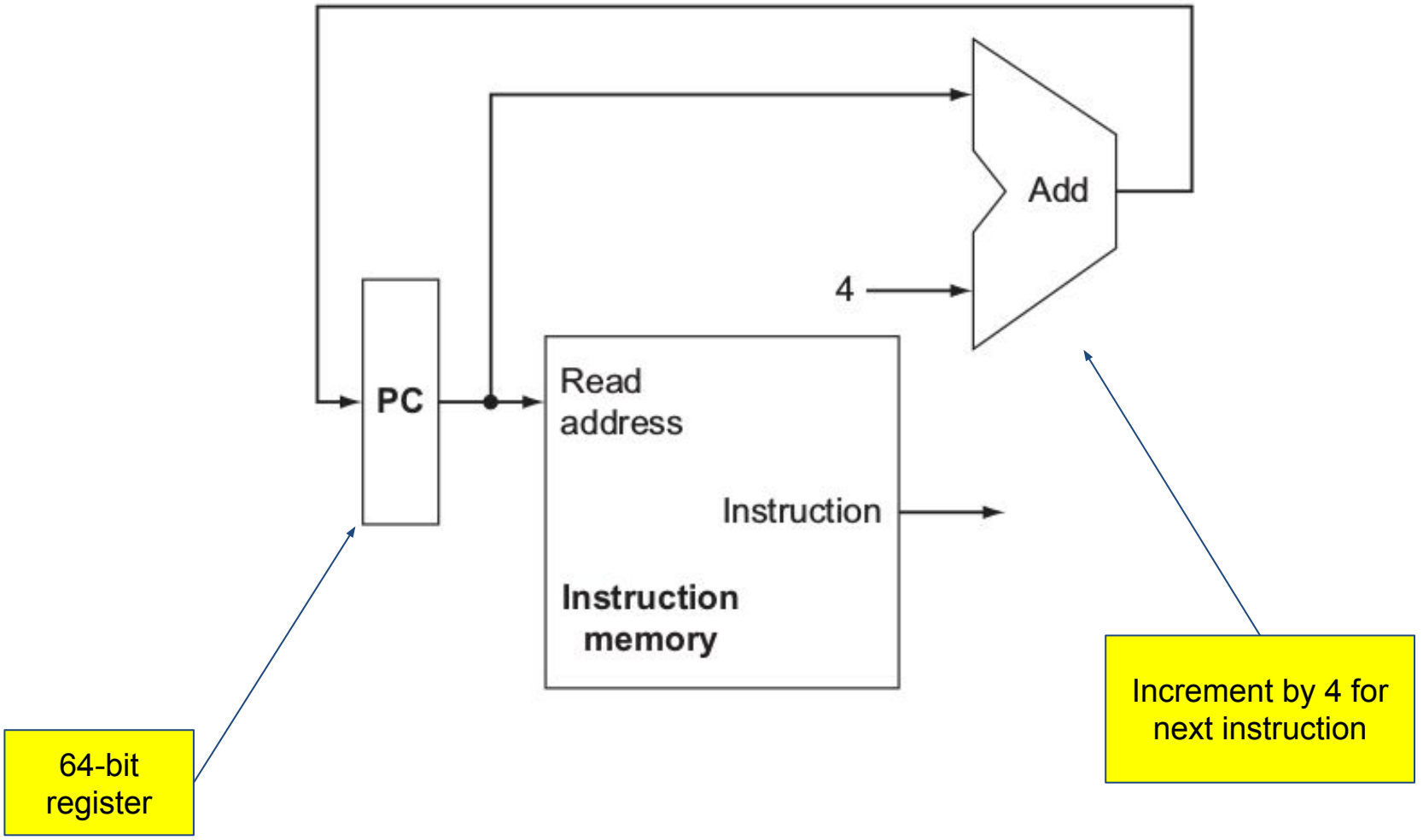
a) **Instruction memory** – to store program instructions and supply individual instructions given an address.

b) **Program counter** – a 64-bits register for holding the address of the current instruction.

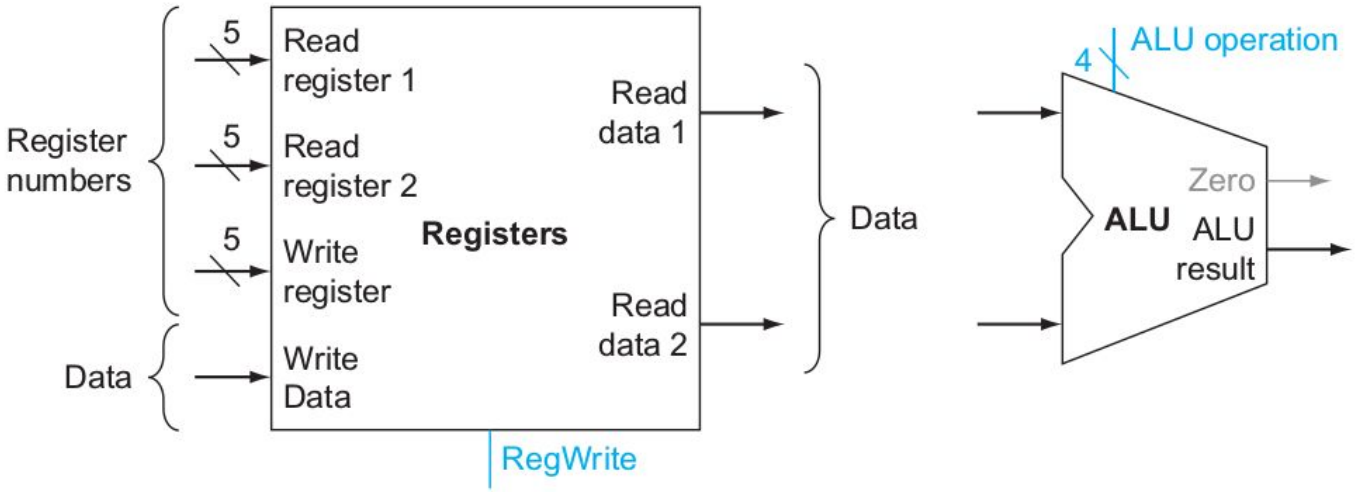
c) **Adder** – to increment the PC to point to the address of the next instruction.

d) **Instruction register** - to store the instruction word that has been fetched from memory.

Instruction Fetch: Datapath (All)



Instruction Execute: Required Elements (R-type)



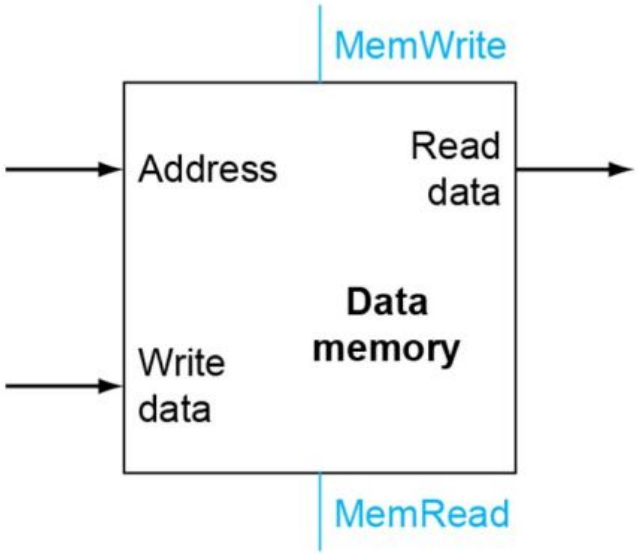
All R-format instructions read two registers, perform an ALU operation on the contents of the registers and write the result to a register. This class of instructions include *add, sub, AND, OR* and *slt*. The processor's 32-bit general-purpose registers (GPRs) are stored in a structure called a register file. This is a collection of registers in which any register can be read or written by specifying it's number.

Therefore to realise the R-format type instructions, we need:

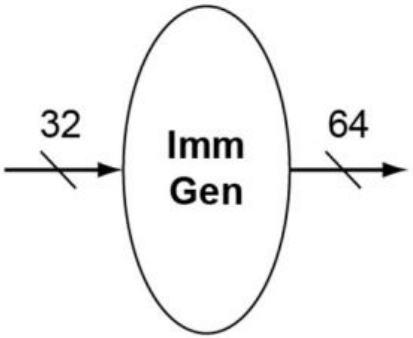
a) **Register file** – for fast temporary storage of operands and result data

b) **ALU** – to carry out the arithmetic and logic operations

Instruction Execute: Required Elements (Load/Store Instructions)



a. Data memory unit



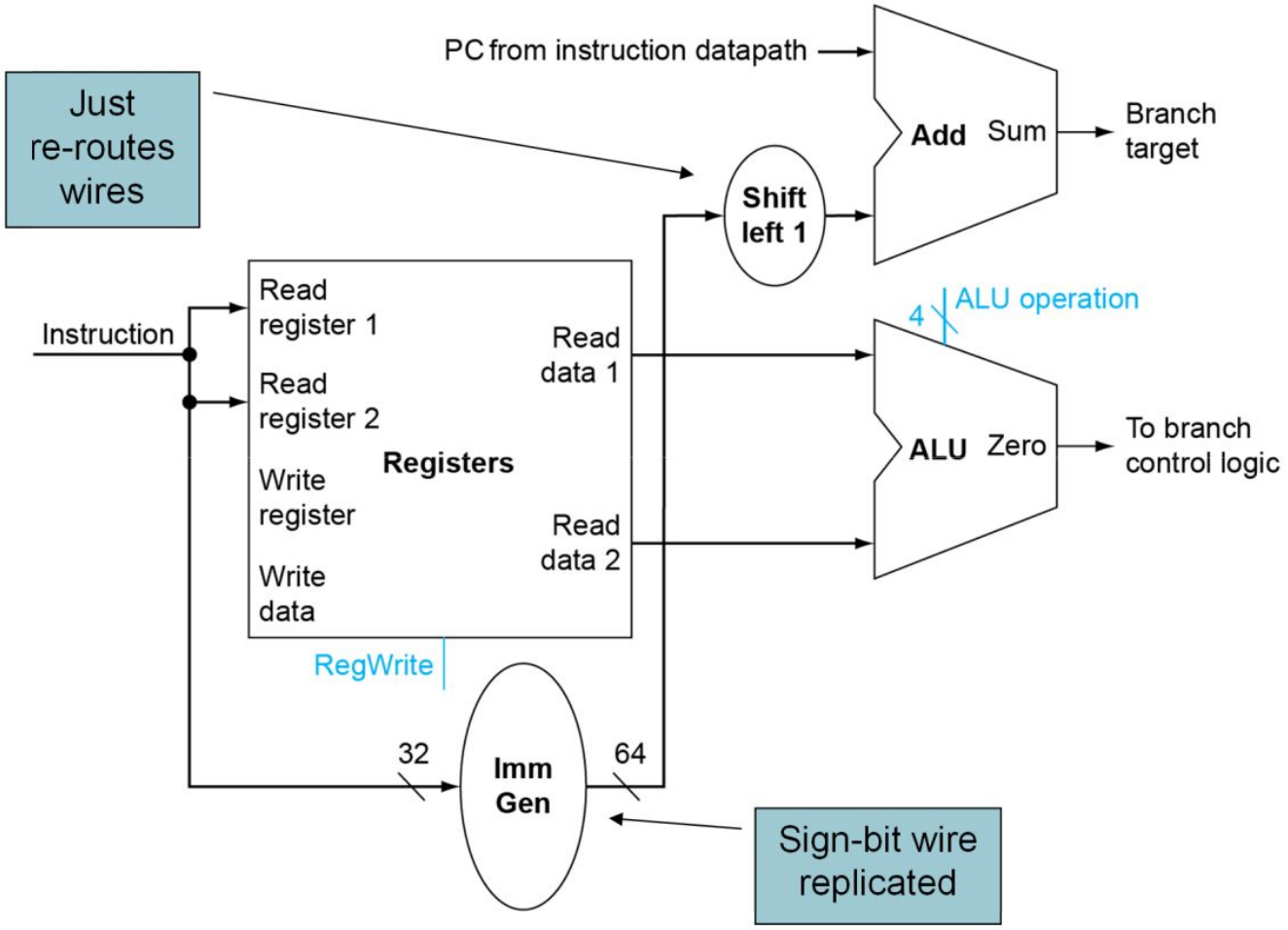
b. Immediate generation unit

- Read register operands
- Calculate address using 12-bit offset
  - Use ALU, but sign-extend offset
- **Load:** Read memory and update register
- **Store:** Write register value to memory

Branch Instructions

- Read register operands
- Compare operands
  - Use ALU, subtract and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Shift left 1 place (halfword displacement)
  - Add to PC value

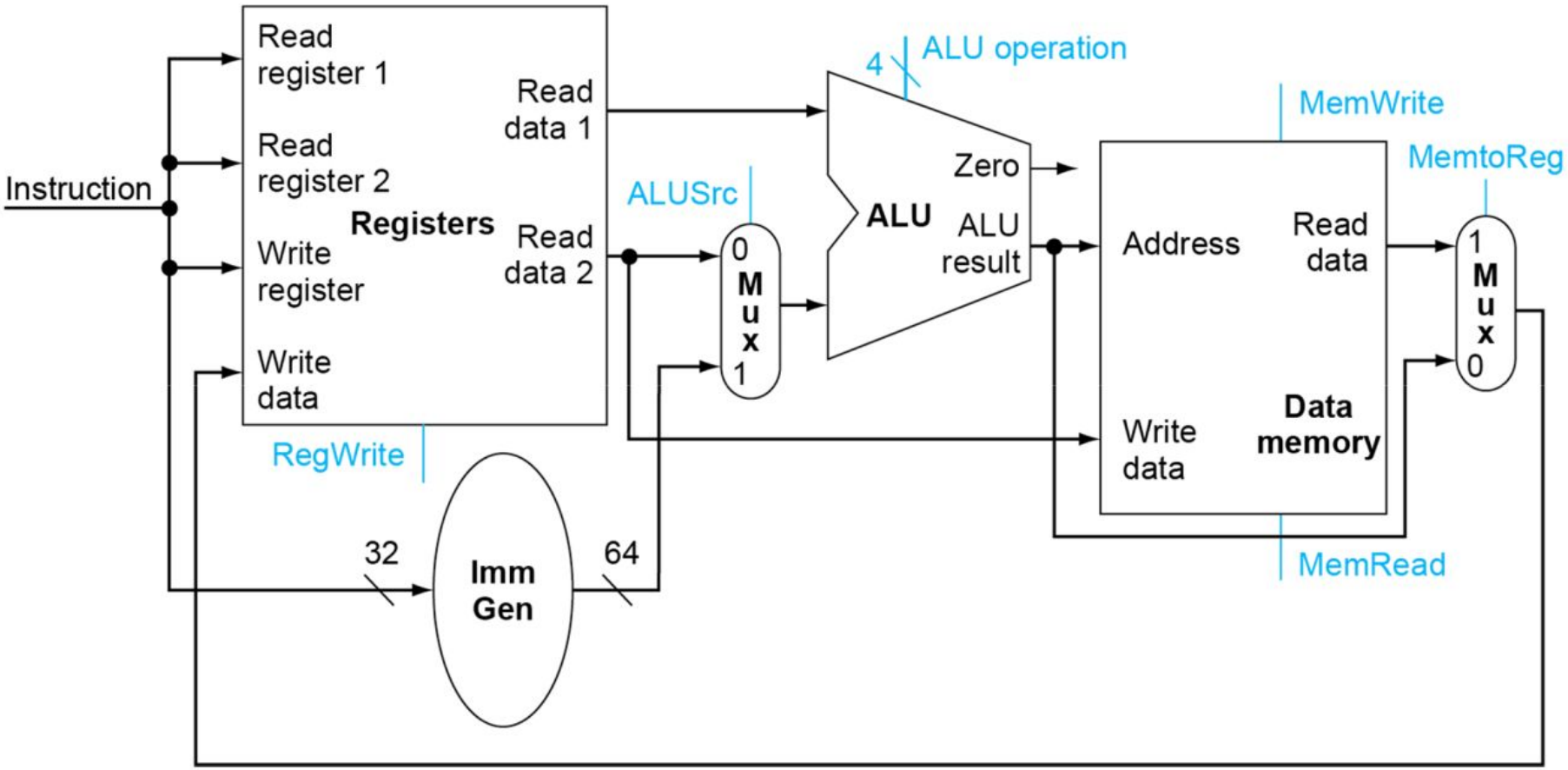
Branch Instructions



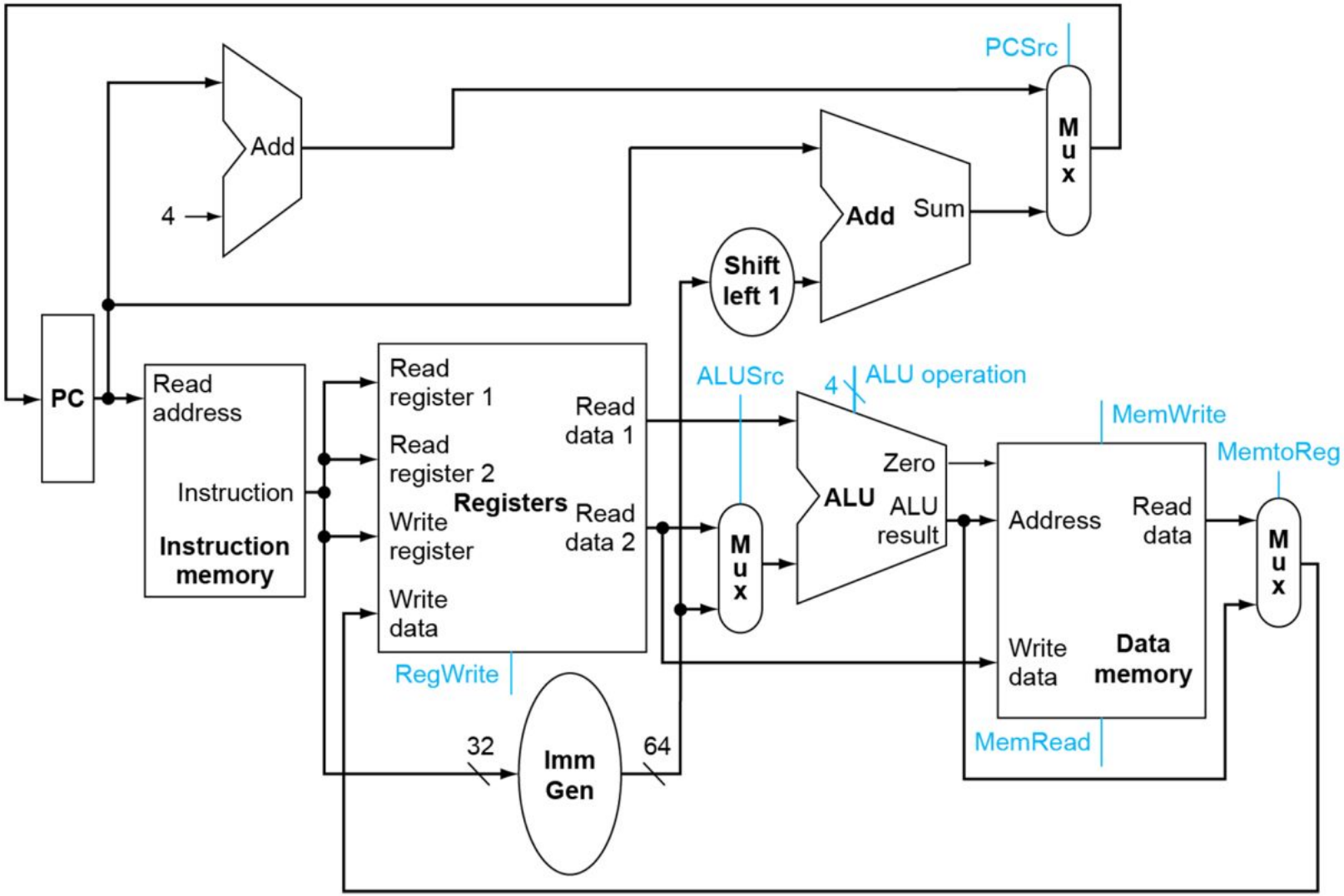
Composing the Elements

- First-cut data path does an instruction in one clock cycle
  - Each datapath element can only do one function at a time
  - Hence, we need separate instruction and data memories
- Use multiplexers where alternate data sources are used for different instructions

Instruction Execute: Datapath (R-type and Load/Store)

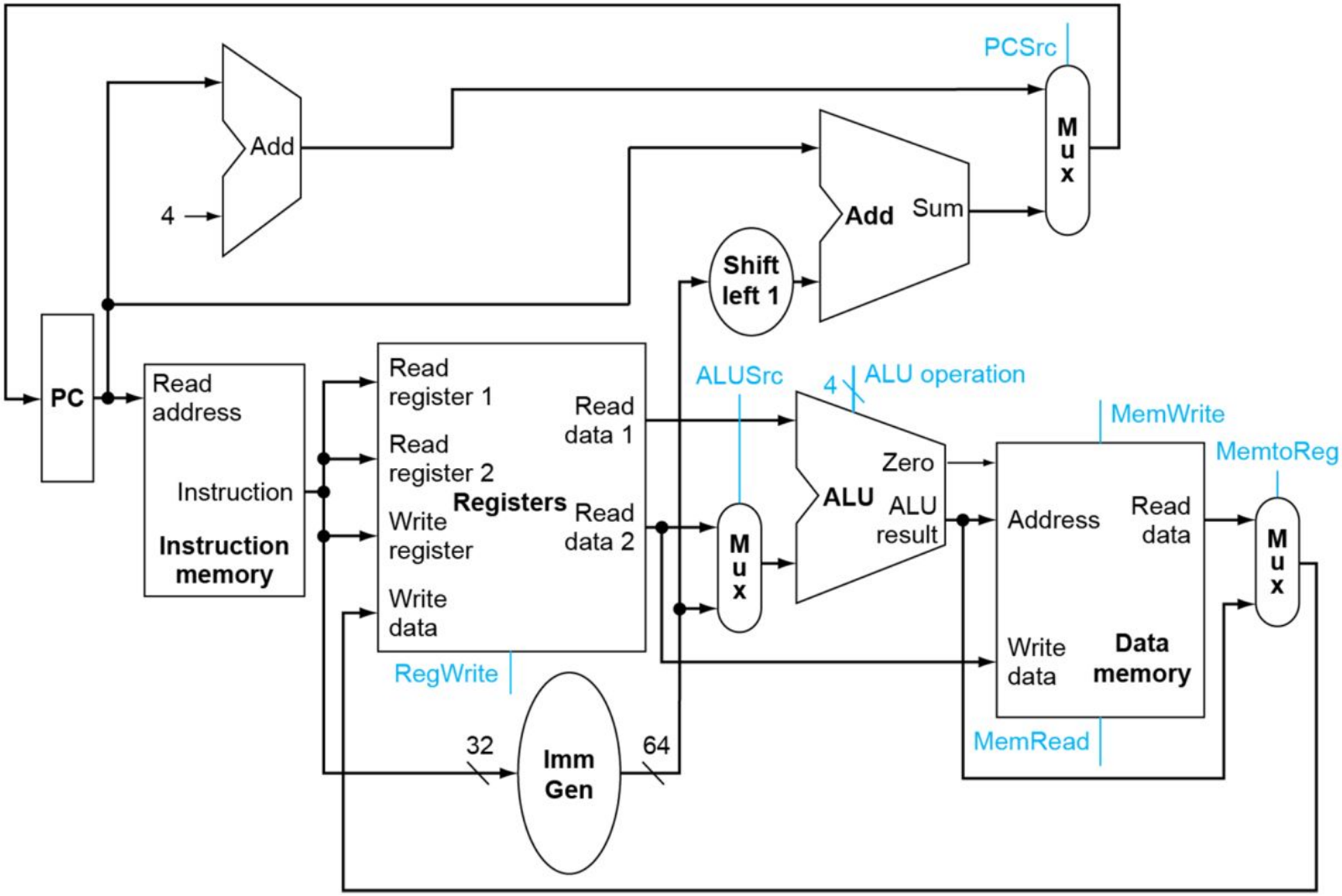


Combined Datapath



PROCESSOR IMPLEMENTATION:
Control Unit

Combined Datapath



Functions of the Control Unit



- The control unit (CU) is a component of a computer's central processing unit (CPU) that directs the operation of the processor.
  - It tells the computer's memory, arithmetic and logic unit and input and output devices how to respond to the instructions that have been sent to the processor.
- It directs the operation of the other units by providing timing and control signals.
  - Most computer resources are managed by the CU.
  - It directs the flow of data between the CPU and the other devices.

The control unit should direct operation of:

(1) Register file
(2) ALU
(3) Multiplexers
(4) Memory (instruction and data)

For all possible machine instructions:

(1) R-format e.g *add, sub, slt, and, or*
(2) I-type e.g *sw, lw, beq*
(3) J-type e.g *j*

Let's modularise it by starting with **ALU control** and then construct the rest of the control afterwards...

ALU Control

- Depending on the instruction class, the ALU performs one of the given operations
  - **Load and store** – ALU used to compute memory address by **addition**
  - **R-type** – ALU used to perform any of the functions (**add, subtract, AND, OR, slt**) depending on the ***funct*** field
  - **Conditional branch** – ALU used to perform **subtraction** to check branch condition

ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract

ALU Control: Inputs and Outputs

- Assume 2-bit ALUOp derived from opcode
  - Combinational logic derives ALU control

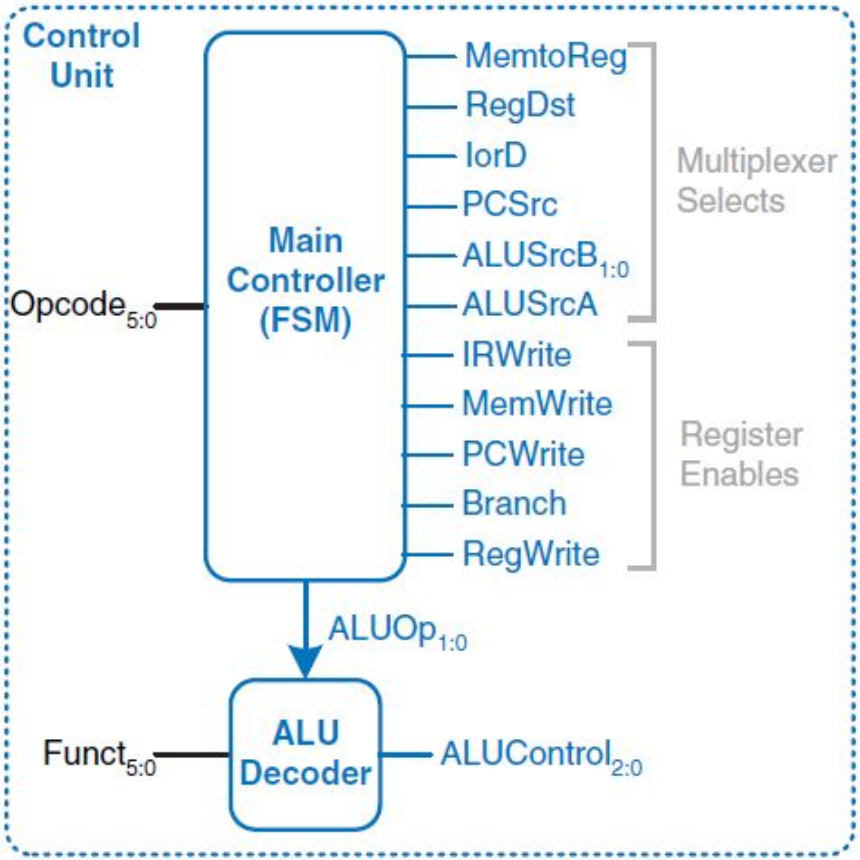
opcode	ALUOp	Operation	Opcode field	ALU function	ALU control
ld	00	load register	XXXXXXXXXXX	add	0010
sd	00	store register	XXXXXXXXXXX	add	0010
beq	01	branch on equal	XXXXXXXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001

Main Control Unit Design: Functions and Structure

The main control unit should direct operation of:

(1) Register file
(2) ALU
(3) Multiplexers
(4) Memory (instruction and data)

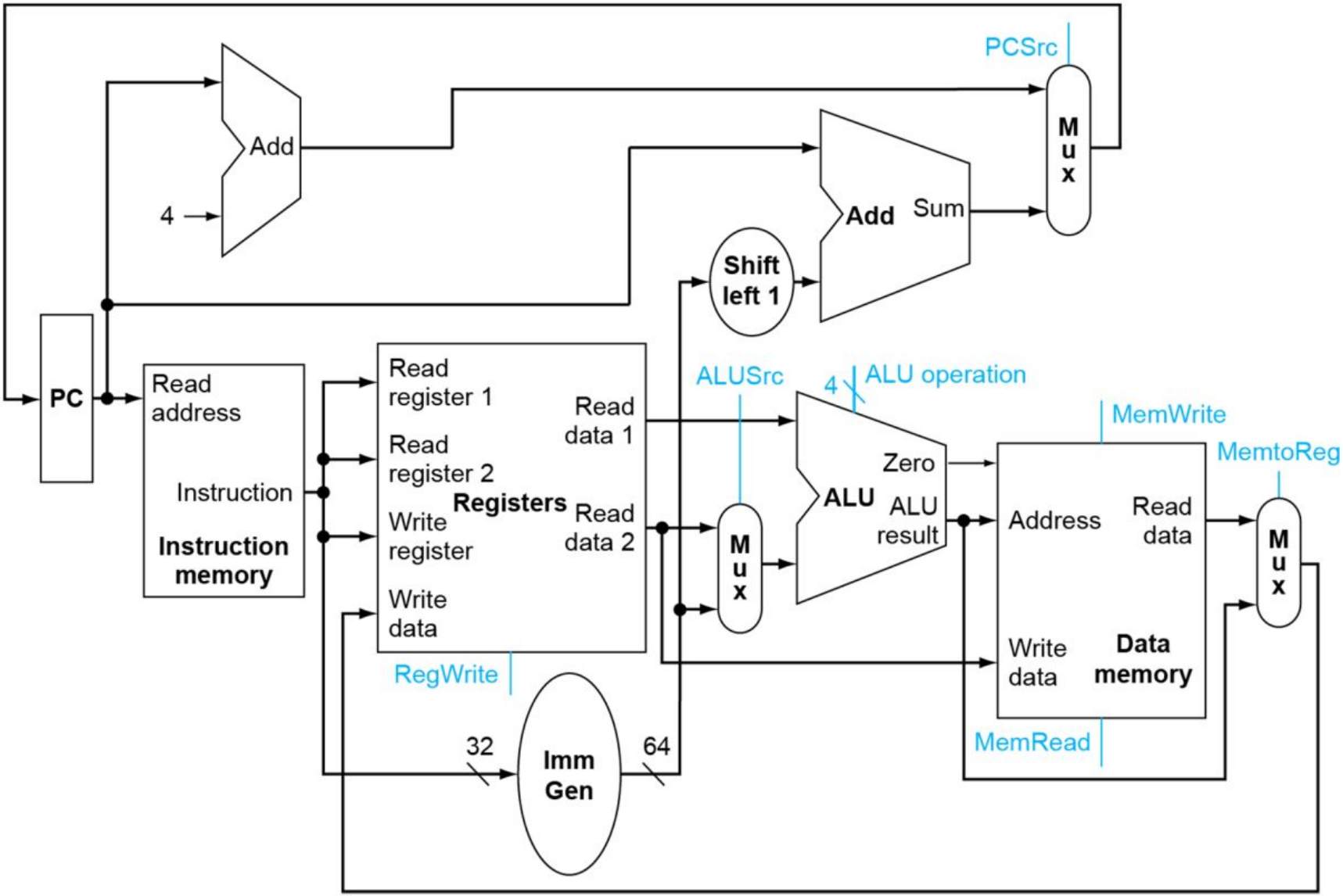
For all possible machine instructions:
(1) R-format e.g *add, sub, slt, and, or*
(2) I-type e.g *lw, addi*
(3) SB-type e.g beq, blt
(4) U-type e.g *lui, auipc*
(5) J-type e.g *jal*



Main Control Unit Design: Instruction Classes

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

Main Control Unit Design: Control Lines



Main Control Unit Design: Control Lines

- Control signals derived from instruction

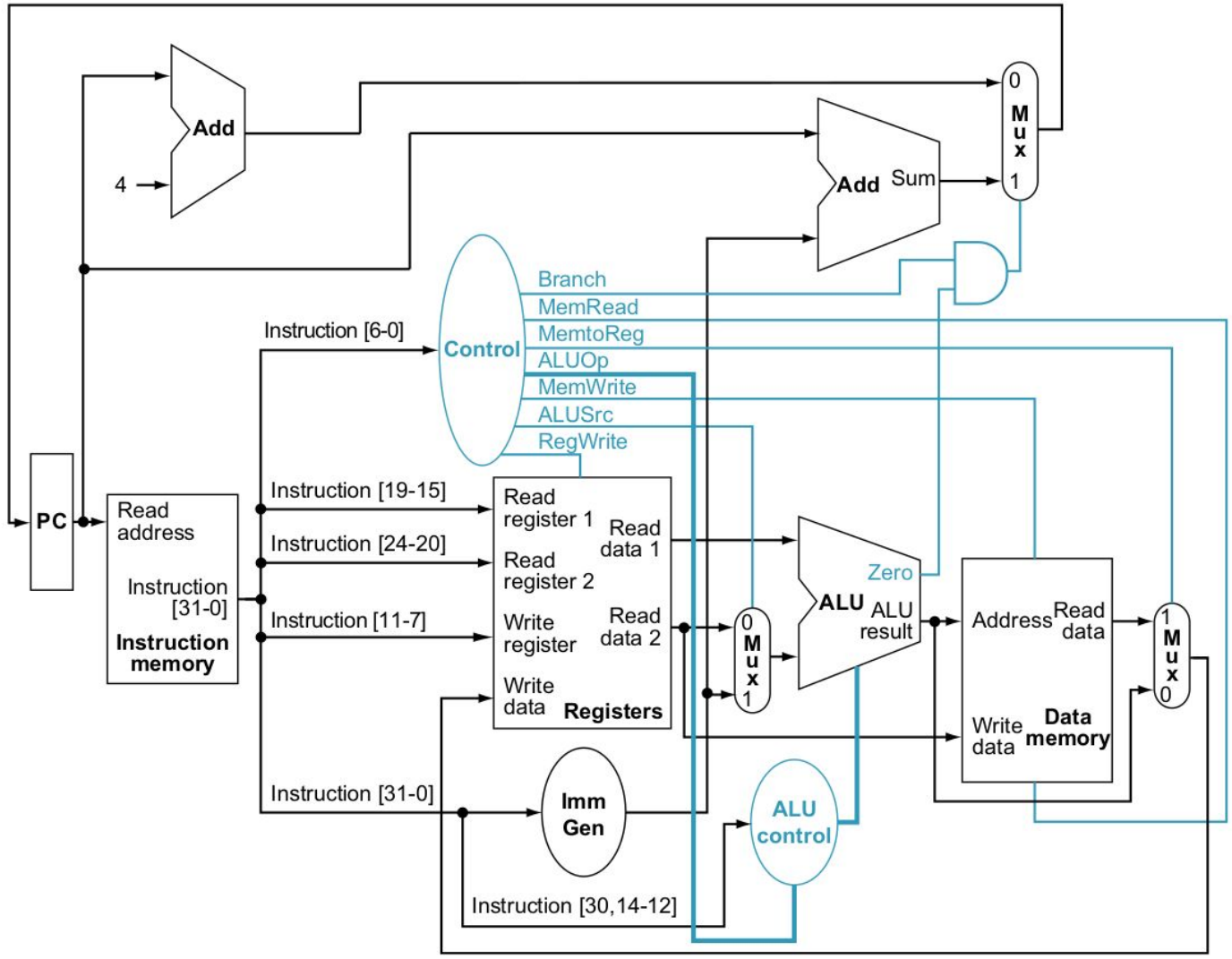
Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

ALUOp		Funct7 field							Funct3 field			
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

Main Control Unit Design: Control Lines

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

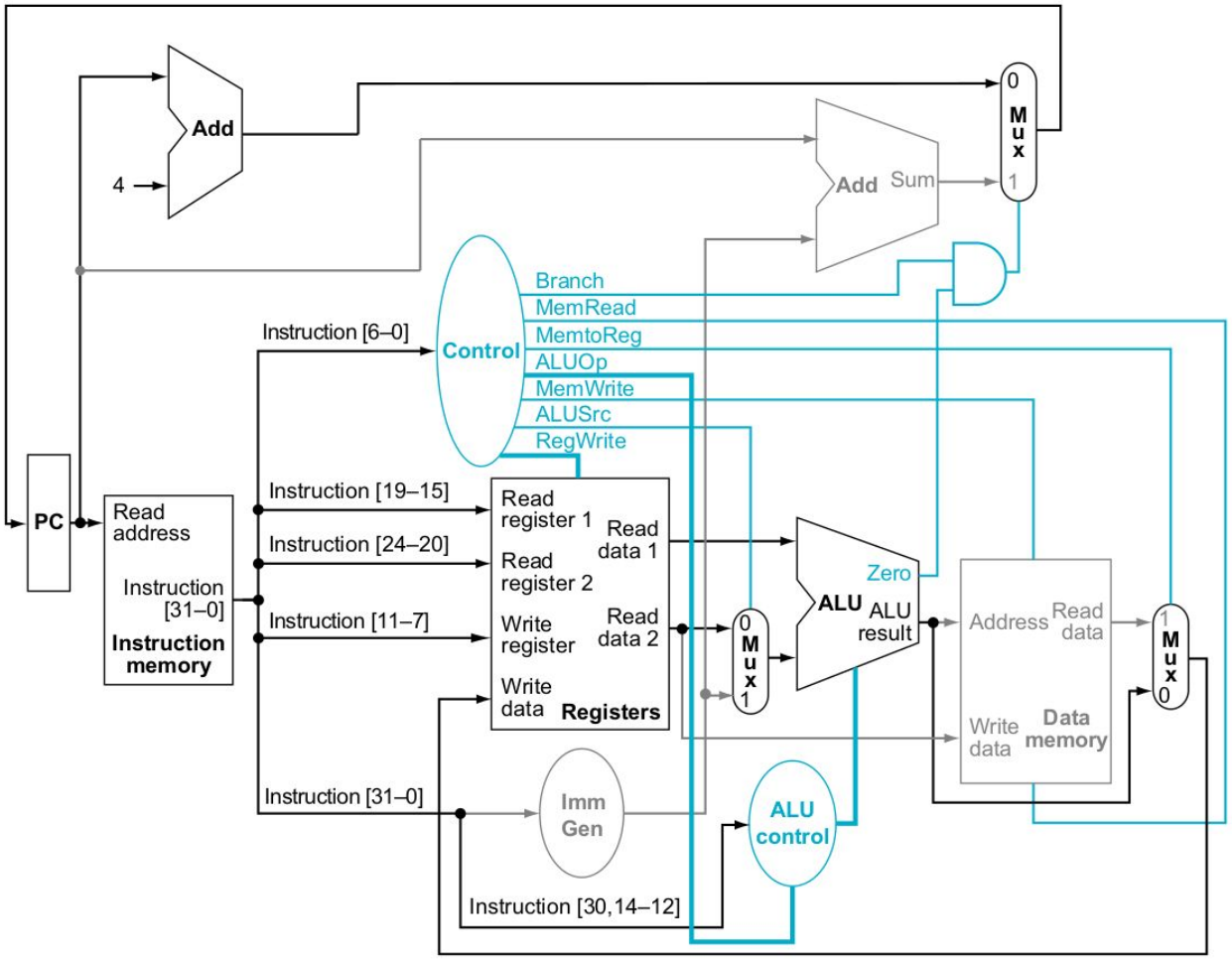
Main Control Unit Design: Control Unit and Control Lines



PROCESSOR IMPLEMENTATION:
Control Design for R-type Instructions

add x1, x2, x3

1) **Fetch** – instruction is fetched and the PC is incremented
2) **Decode** – two registers x2 and x3 are read from the register file; main CU computes the setting of the control lines
3) **Execute** – ALU operates on the data read from the register file, using portions of the opcode to generate appropriate ALU function
4) **Write Back** – result from the ALU is written into the destination register x1 in the register file



PROCESSOR IMPLEMENTATION:
Control Design for Memory Instructions

lw x1, offset(x2)

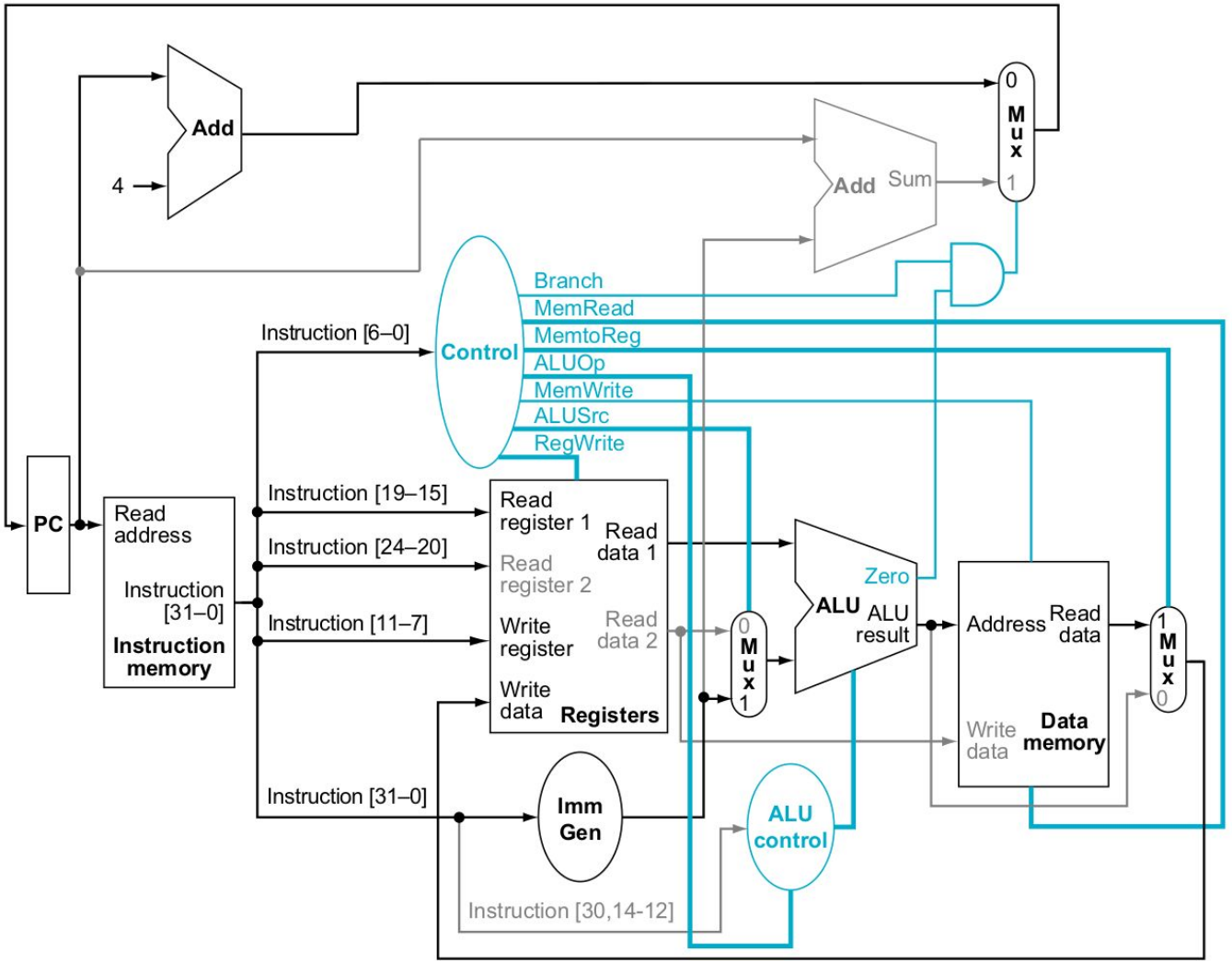
1) **Fetch** – instruction is fetched and the PC is incremented

2) **Decode** – a register x2 is read from the register file; main CU computes the setting of the control lines

3) **Execute** – ALU computes the sum of the value read from the register file and the sign-extended 12 bits of the instruction

4) **Memory Access** - The sums is used as the address for reading data from the data memory

5) **Write Back** – data from the memory unit is written into x1 in the register file



PROCESSOR IMPLEMENTATION:
Control Design for Branch Instructions

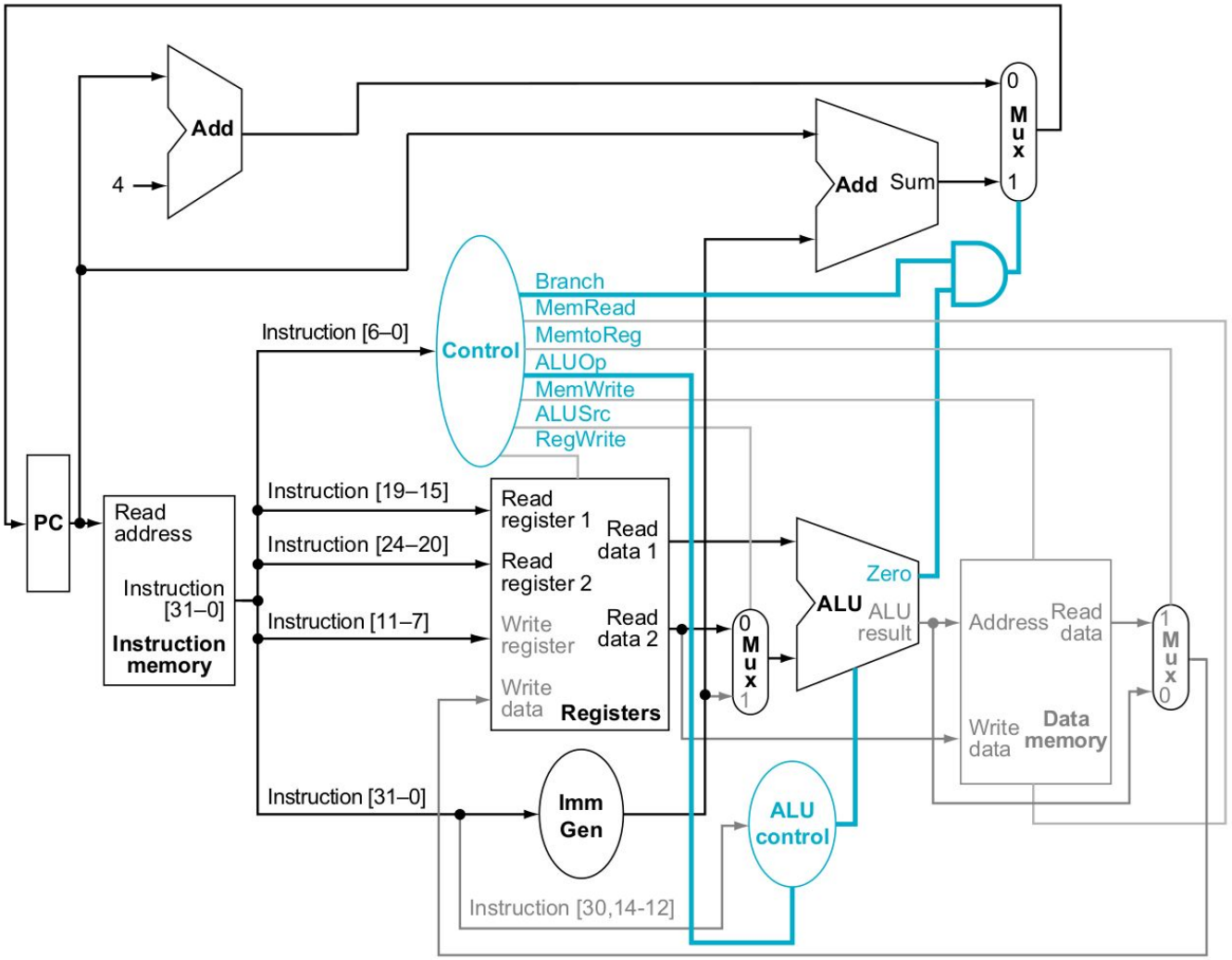
beq x1, x2, offset

1) **Fetch** – instruction is fetched and the PC is incremented

2) **Decode** – two registers x1 and x2 are read from the register file; main CU computes the setting of the control lines

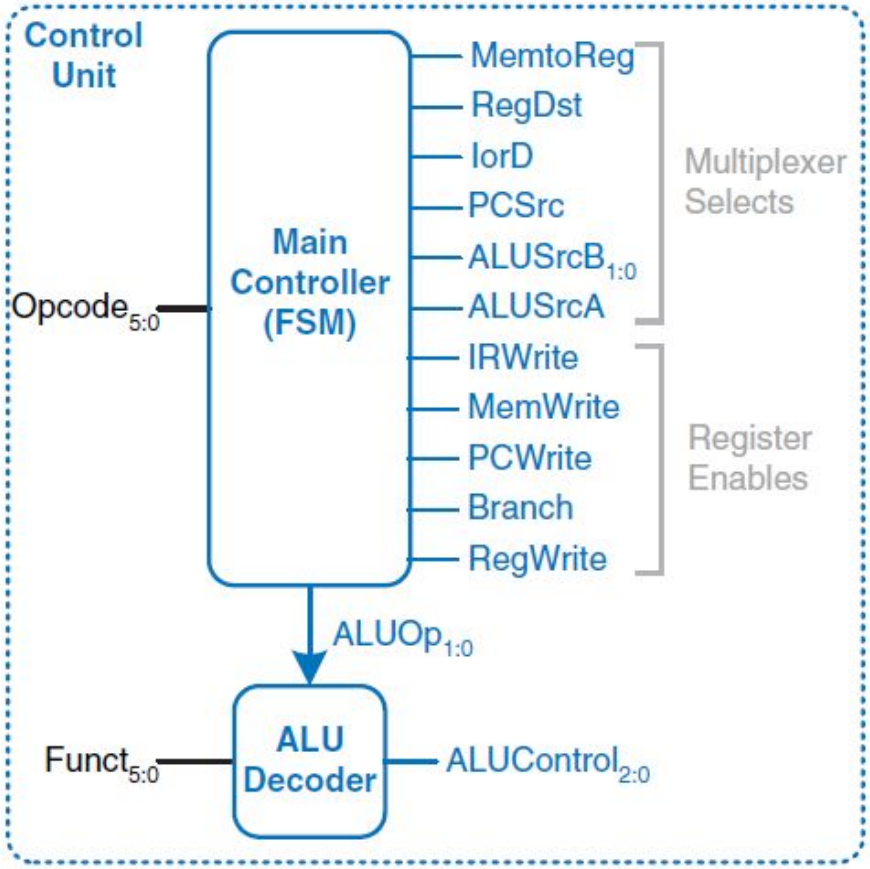
3) **Execute** – ALU computes the difference of the values read from the register file. The value of PC is added to the sign-extended, 12 bits of the instruction (offset) left shifted by one; the result is the branch target address

4) **Write Back** – zero result from the ALU is used to decide which adder result to store into the PC



PROCESSOR IMPLEMENTATION:
Finalising Control Design

Main Control Unit Design: Structure



Main Control Unit Design: Truth Table

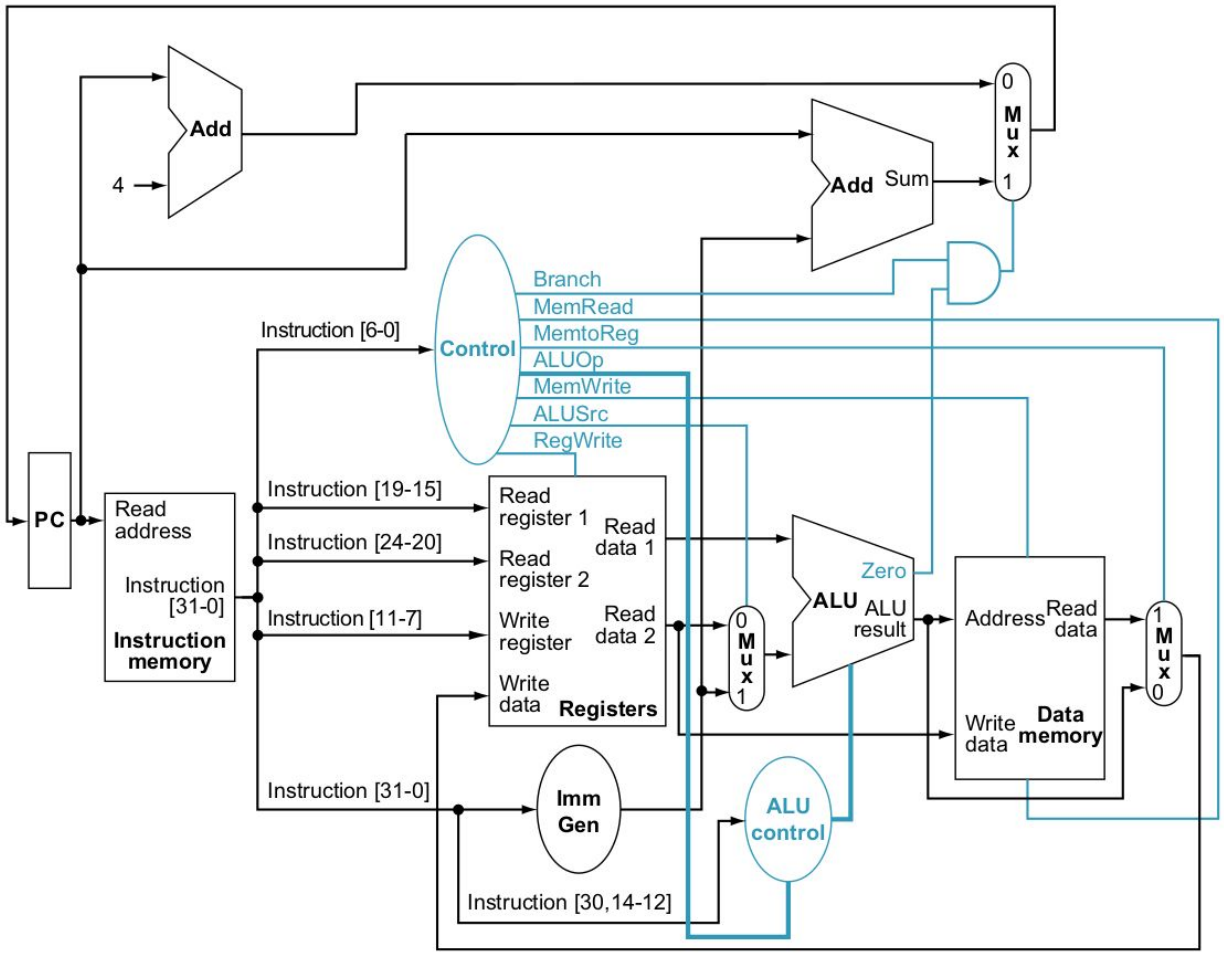
Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Performance Analysis: Single Cycle Implementation



- Each instruction in the single-cycle processor takes one clock cycle, so the CPI is 1
- The processor clock cycle is determined by the instruction with the worst critical path
- The critical path for the lw instruction is as follows
  - PC loads new address on the active clock edge
  - The instruction memory reads the next instruction
  - The register file reads the operand register. While the register file is reading, the immediate offset field is sign-extended and selected at the multiplexer to determine second ALU input.
  - The ALU adds the operands to find the effective read address
  - The data memory reads from this address
  - The MemtoReg register selects ReadData
  - Result must be setup at the register file before the next active clock edge, so that it can be properly written

Performance Analysis: Single Cycle Implementation



$$T_c = t_{pcq_PC} + t_{mem} + \max[t_{RFread}, t_{igen} + t_{mux}] + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

$$T_c = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup}$$

Exercise: Performance Analysis



Q: Bohlale is planning to build a 32-bit single-cycle RISC-V processor in a 65 nm CMOS technology process. She has found that the logic elements that form part of the proposed CPU datapath have the performance delays given in the table below.

Kindly assist Bohlale in computing the execution time for a program with 10 million instructions.

Element	Parameter	Delay (ps)
register clk-to-Q	t_{pcq}	30
register setup	t_{setup}	20
multiplexer	t_{mux}	25
ALU	t_{ALU}	200
memory read	t_{mem}	250
register file read	t_{RFread}	150
register file setup	$t_{RFsetup}$	20

Exercise: Performance Analysis (Solution)



Q: Bohlale is planning to build a 32-bit single-cycle RISC-V processor in a 65 nm CMOS technology process. She has found that the logic elements that form part of the proposed CPU datapath have the performance delays given in the table below.

Kindly assist Bohlale in computing the execution time for a program with 10 million instructions.

Element	Parameter	Delay (ps)
register clk-to-Q	t_{pcq}	30
register setup	t_{setup}	20
multiplexer	t_{mux}	25
ALU	t_{ALU}	200
memory read	t_{mem}	250
register file read	t_{RFread}	150
register file setup	$t_{RFsetup}$	20

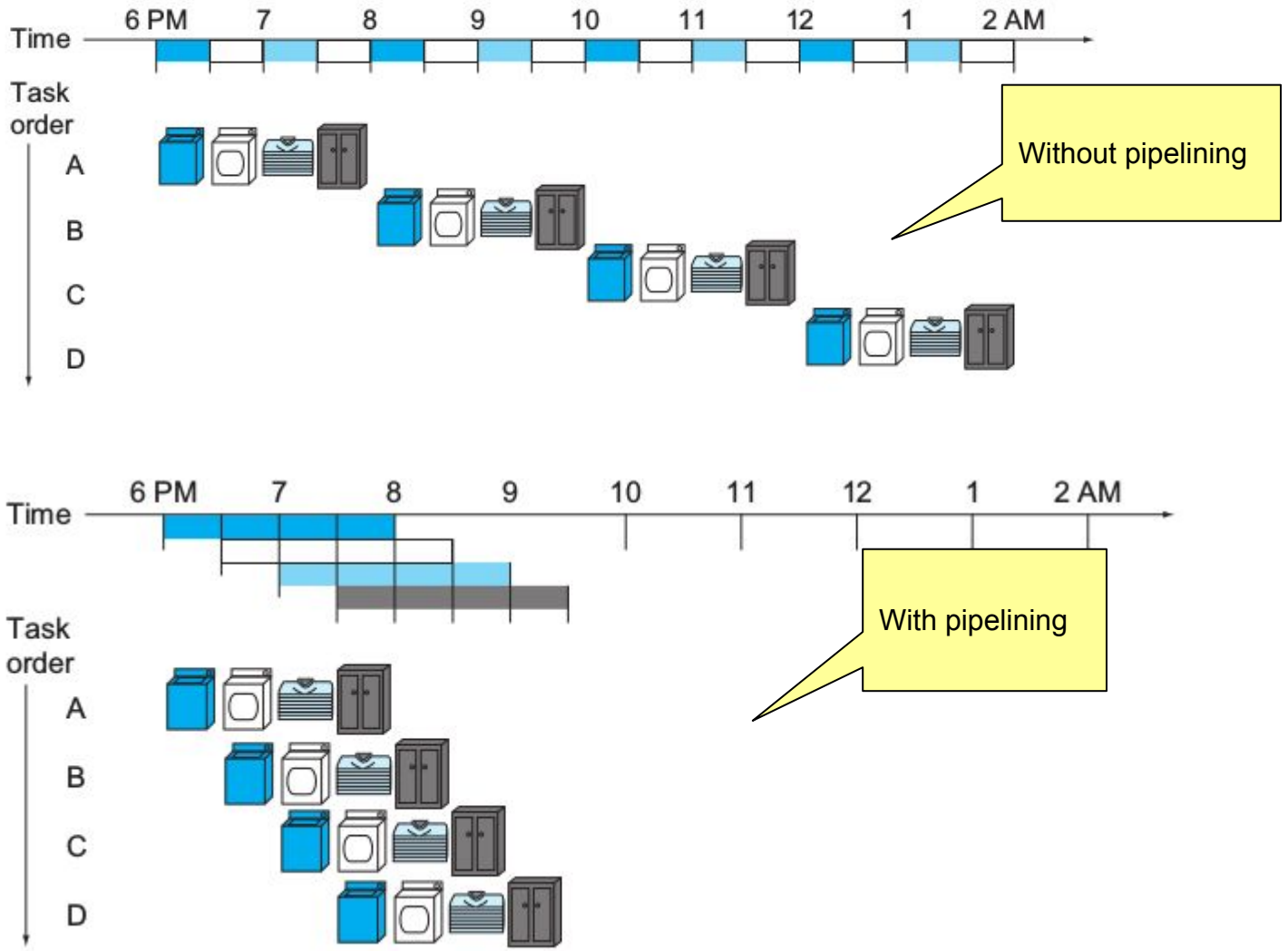
1. The execution time is given by the CPU equation: T = IC x CPI x CCT
2. CPI =1 , since this is a single-cycle processor.
3. IC = 10 million, given.
4. We need to determine the CCT of this processor given the datapath performance delays. This is given by:
   $CCT = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup}$
   $= 30 + 2x250 + 150 + 200 + 25 + 20 = 925ps$
5. Therefore, $T = 10 \times 10^6 \times 1 \times 925 \times 10^{-12}$ = **92.5sec**

Performance Issues

- Longest delay determines clock period
  - **Critical path:** load instruction
  - Instruction memory ⟶ register file ⟶ ALU ⟶ data memory ⟶ register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by pipelining

PROCESSOR IMPLEMENTATION:
Pipelining

Pipelining Overview and Motivation



RISC-V Pipeline

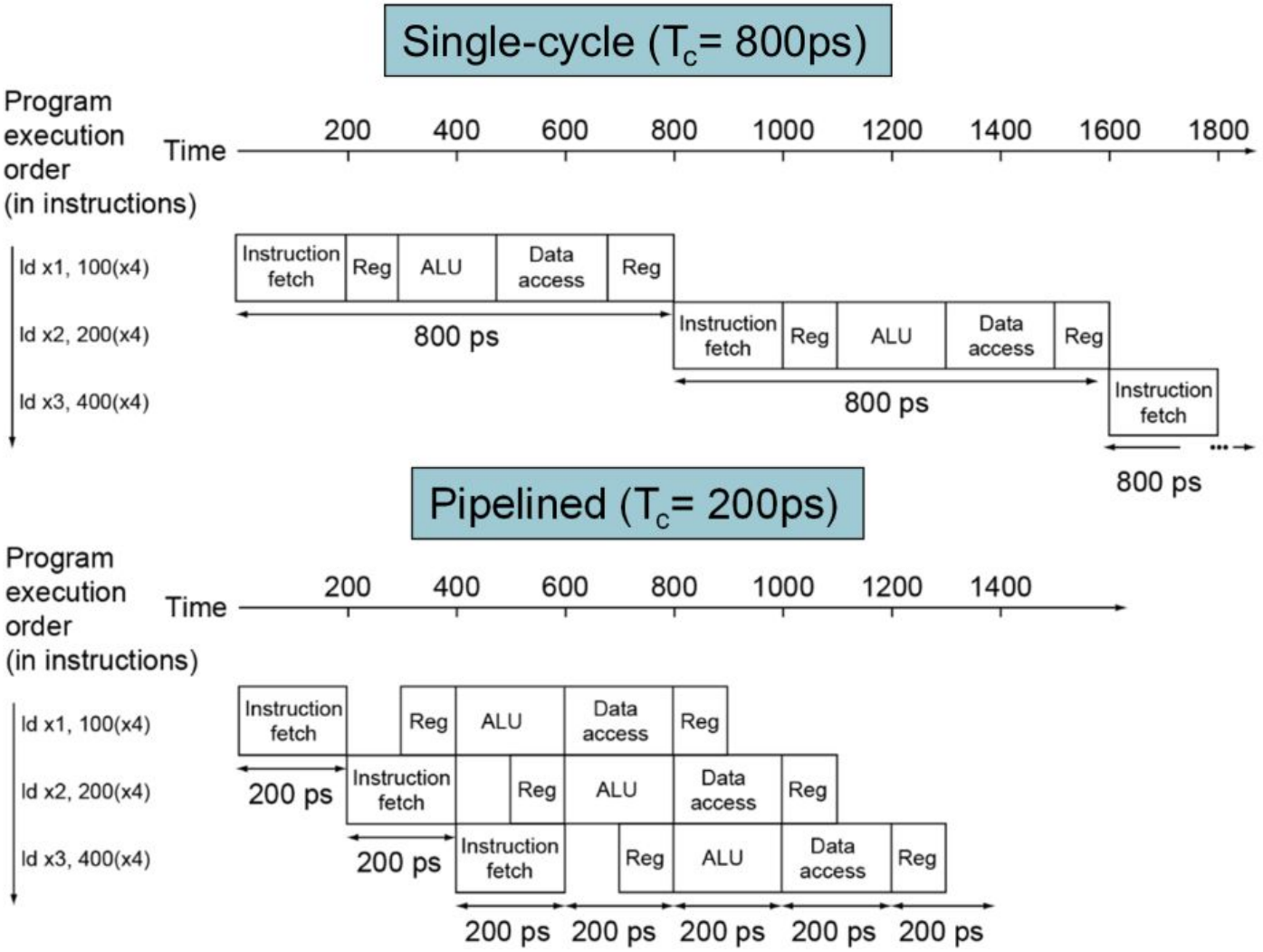
- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

Exercise: Pipelined Performance

Question: Compare the average time between instructions of a single-cycle implementation, in which all instructions take one clock cycle, to a pipelined implementation.

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
ld	200ps	100 ps	200ps	200ps	100 ps	800ps
sd	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Exercise: Pipelined Performance



Pipeline Speedup

- If all stages are balanced
  - i.e., all take the same time
  - $\text{Time between instructions}_{\text{pipelined}} = \dfrac{\text{Time between instructions}_{\text{nonpipeline}}}{\text{Number of stages}}$
- If not balanced, speedup is less
- Speedup due to increased throughput
  - Latency (time for each instruction) does not decrease

Pipelining and ISA Design

- RISC-V ISA designed for pipelining
  - All instructions are 32-bits
  - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to 17-byte instructions
  - Few and regular instruction formats
    - Can decode and read registers in one step
  - Load/store addressing
    - Can calculate address in 3rd stage, access memory in 4th stage

Hazards

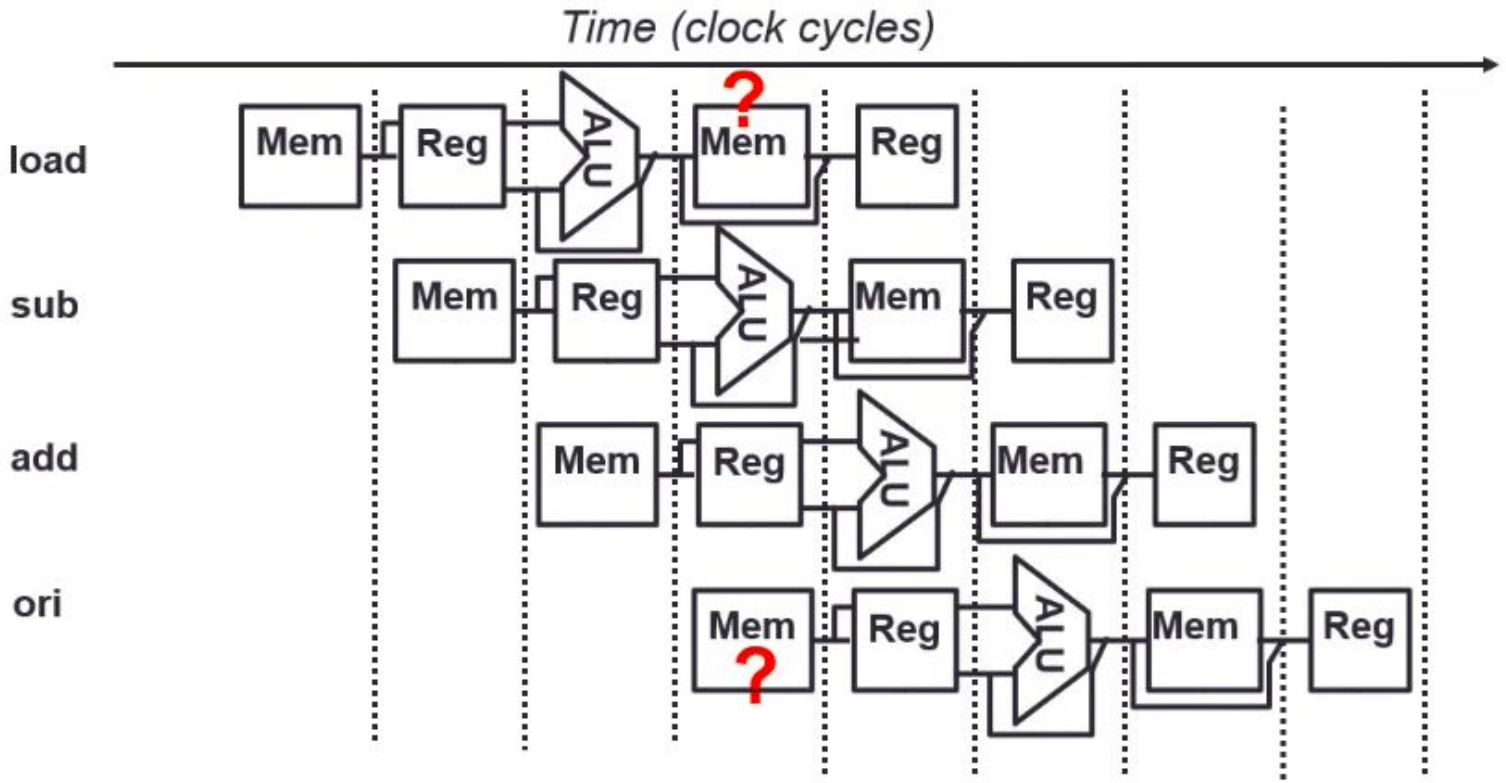
- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
  - A required resource is busy
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction

Structural Hazard

- Conflict for use of a resource
- In RISC-V pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to stall for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

Structural Hazard Example

- Assume single memory implementation

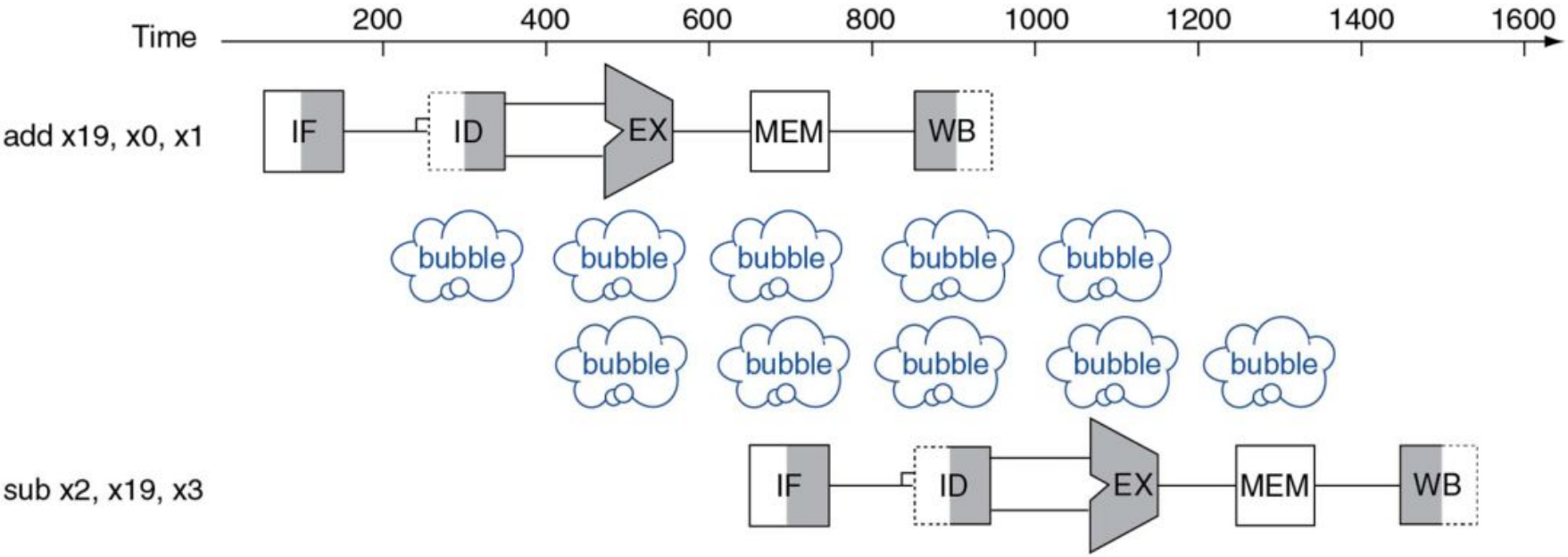


Data Hazard

- An instruction depends on completion of data access by a previous instruction
- Example

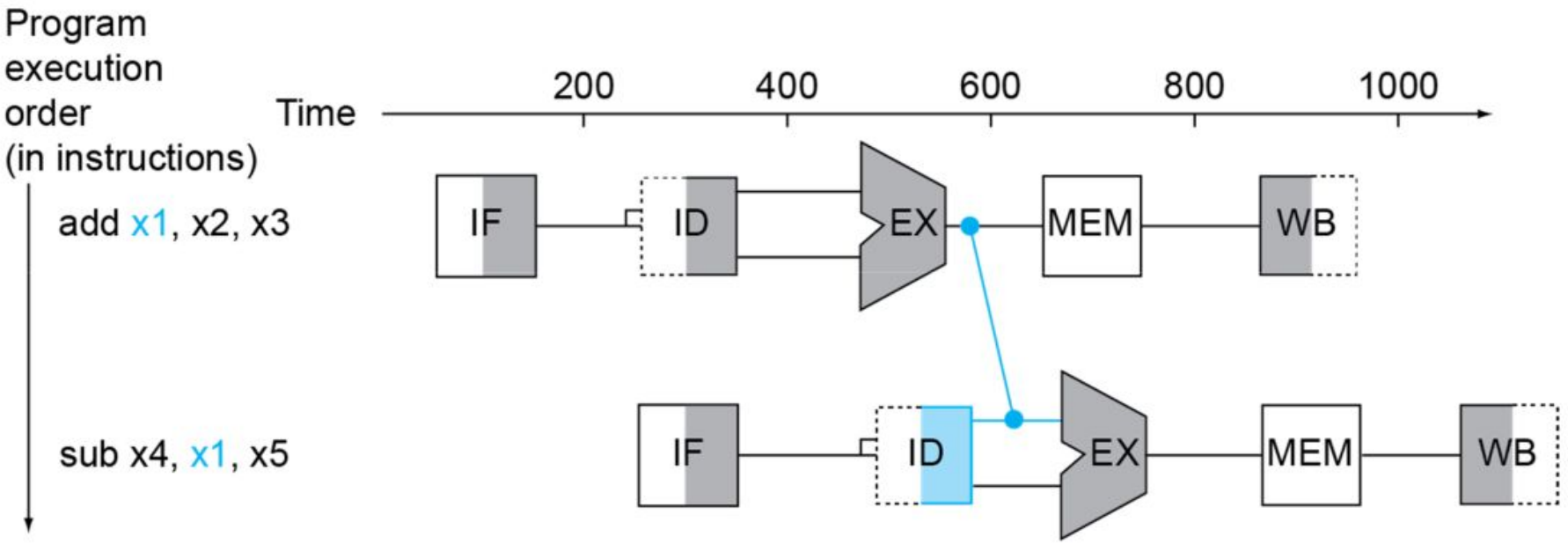
  add x19, x0, x1

  sub x2, x19, x3



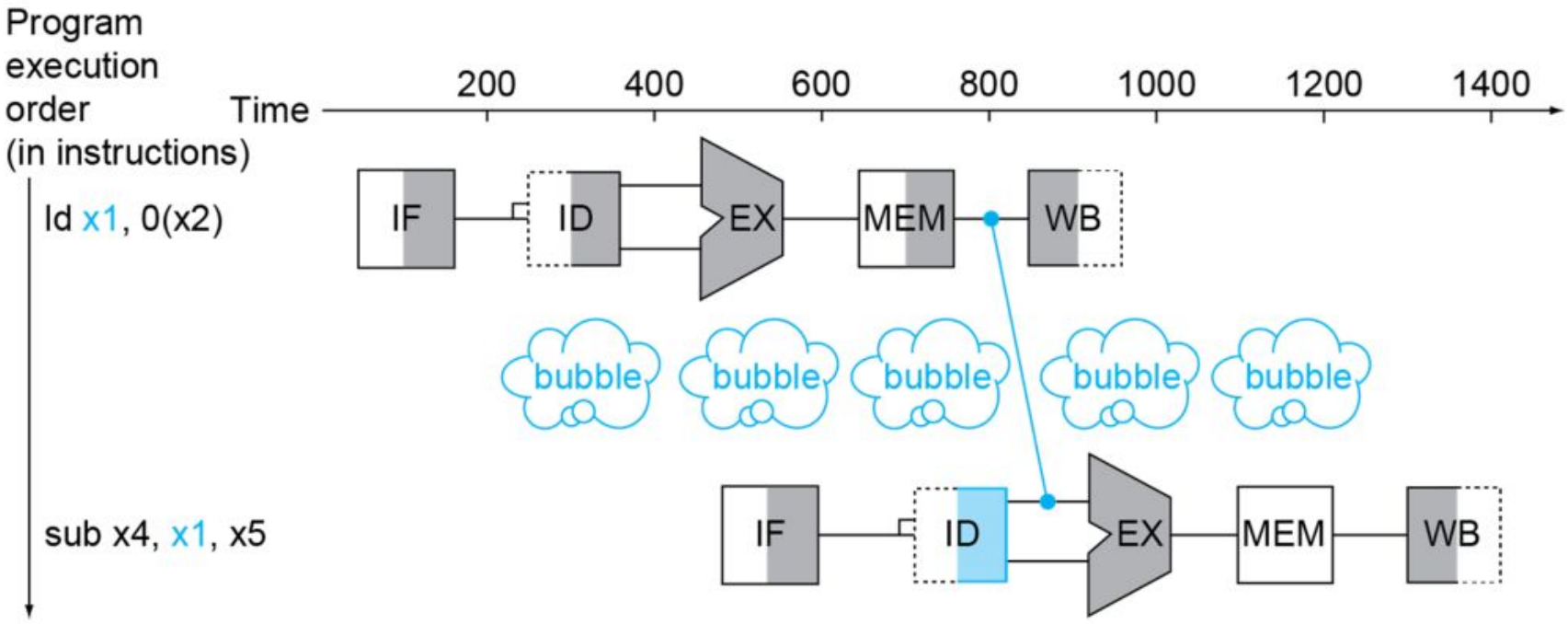
Forwarding (aka Bypassing)

- Use result when it is computed
  - Don't wait for it to be stored in a register
  - Requires extra connections in the datapath



Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!



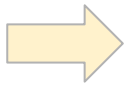
Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code example

```
a = b + e;
c = b + f;
```

- C code example
- Generated RISC-V code assuming all variables in memory and base address in x31

```
lw x1, 0(x31)
lw x2, 8(x31)
add x3, x1, x2
sw x3, 24(x31)
ld x4, 16(x31)
add x5, x1, x4
sw x5, 32(x31)
```



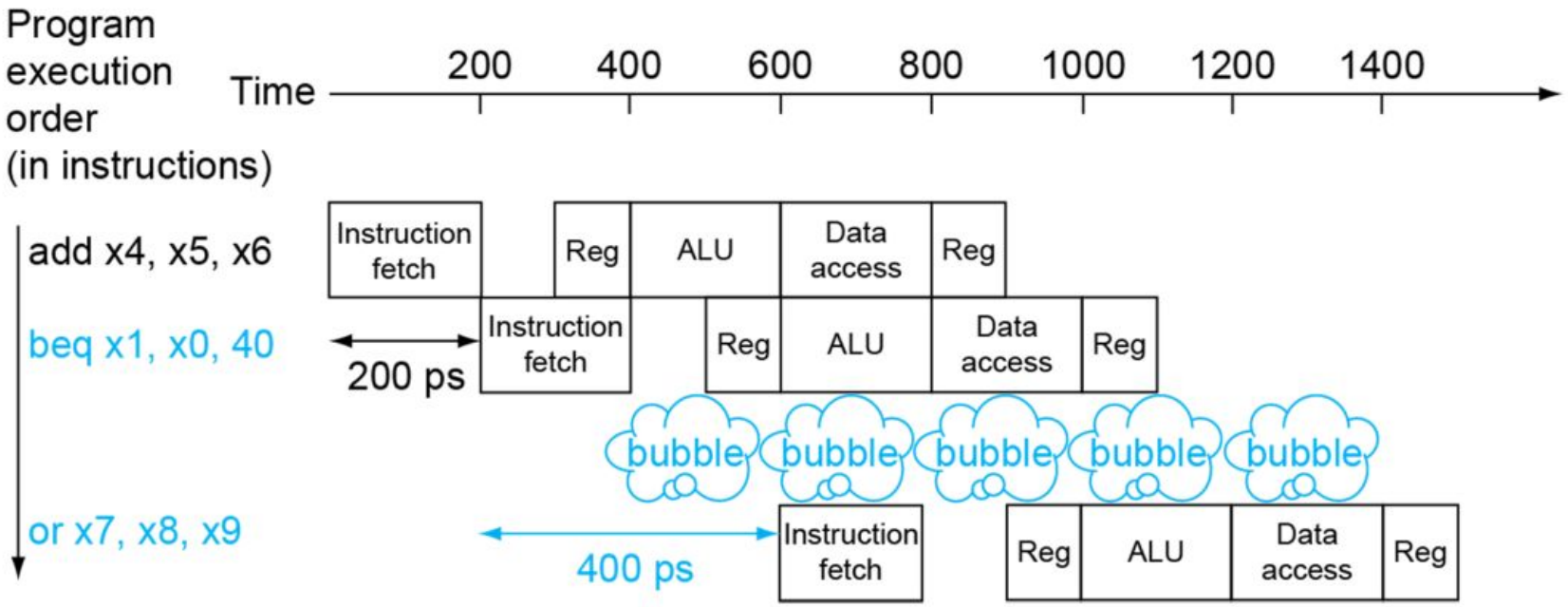
```
lw x1, 0(x31)
lw x2, 8(x31)
ld x4, 16(x31)
add x3, x1, x2
sw x3, 24(x31)
add x5, x1, x4
sw x5, 32(x31)
```

Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can't always fetch correct instruction
    - Still working on ID stage of branch
- In RISC-V pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

Stall on Branch

- Wait until branch outcome determined before fetching next instruction



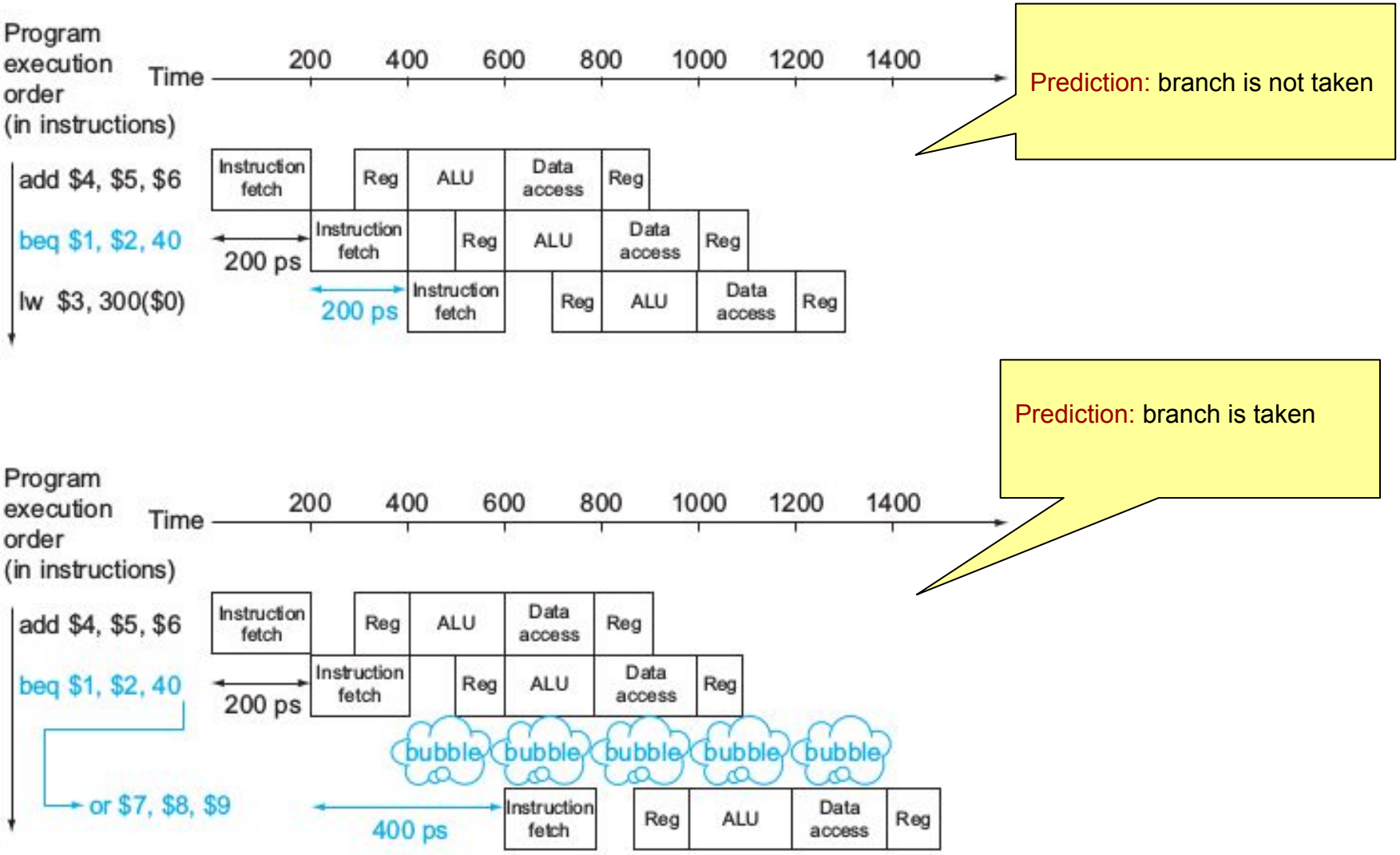
Branch Prediction

- Longer pipelines can't readily determine branch outcome early
  - Stall penalty becomes unacceptable
- Predict outcome of branch
  - Only stall if prediction is wrong
- In RISC-V pipeline
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

More Realistic Branch Prediction

- Static branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not taken
- Dynamic branch prediction
  - Hardware measures actual branch behavior
    - e.g., record recent history of each branch
  - Assume future behavior will continue the trend
    - When wrong, stall while re-fetching, and update history

Branch Prediction Example

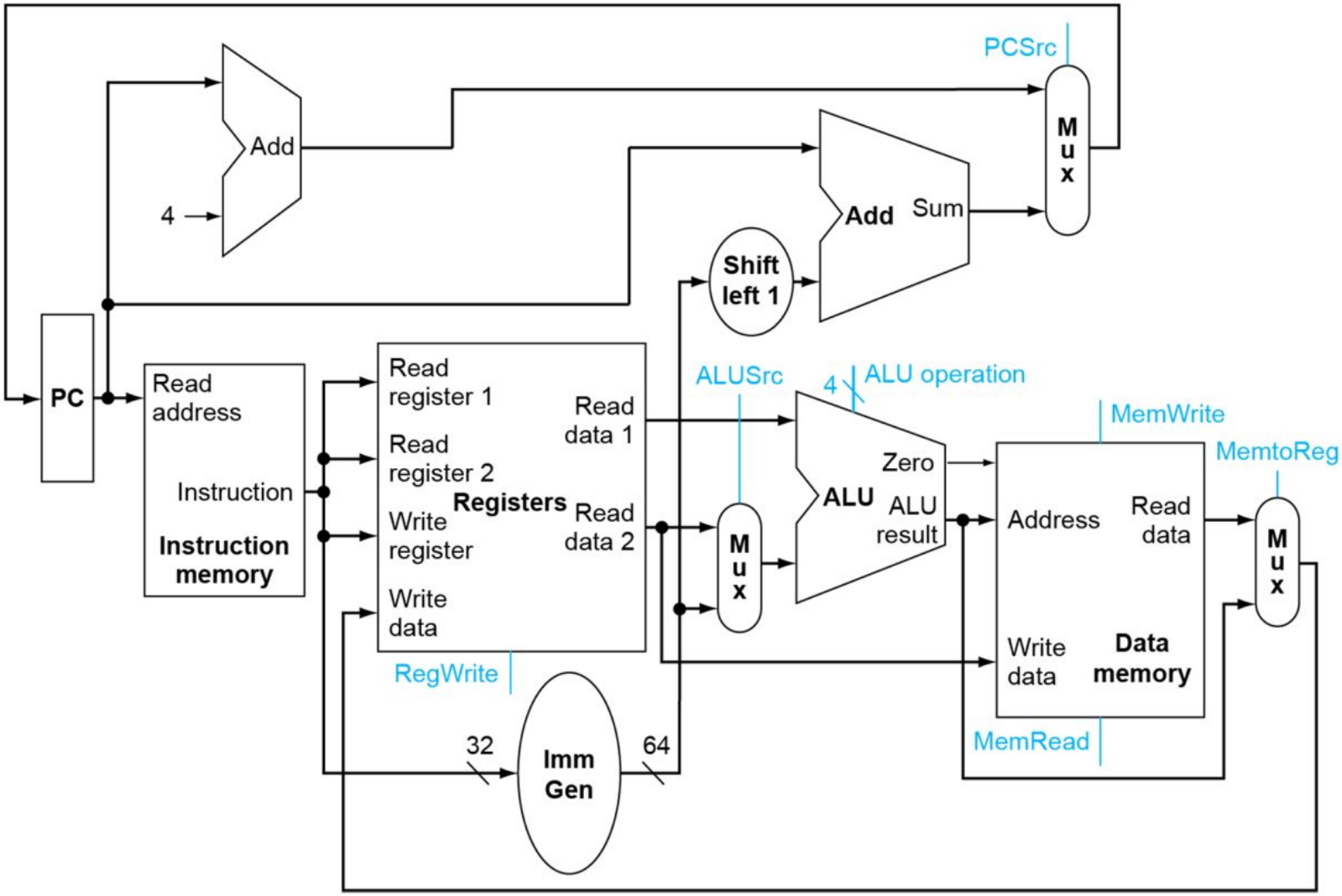


Pipeline Summary

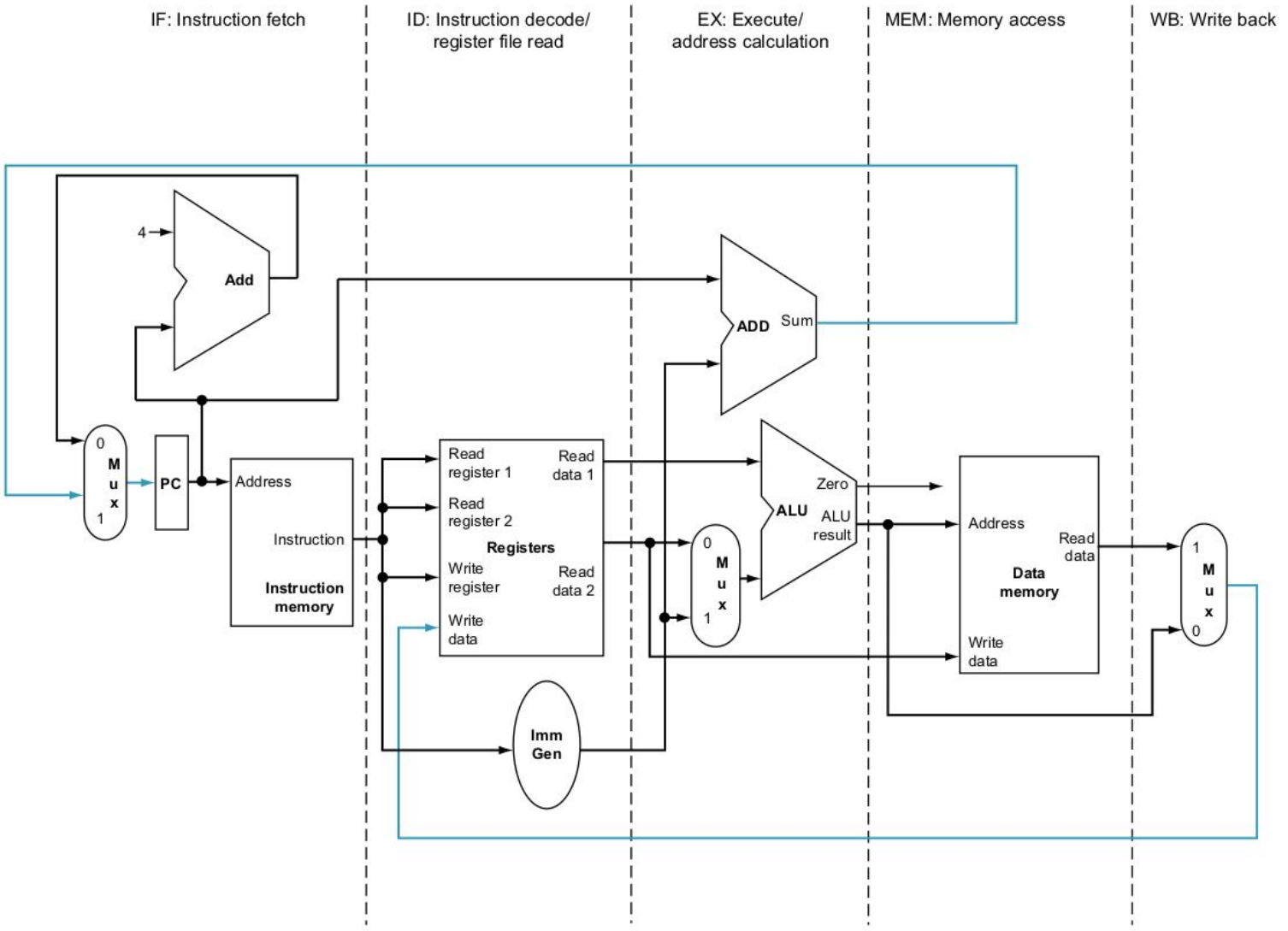
- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards
  - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

PROCESSOR IMPLEMENTATION:
Pipelined Datapath

RISC-V Unpipelined Datapath

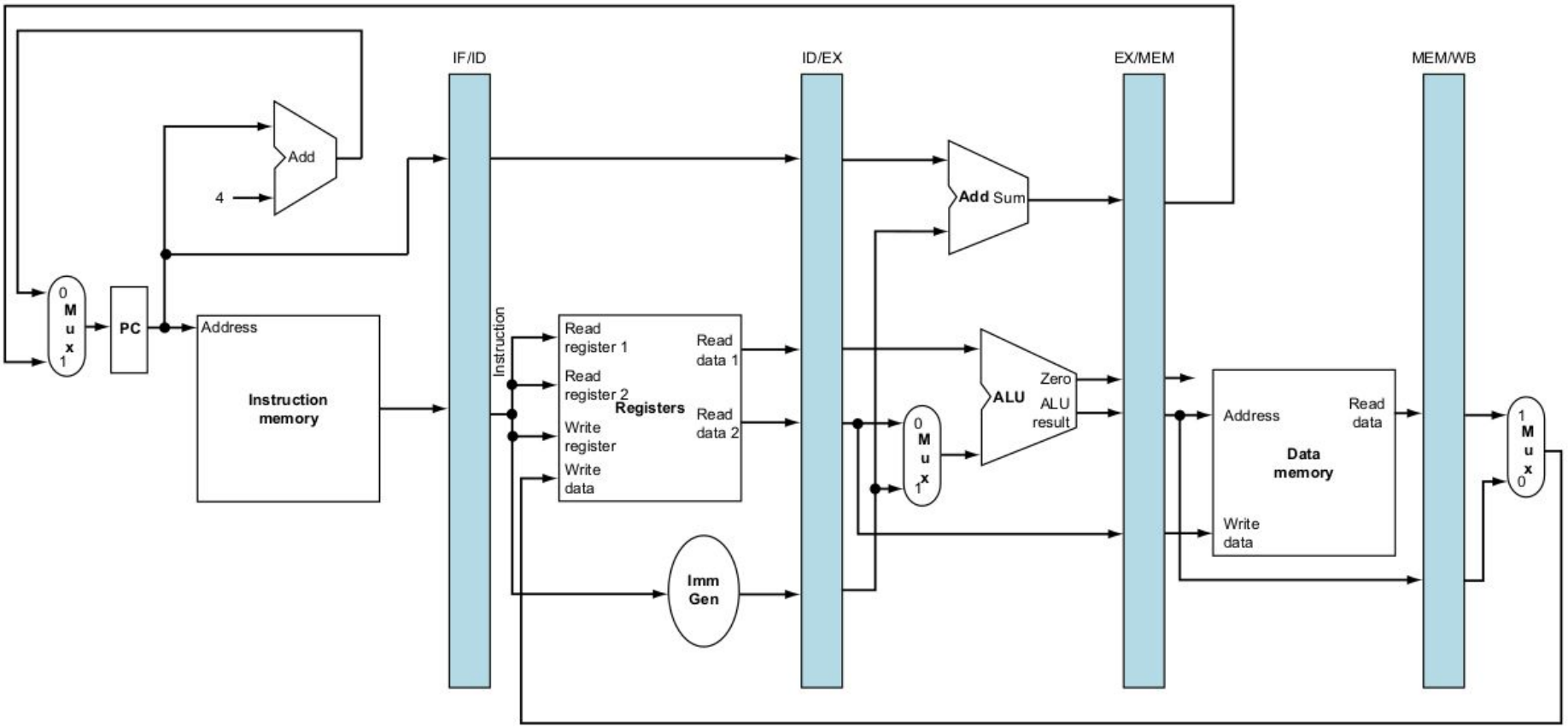


RISC-V Pipelined Datapath



Pipeline Registers

- Need registers between stages
  - To hold information produced in previous cycle



Pipeline Operation

- Cycle-by-cycle flow of instructions through the pipelined datapath
  - "Single-clock-cycle" pipeline diagram
    - Shows pipeline usage in a single cycle
    - Highlight resources used
  - c.f. "multi-clock-cycle" diagram
    - Graph of operation over time
- We'll look at "single-clock-cycle" diagrams for load & store

Exercise: Pipelined Execution

Question: The pipelined MIPS processor is running the following program. Which registers are being written, and which are being read on the fifth cycle? Recall that the processor has a hazard unit.

```
addi x1, x2, 5

sub x0, x1, $t2

lw $t3, 15($s1)

sw $t5, 72($t0)

or $t2, $s4, $s5
```

Exercise: Handling Hazards

Question: Using an instruction execution timing diagram, show the forwarding and stalls needed to execute the following instructions on a pipelined MIPS processor.

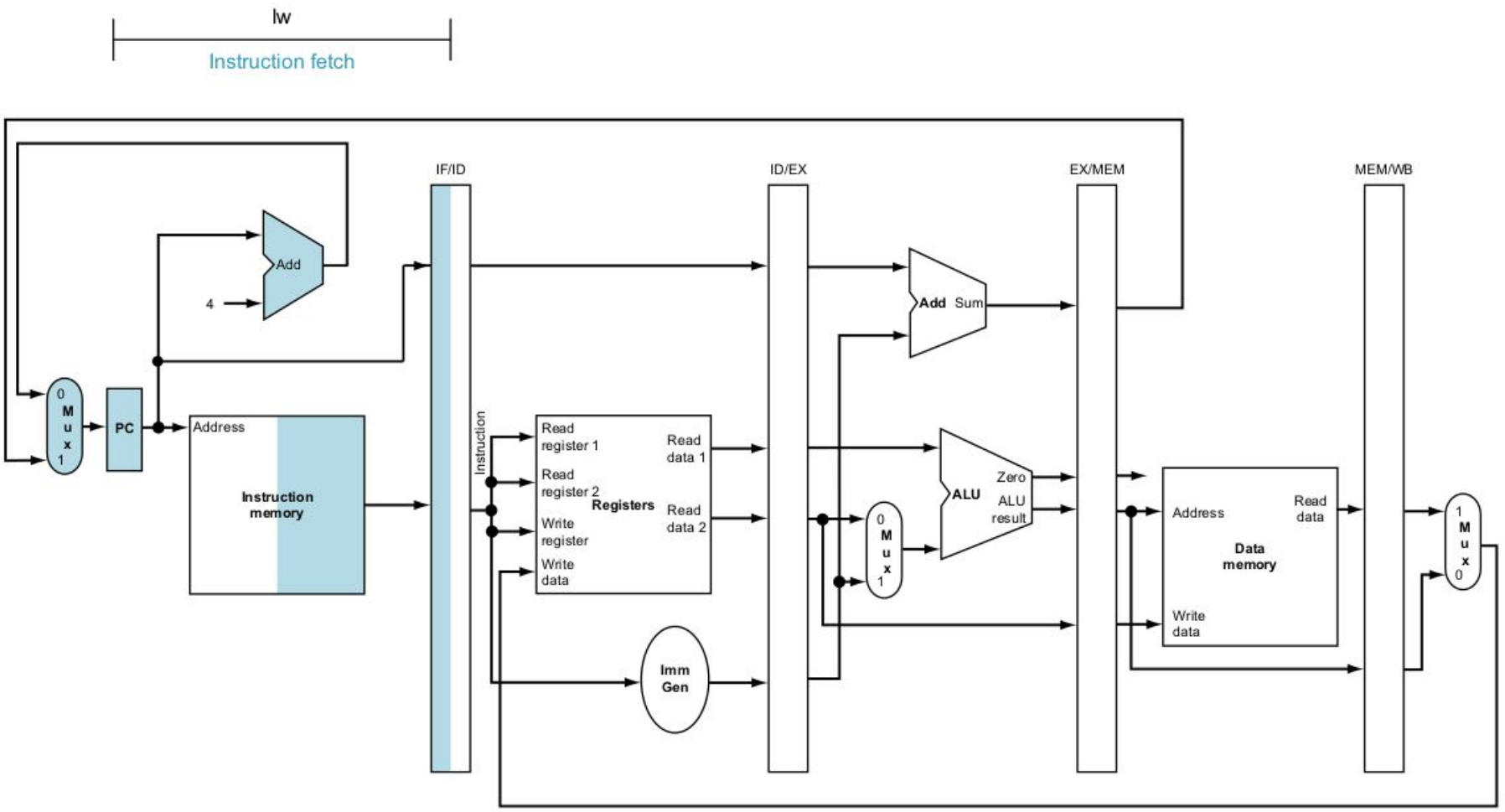
```
add $t0, $s0, $s1
sub $t0, $t0, $s2
lw $t1, 60($t0)
and $t2, $t1, $t0
```

Exercise: Handling Hazards

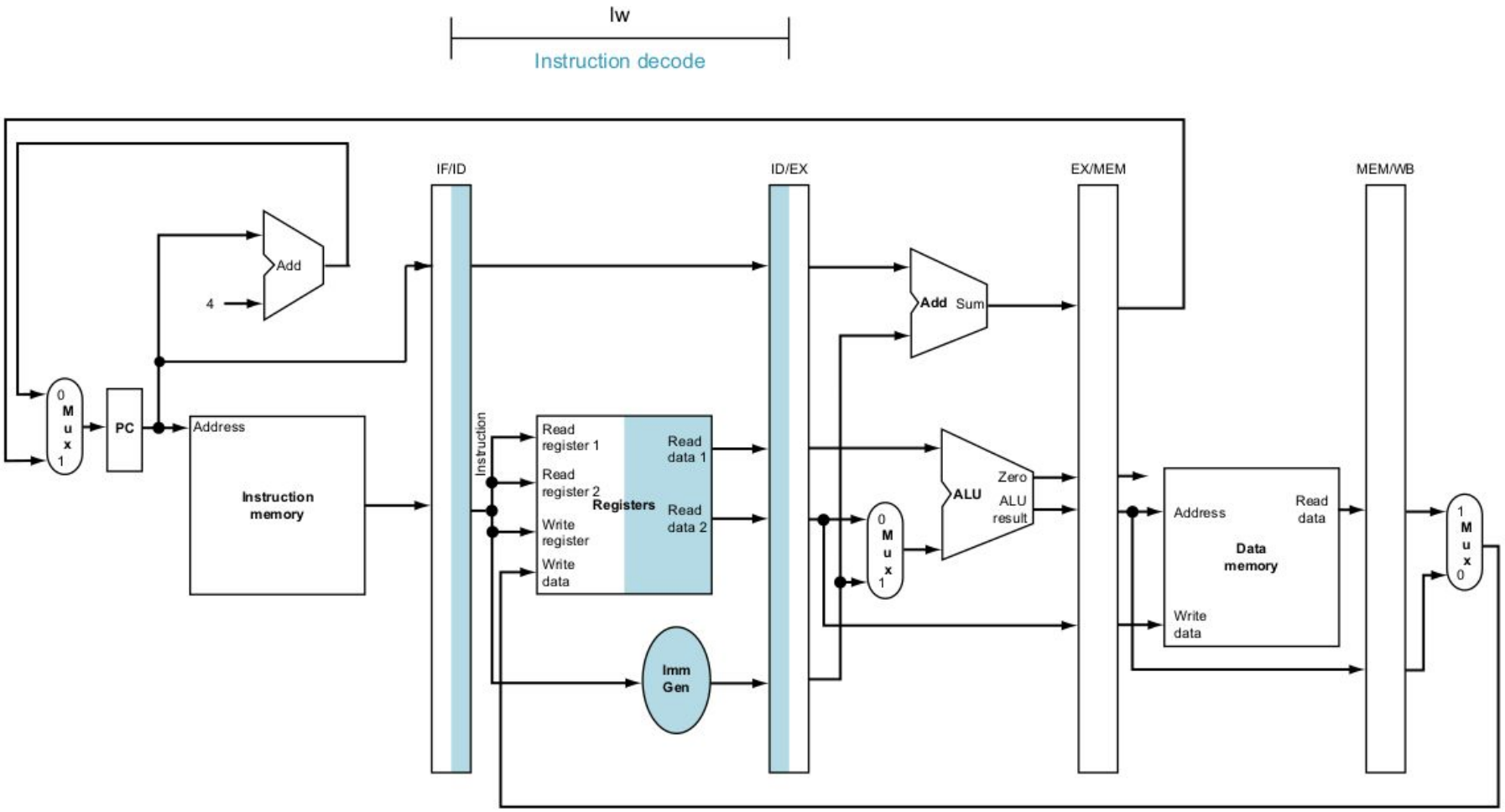
Question: For each code sequence below, state whether it must stall, can avoid stalls using only forwarding, or can execute without stalling or forwarding.

Sequence 1	**Sequence 2**	**Sequence 3**
lw x10, 0(x10) add x11, x10, x10	add x11, x10, x10 addi x12, x10, 5 addi x14, x11, 5	addi x11, x10, 1 addi x12, x10, 2 addi x13, x10, 3 addi x14, x10, 4 addi x15, x10, 5

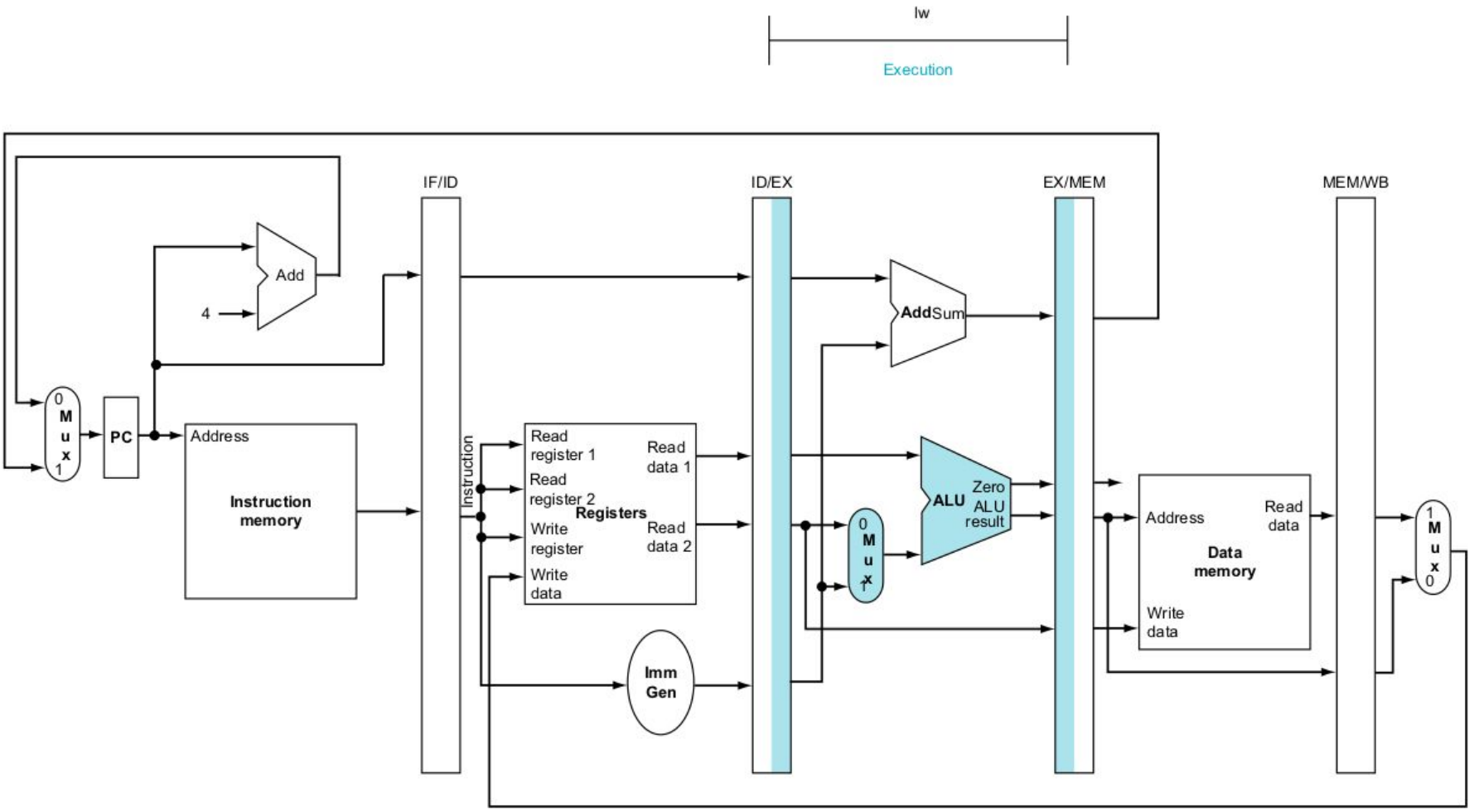
Id: instruction fetch



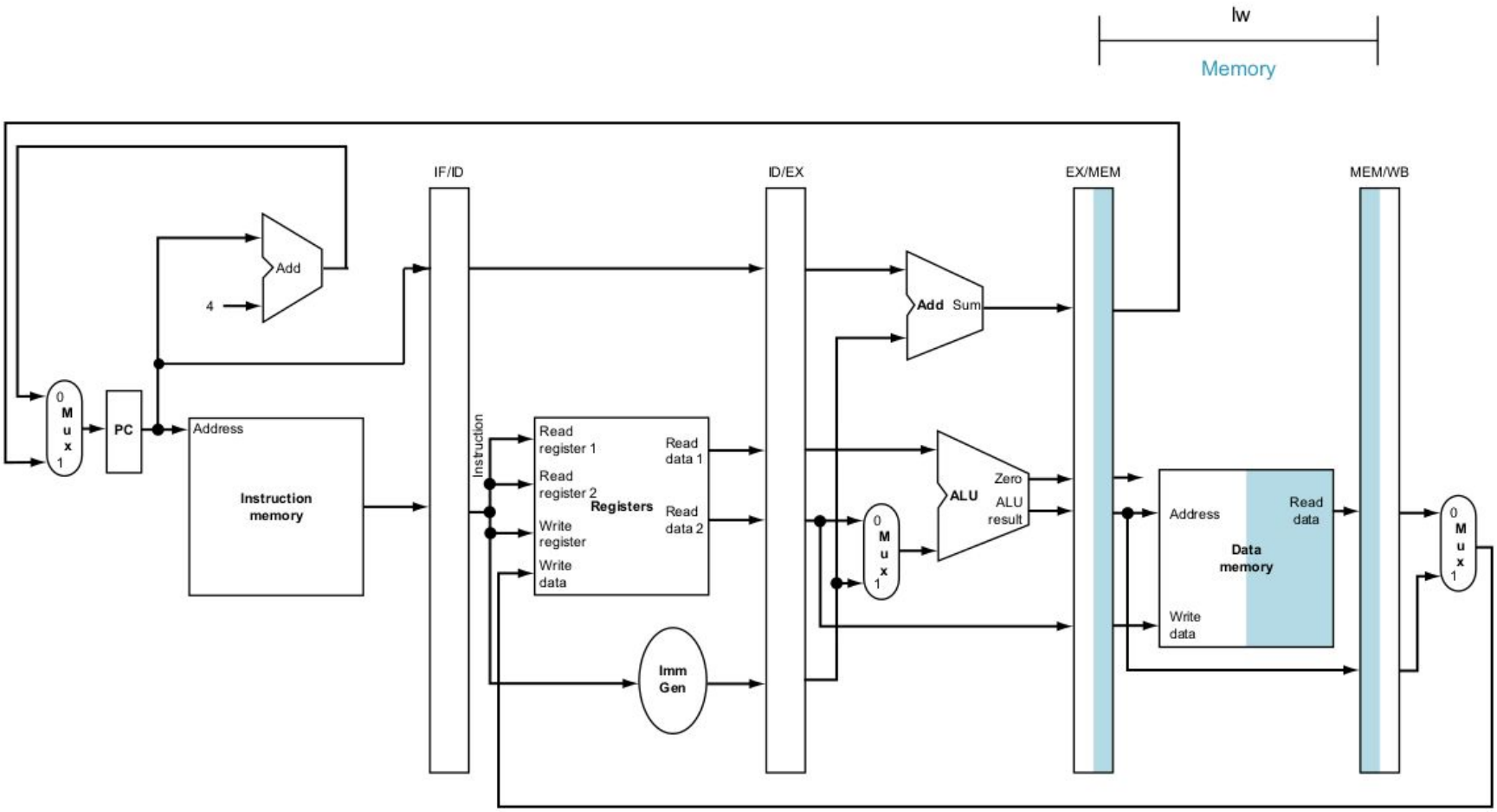
ld: instruction decode



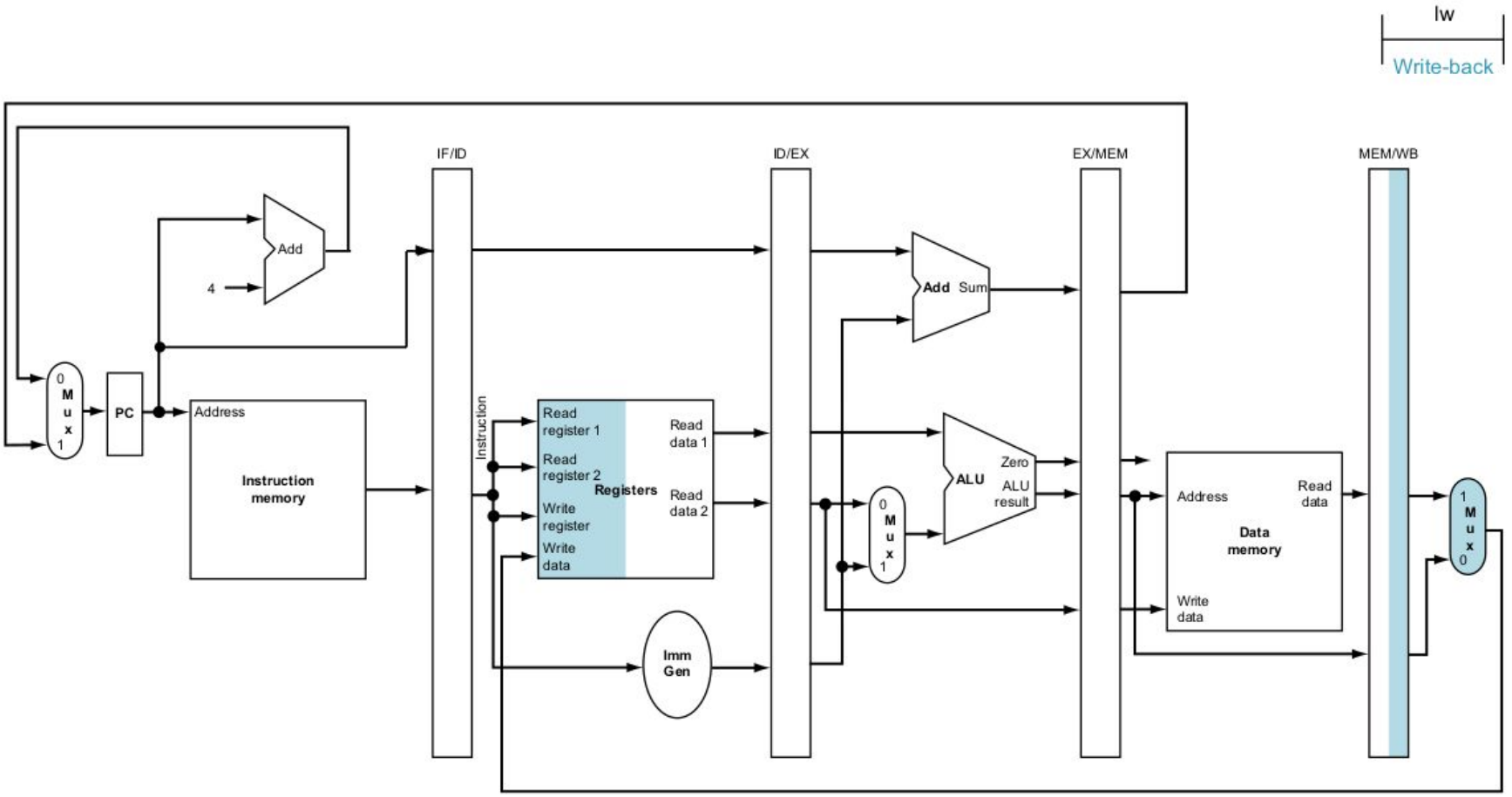
ld: instruction execute



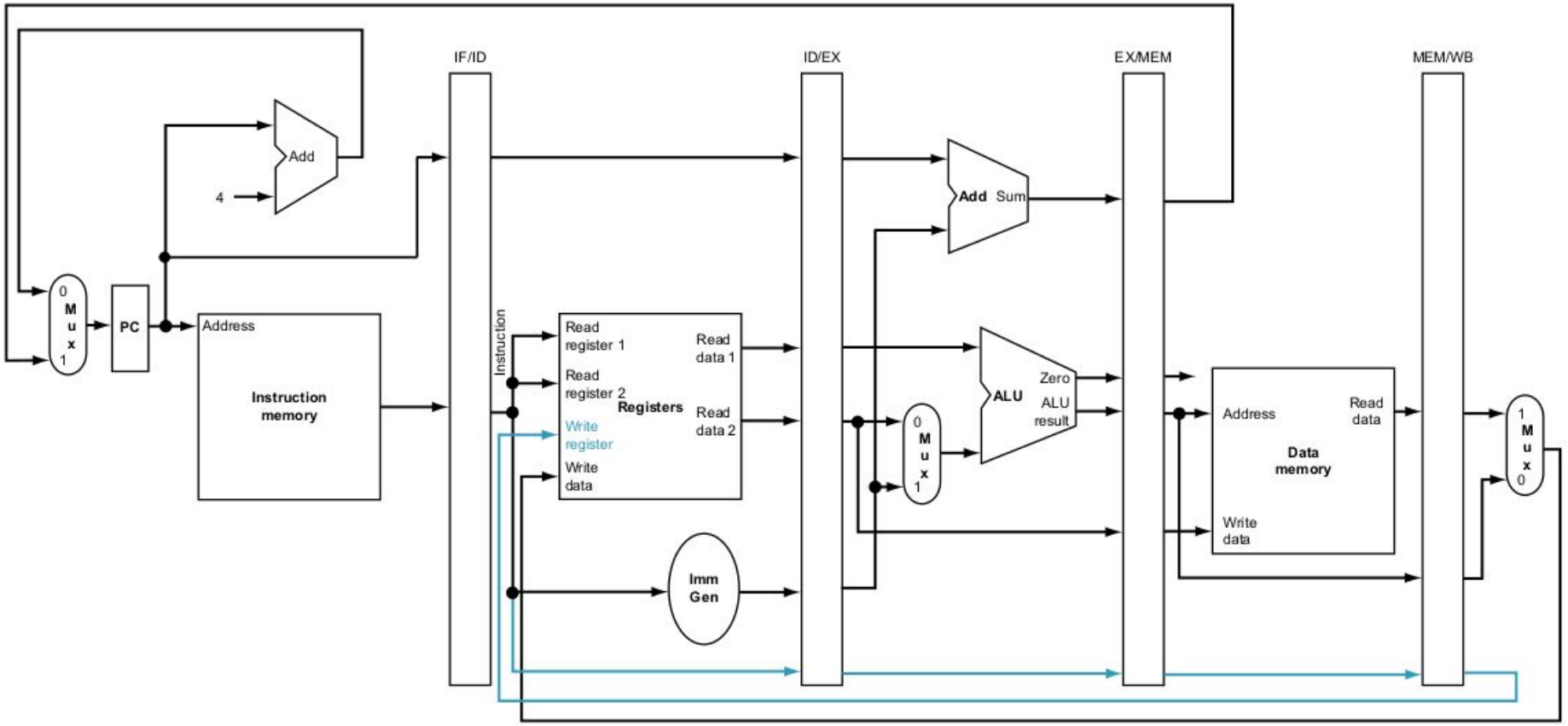
ld: memory access



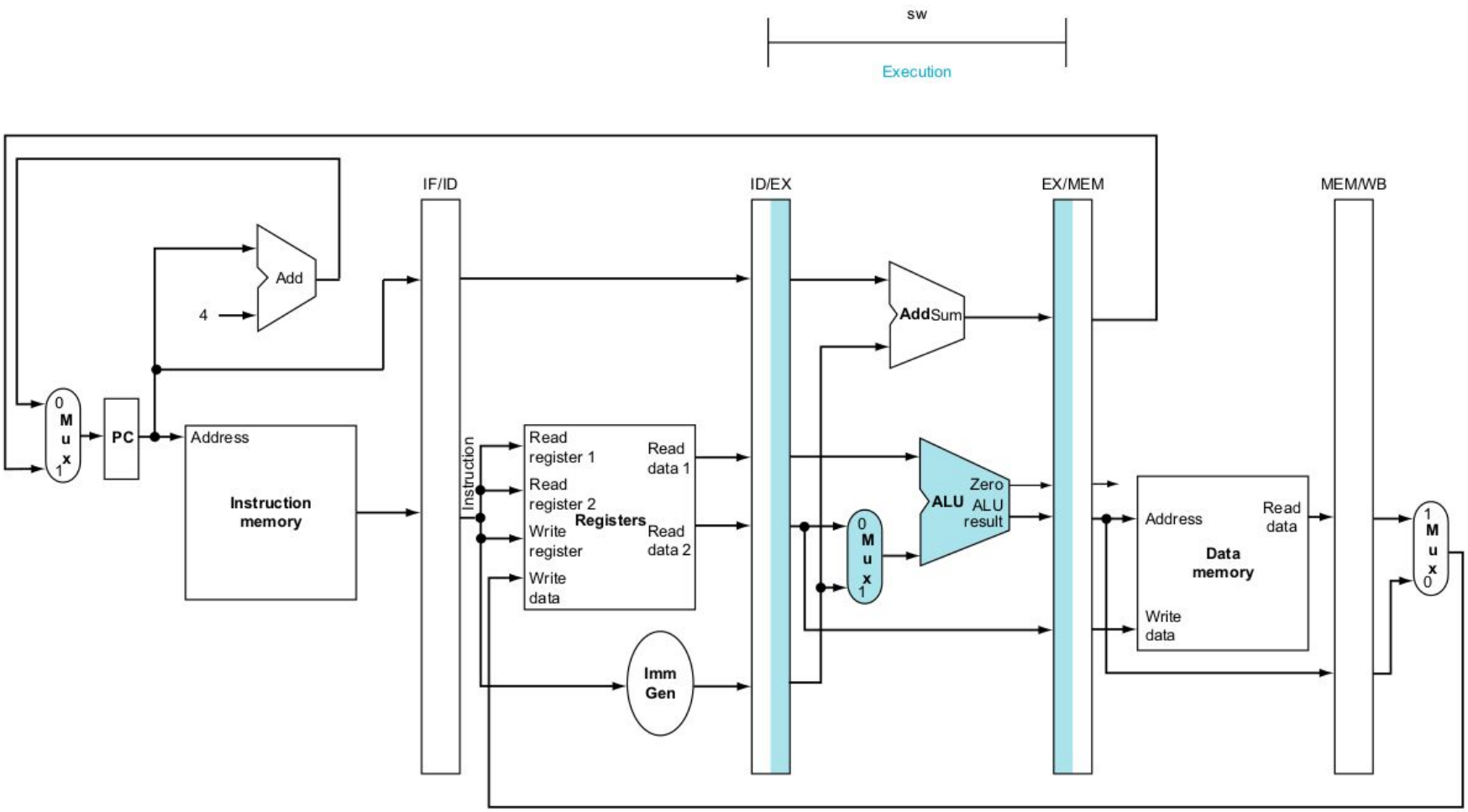
ld: write-back



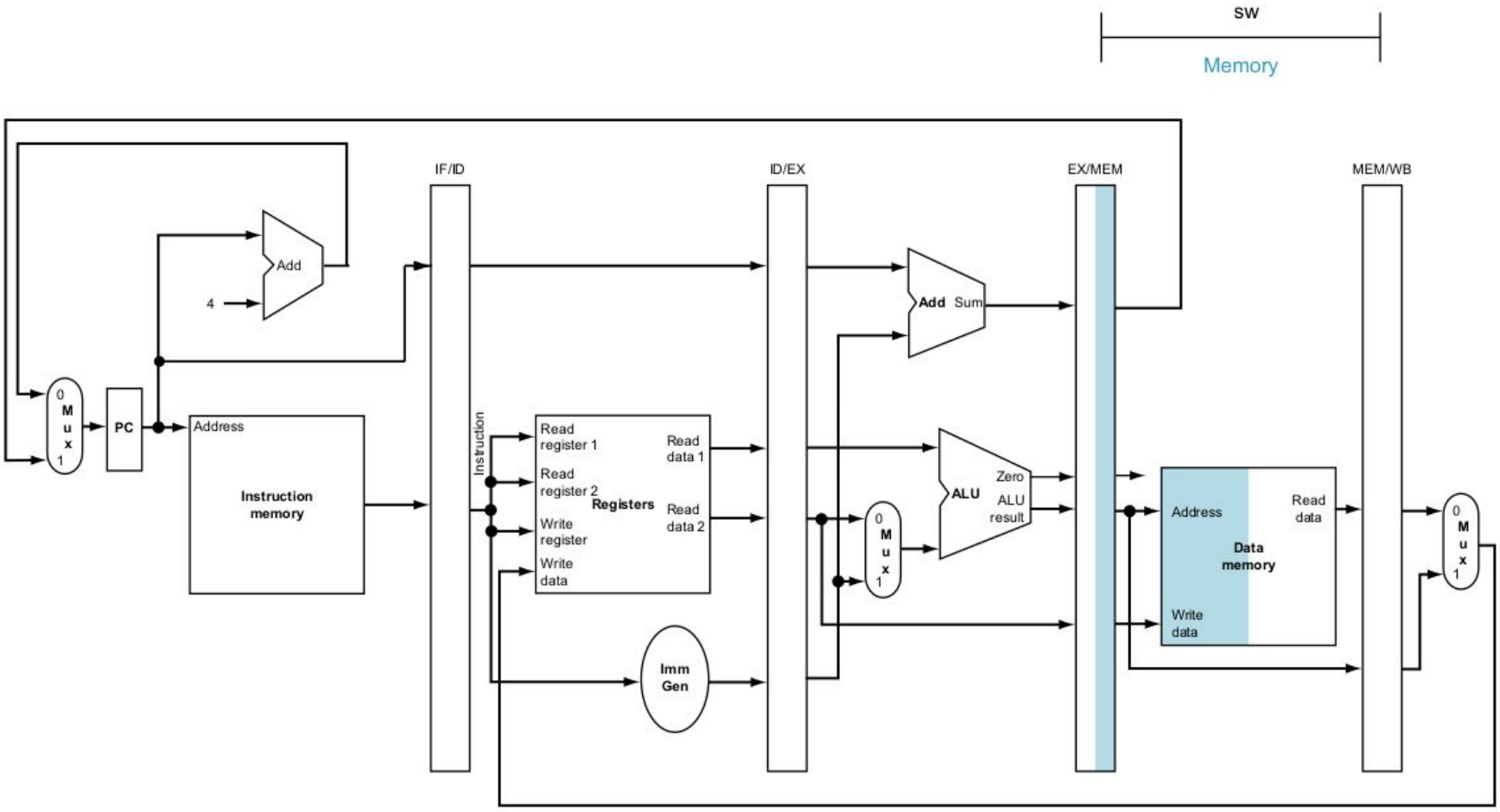
Id: write-back (Revised to Handle wb correctly)



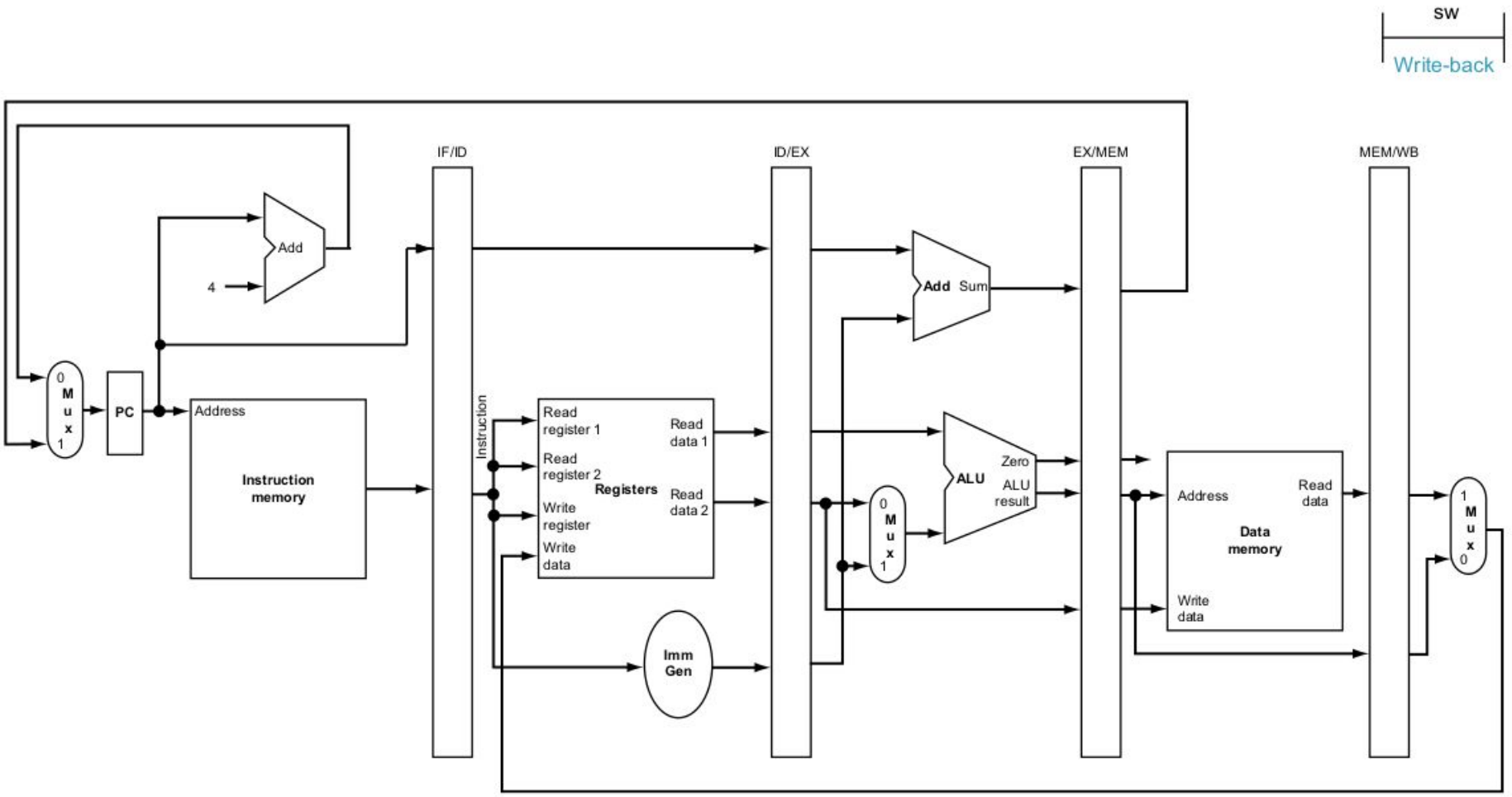
sw: instruction execute



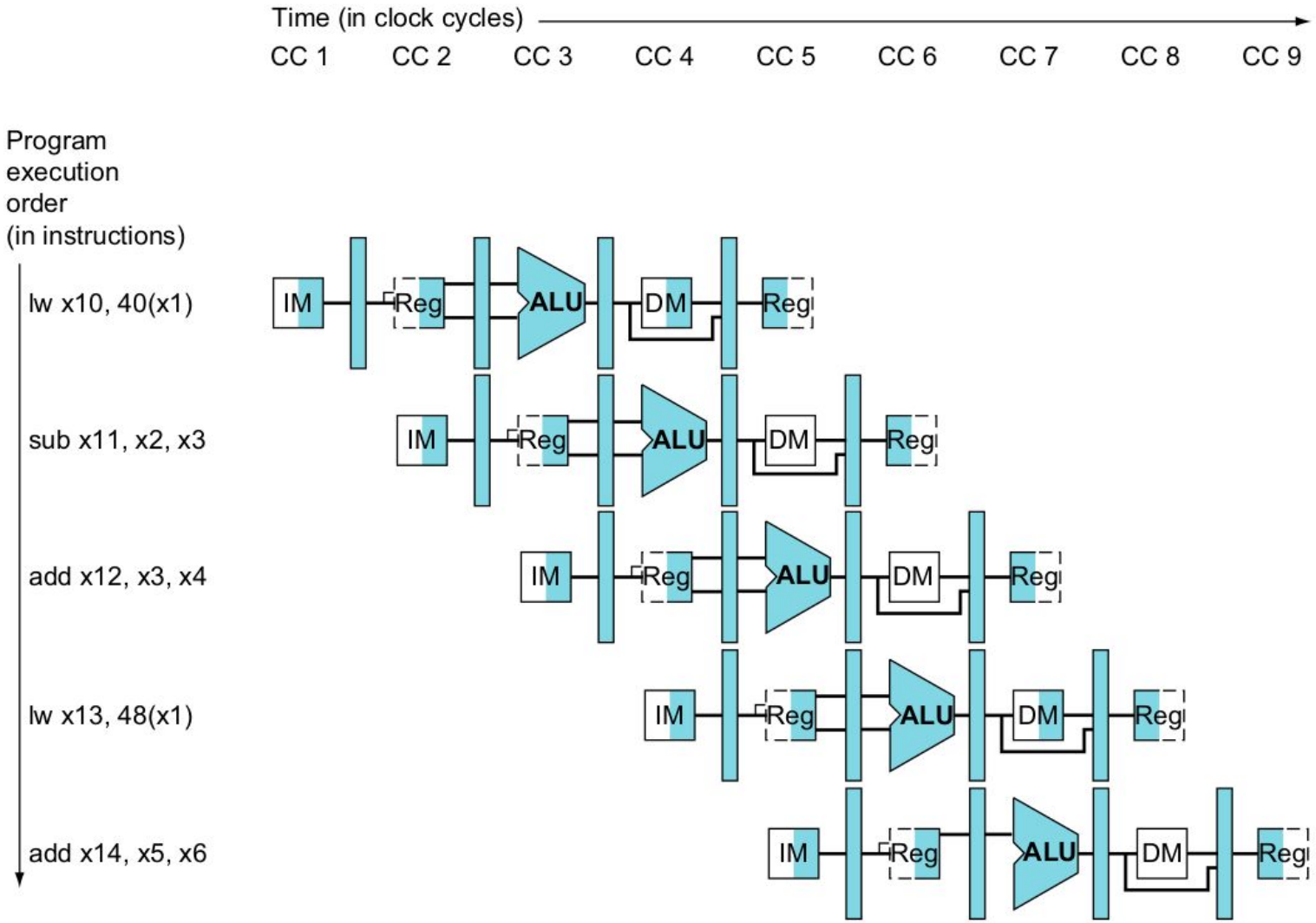
sw: memory


SW
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Imm Gen
Add Sum
ALU
Zero
ALU result
Address
Data memory
Read data
Write data

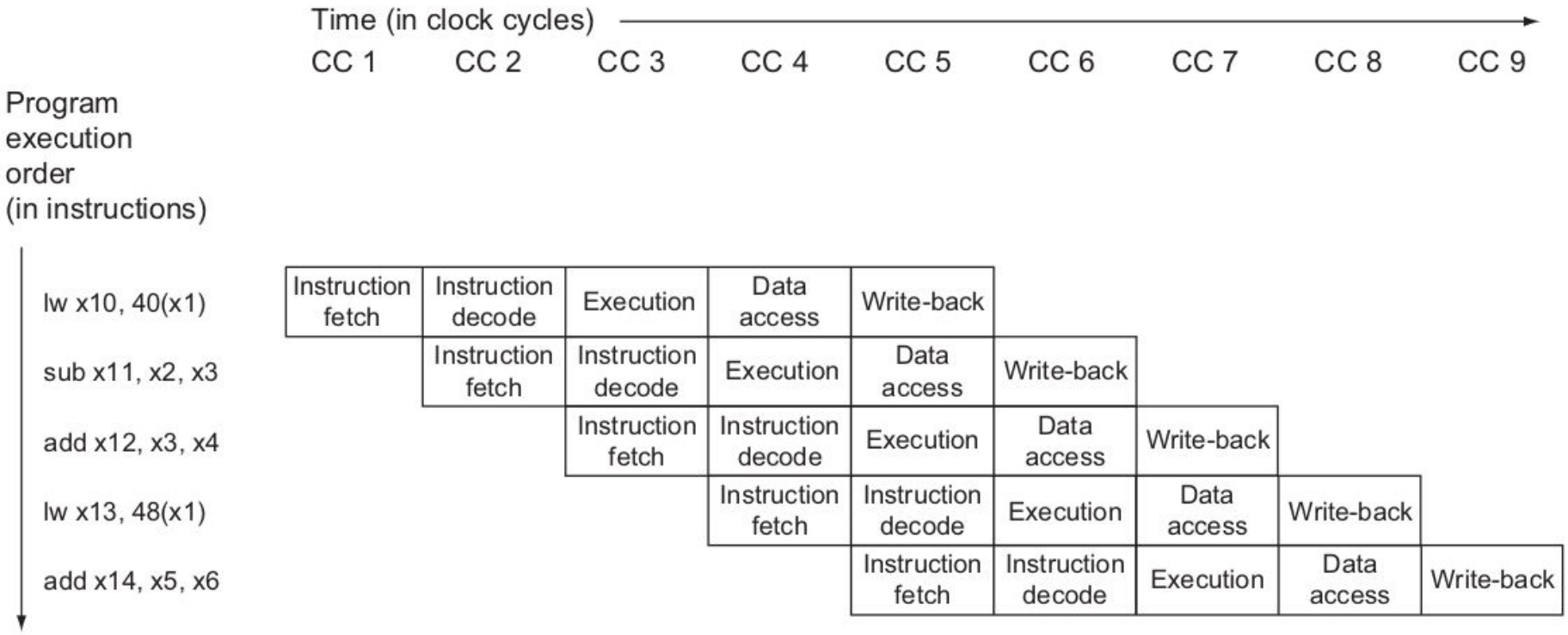
sw: write-back



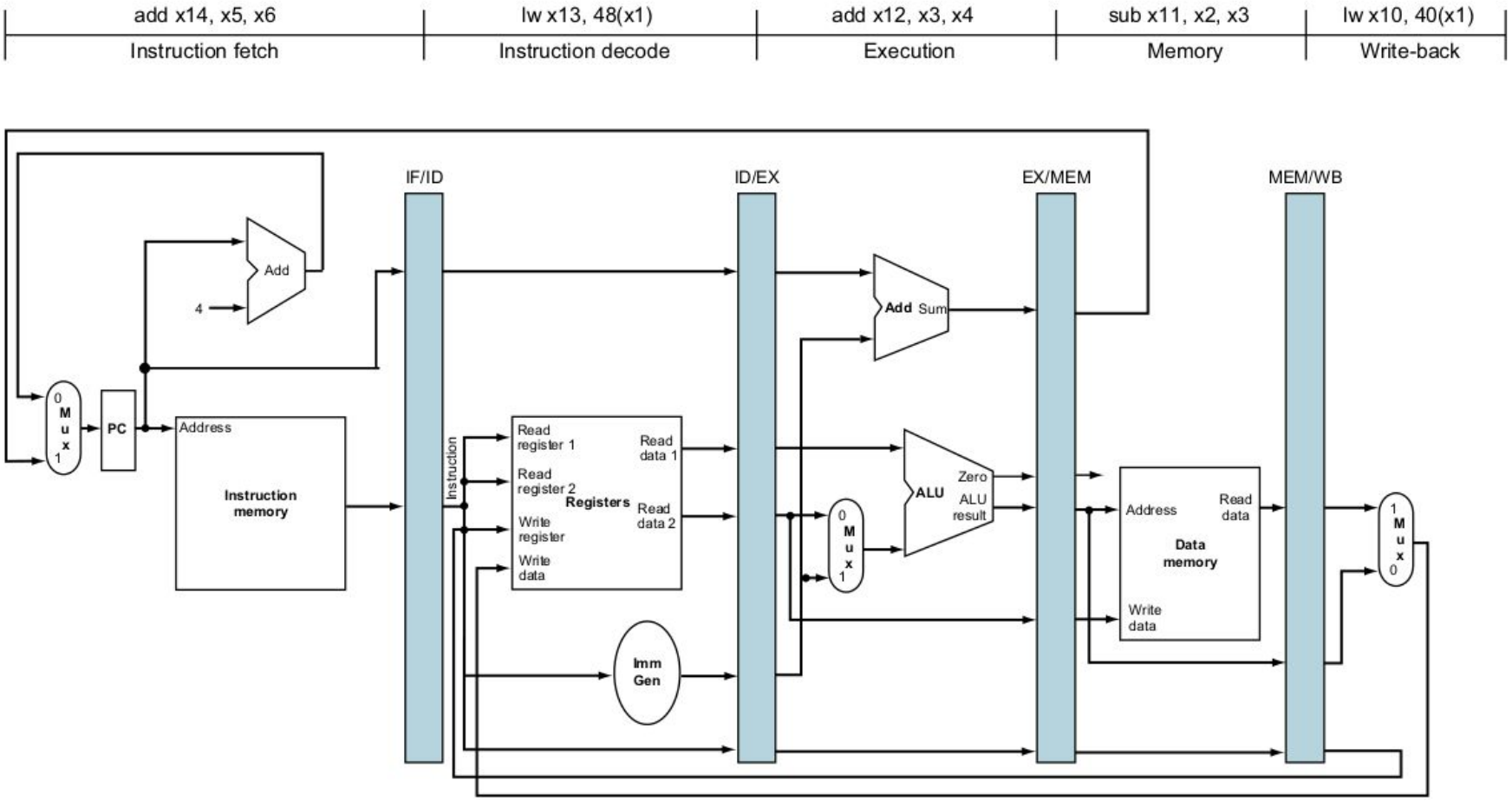
Multi-Cycle Pipeline Diagram



Multi-Cycle Pipeline Diagram



Single-Cycle Pipeline Diagram

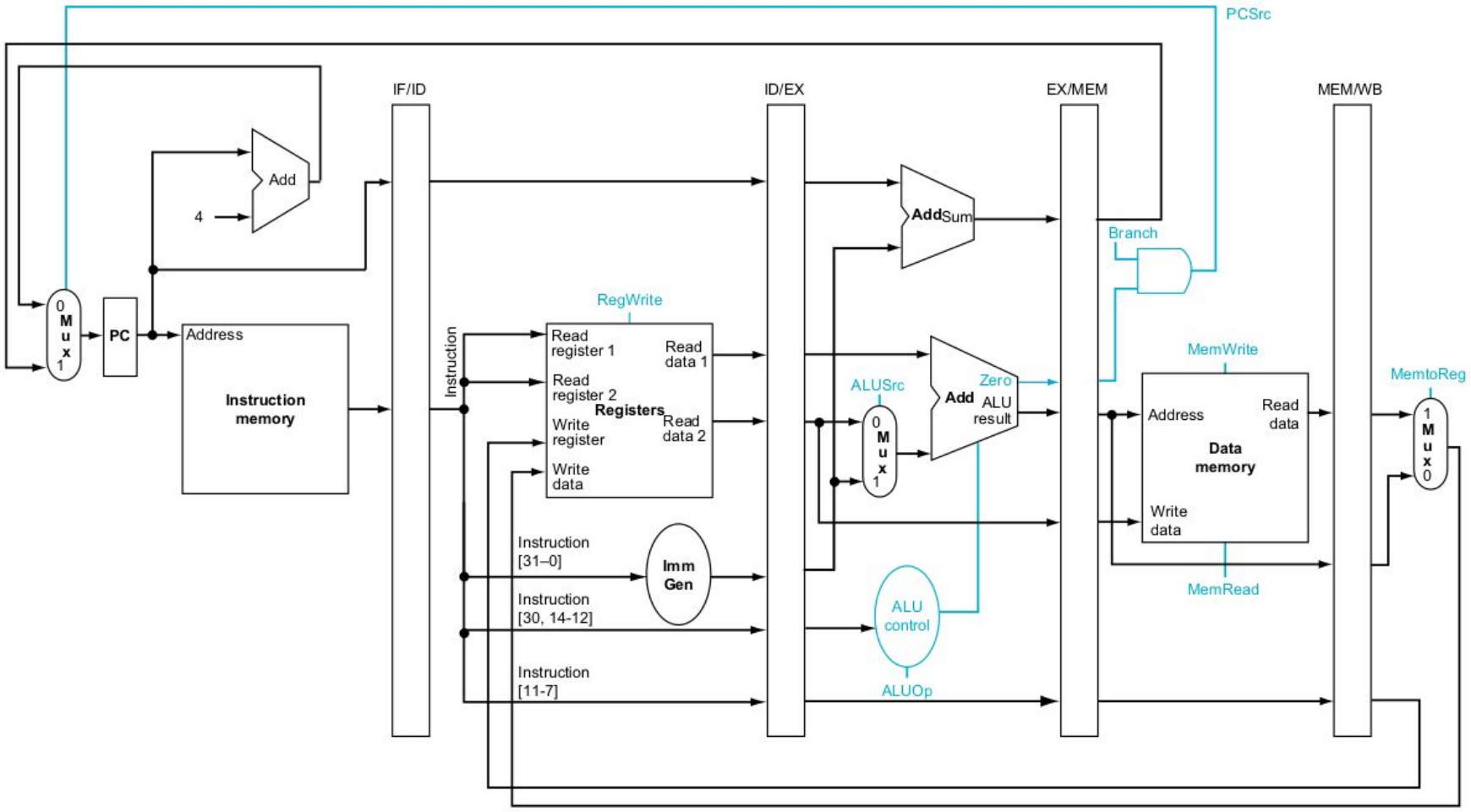


Exercise: Processor Extension

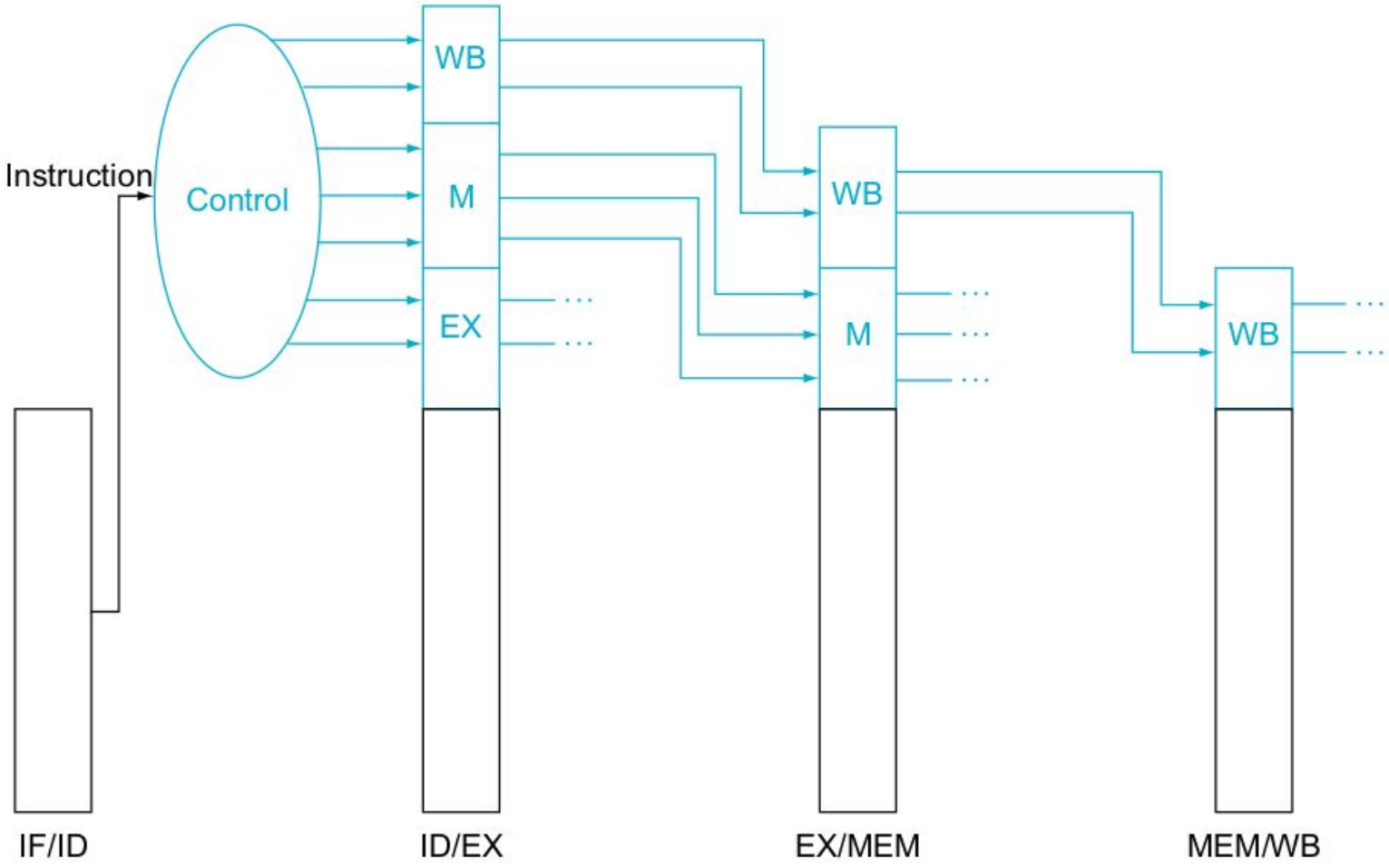
Question: Explain how to extend the pipelined MIPS processor to handle the following instructions.

a) addi
b) jr
c) bne

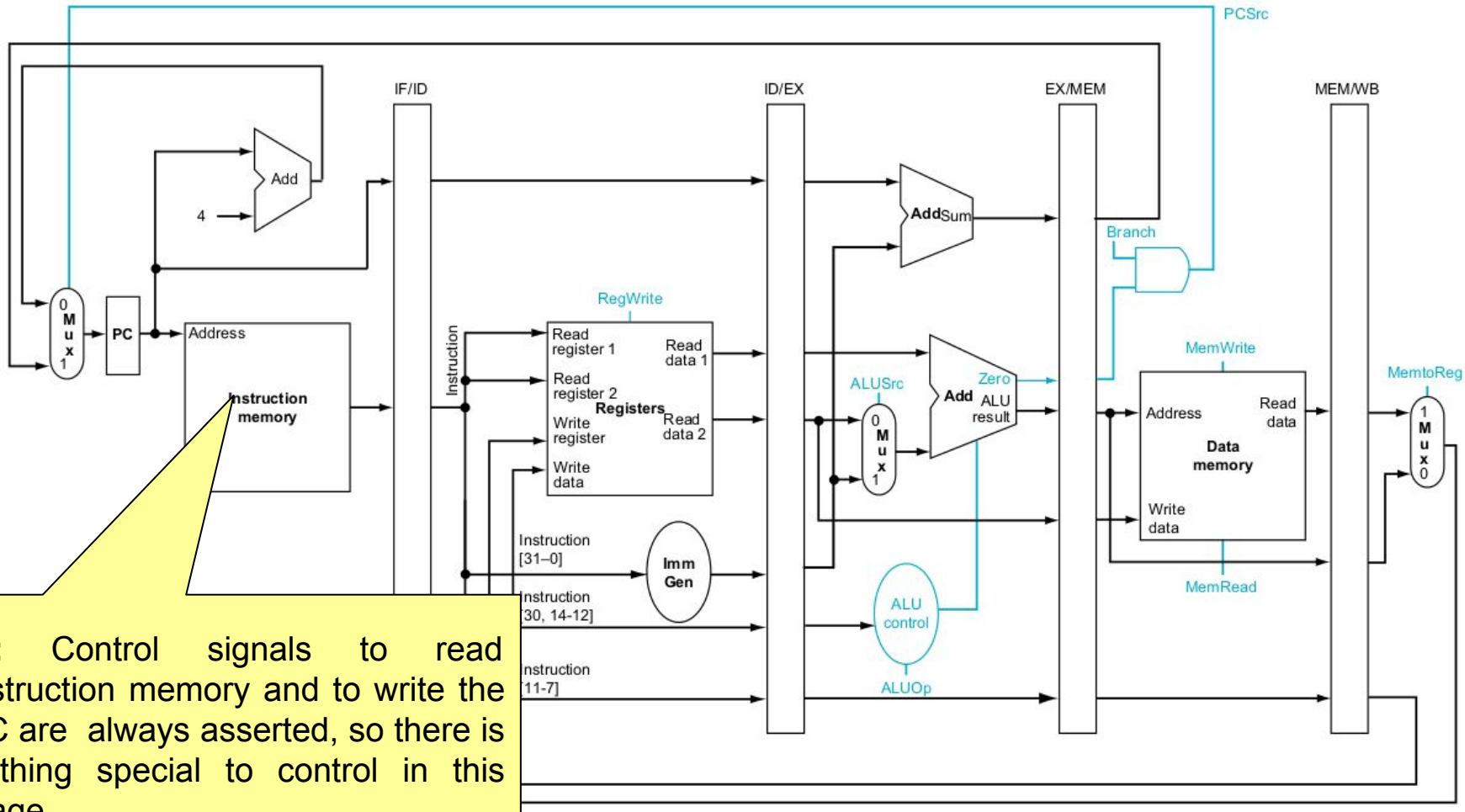
Pipelined Control



Pipelined Control

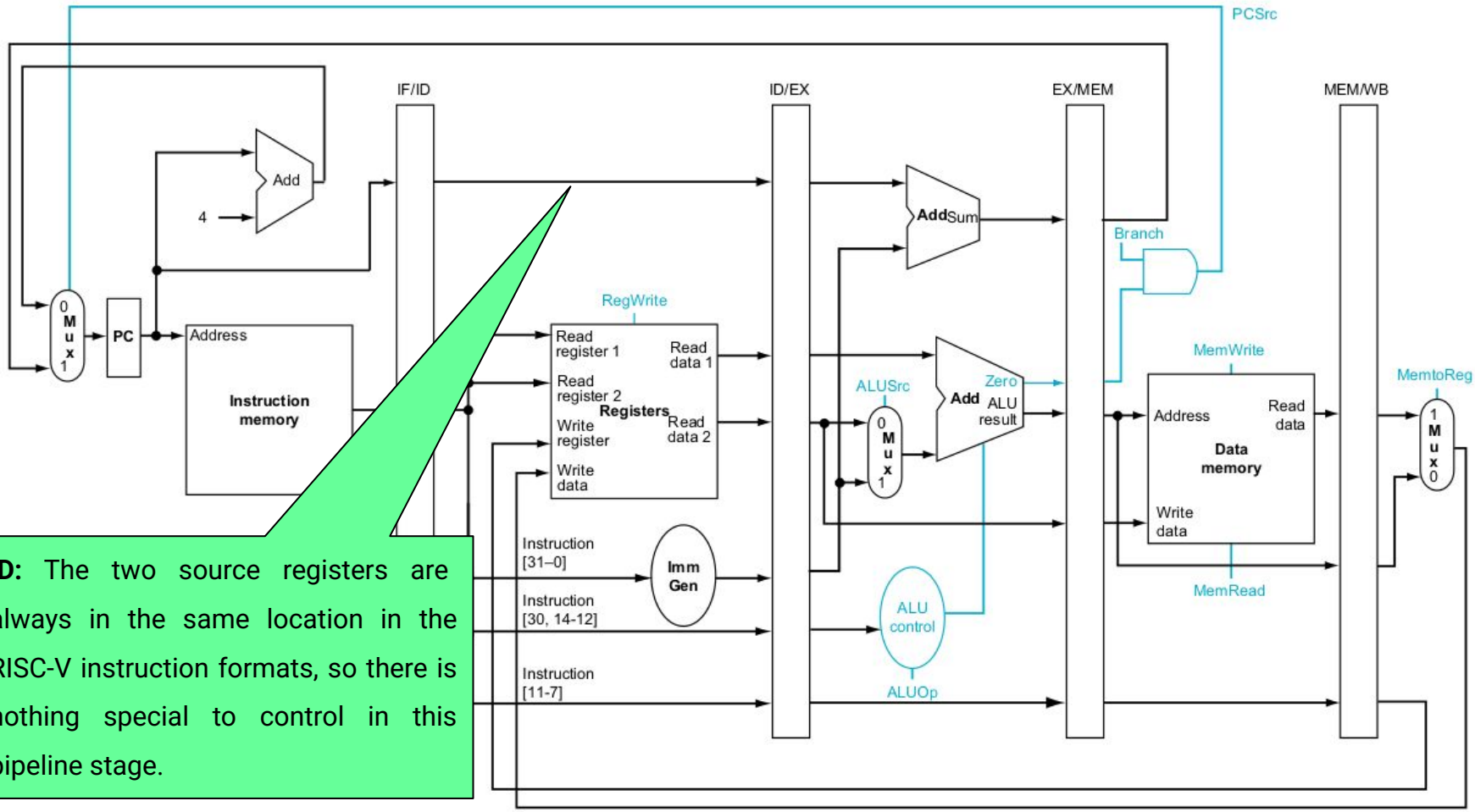


Pipeline Control Specification: Instruction Fetch



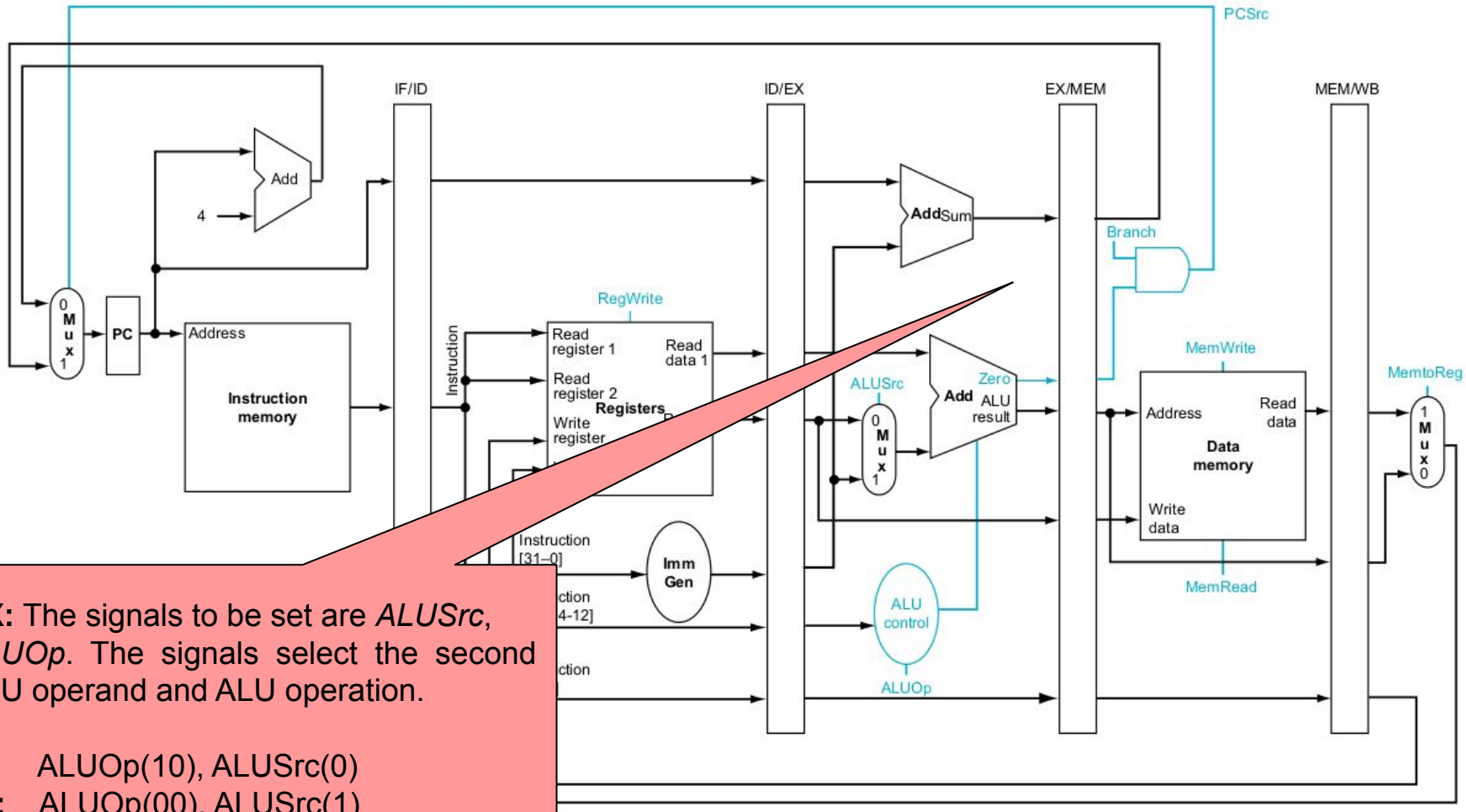
IF: Control signals to read instruction memory and to write the PC are always asserted, so there is nothing special to control in this stage.

Pipeline Control Specification: Instruction Decode



ID: The two source registers are always in the same location in the RISC-V instruction formats, so there is nothing special to control in this pipeline stage.

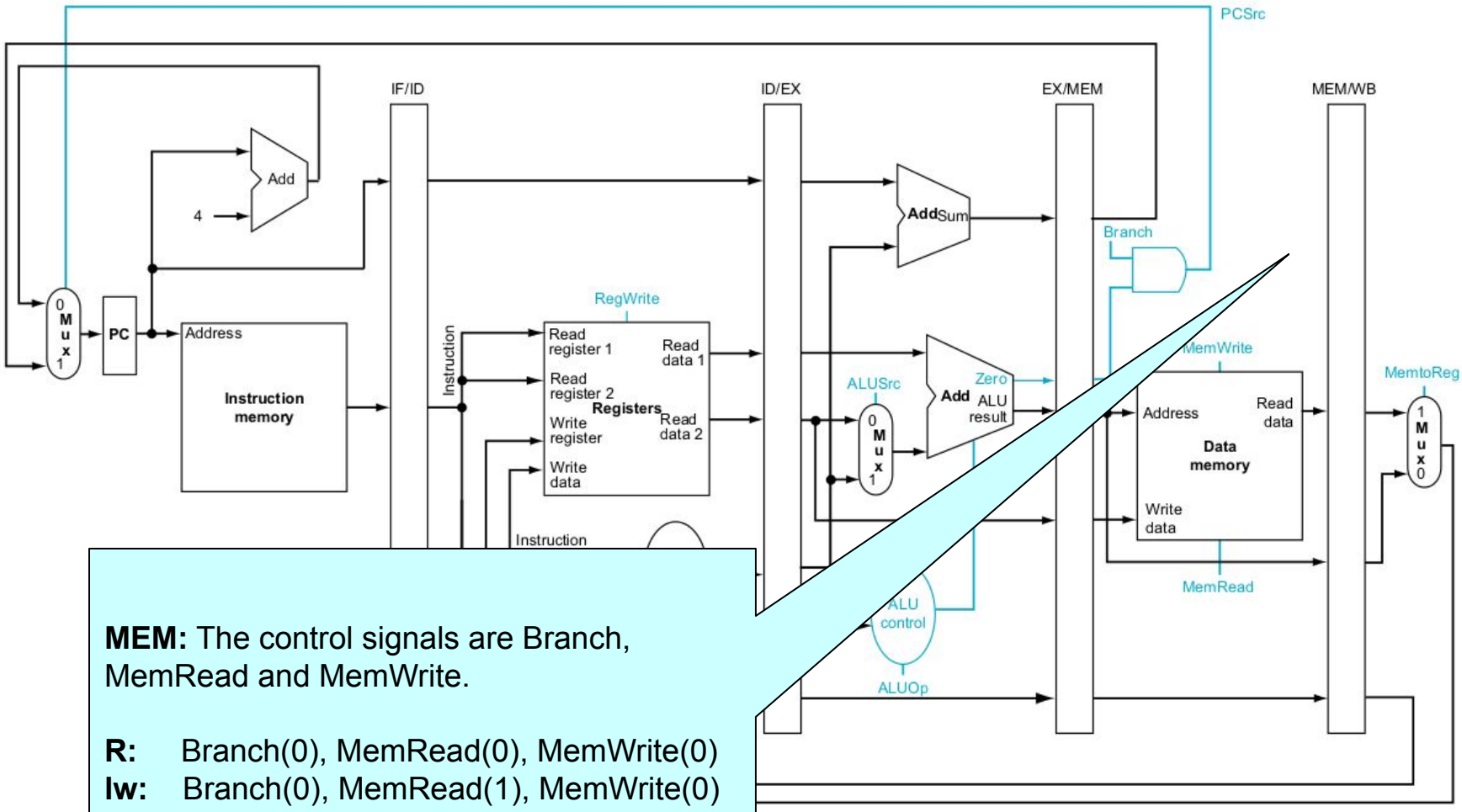
Pipeline Control Specification: Instruction Execute



EX: The signals to be set are *ALUSrc*, *ALUOp*. The signals select the second ALU operand and ALU operation.

R: ALUOp(10), ALUSrc(0)
lw: ALUOp(00), ALUSrc(1)
sw: ALUOp(00), ALUSrc(1)
beq: ALUOp(01), ALUSrc(0)

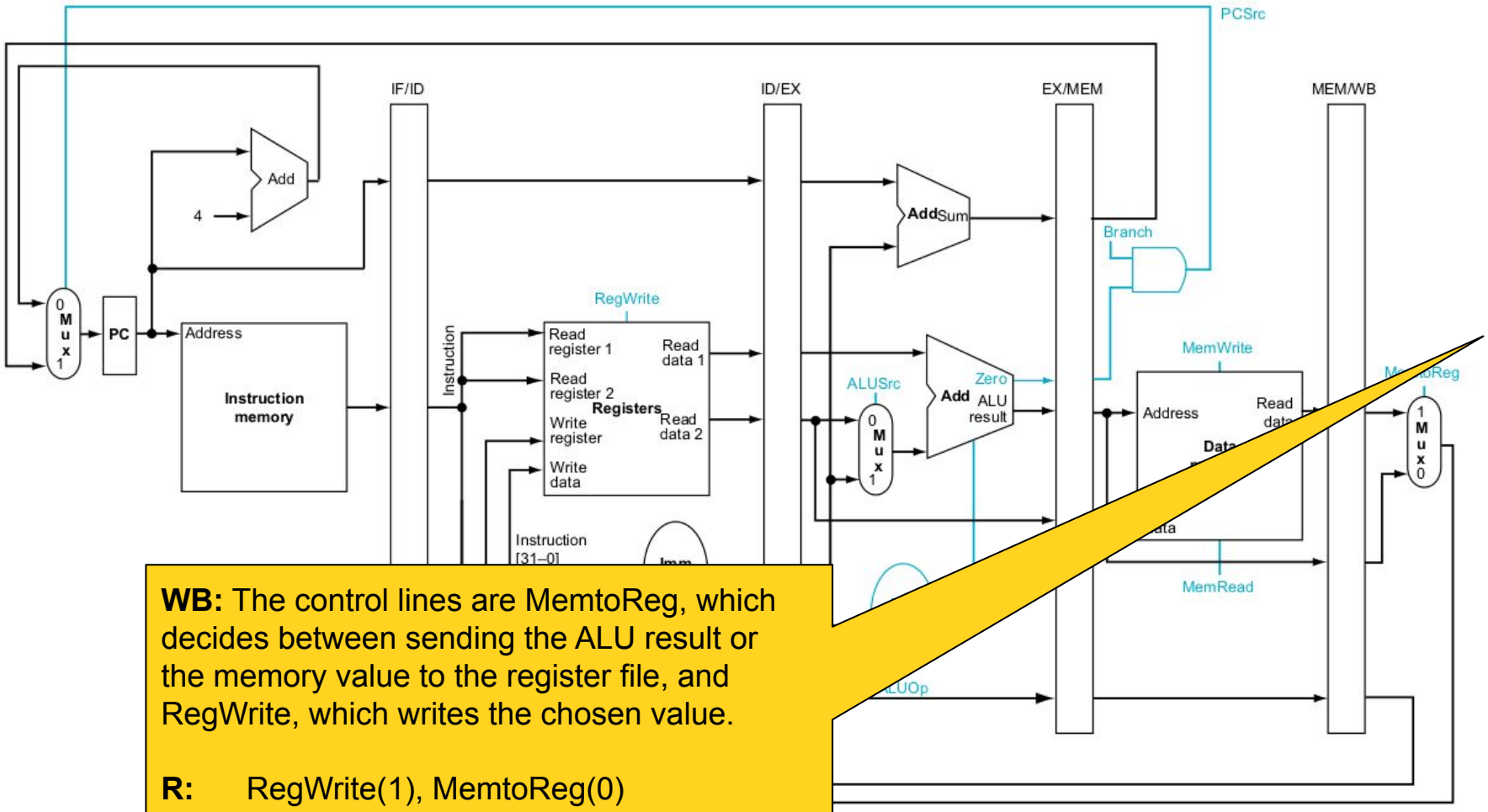
Pipeline Control Specification: Memory Access



MEM: The control signals are Branch, MemRead and MemWrite.

R: Branch(0), MemRead(0), MemWrite(0)
lw: Branch(0), MemRead(1), MemWrite(0)
sw: Branch(0), MemRead(0), MemWrite(1)
beq: Branch(1), MemRead(0), MemWrite(0)

Pipeline Control Specification: Write Back



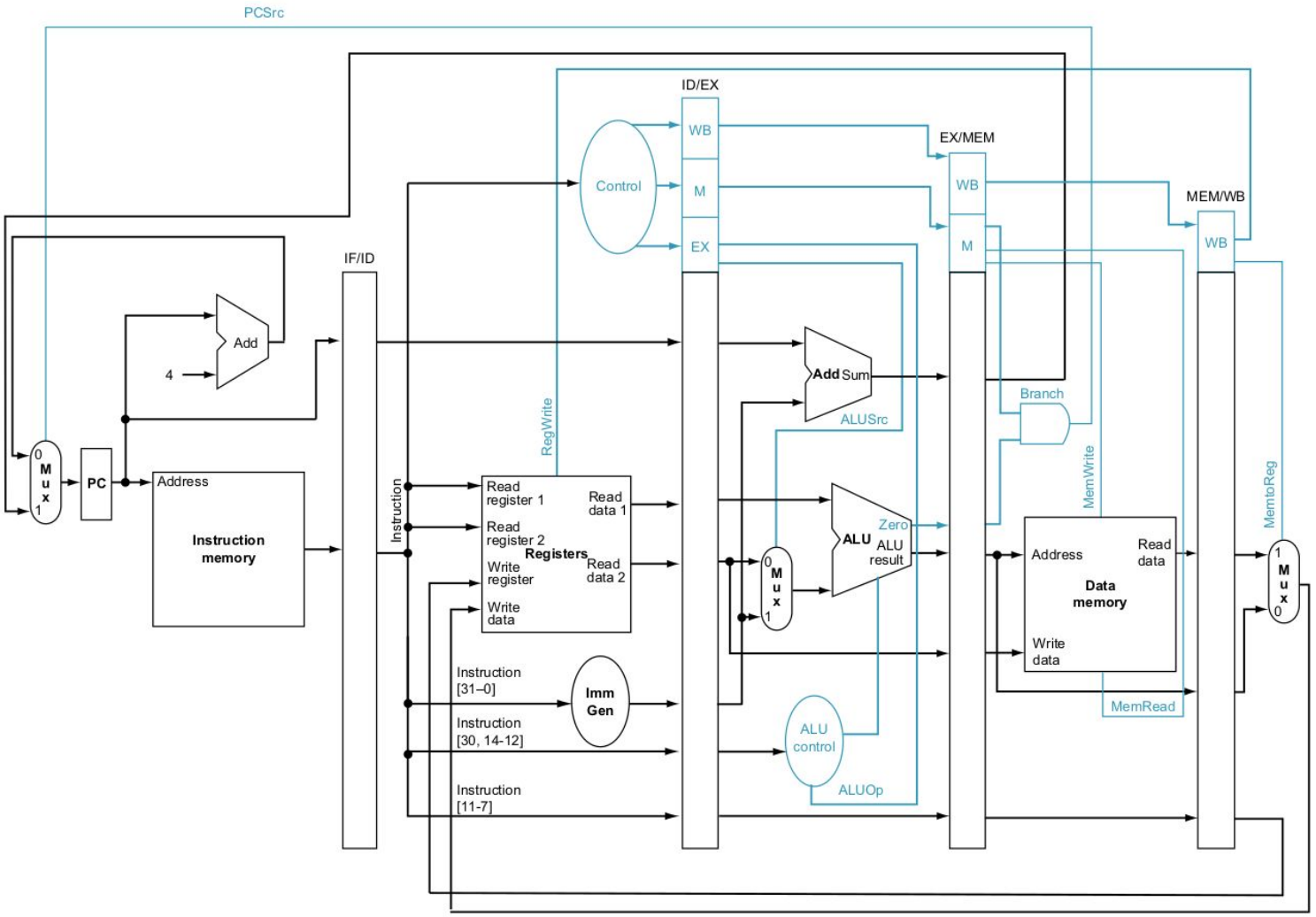
WB: The control lines are MemtoReg, which decides between sending the ALU result or the memory value to the register file, and RegWrite, which writes the chosen value.

R: RegWrite(1), MemtoReg(0)
lw: RegWrite(1), MemtoReg(1)
sw: RegWrite(0), MemtoReg(x)
beq: RegWrite(0), MemtoReg(x)

Pipelined Datapath: All Control Signals

Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

Pipelined Control



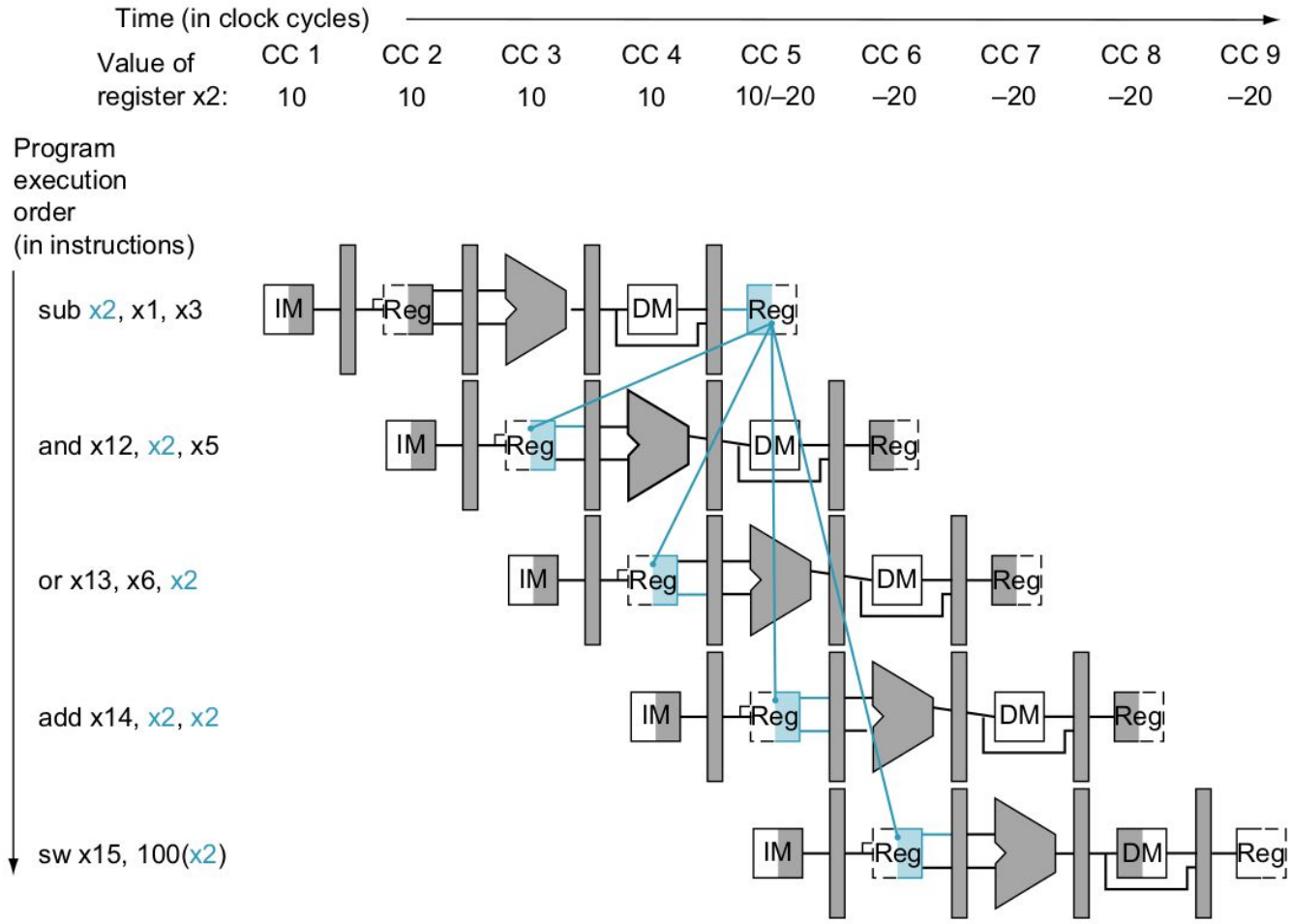
PIPELINED PROCESSOR IMPLEMENTATION:
Handling Data Hazards - Forwarding

Data Hazards in ALU Instructions

```
sub x2, x1, x3
and x12, x2, x5
or x13, x6, x2
add x14, x2, x2
sw x15, 100(x2)
```

- We can resolve hazards with forwarding
  - How do we detect when to forward?

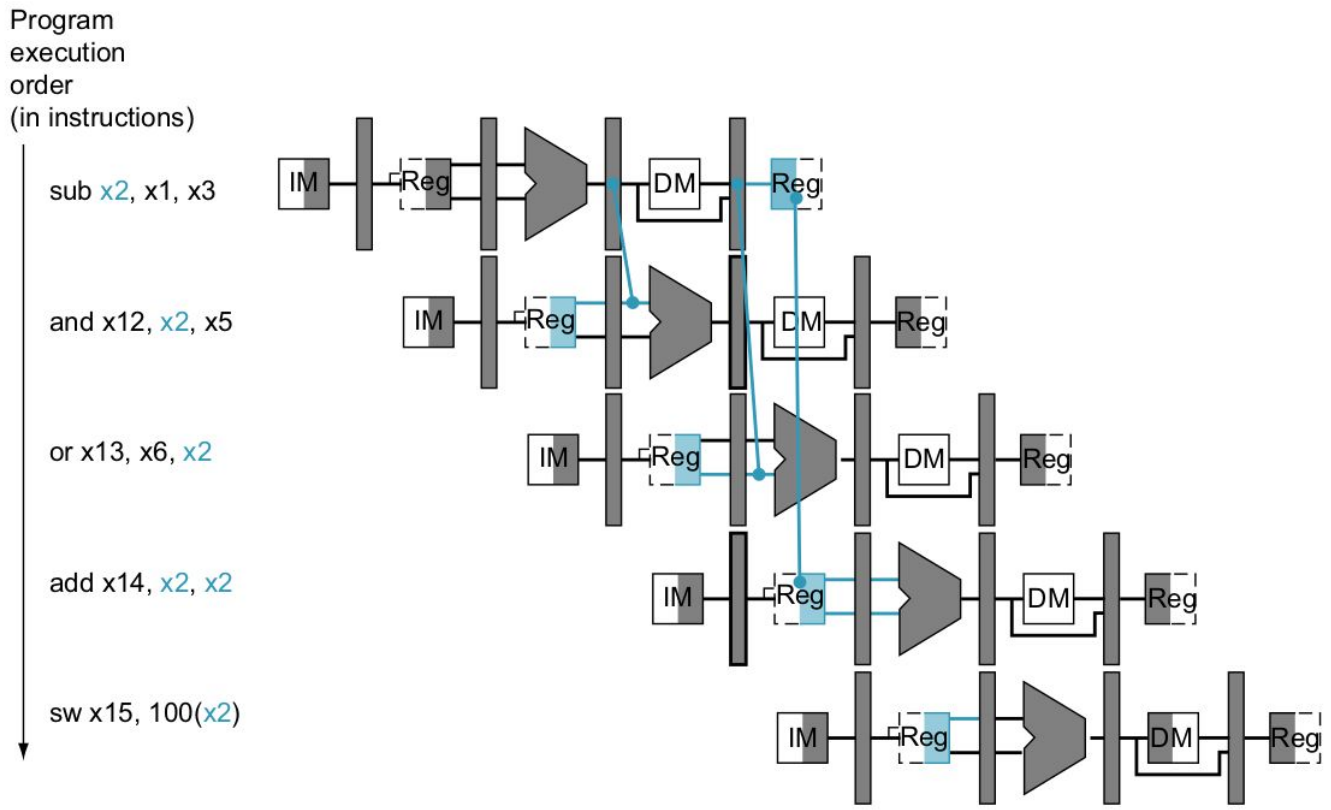
Example: Data Hazards



Example: Data Hazards Resolved with Forwarding

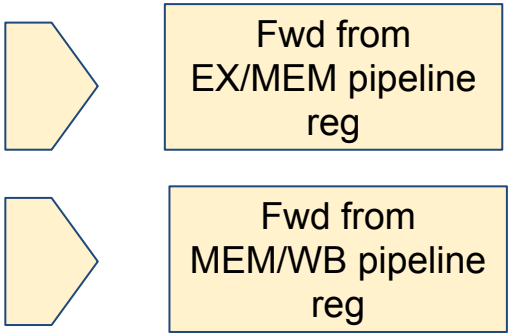
Time (in clock cycles) →

	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9
value of register x2:	10	10	10	10	10/–20	–20	–20	–20	–20



Forwarding Unit Design: Detection

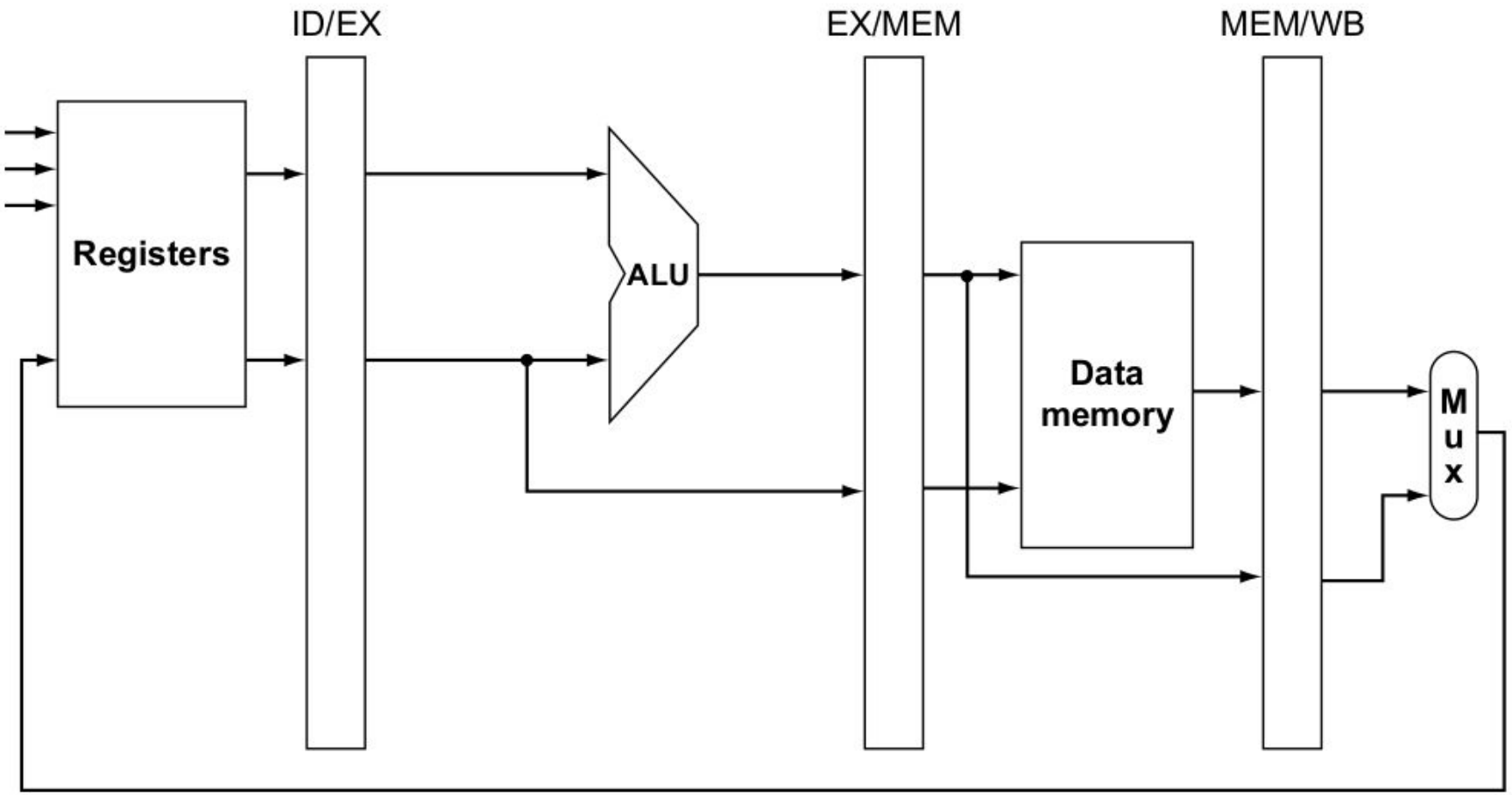
- Pass register numbers along pipeline
  - e.g., ID/EX.RegisterRs1 = register number for Rs1 sitting in ID/EX pipeline register
- ALU operand register numbers in EX stage are given by
  - ID/EX.RegisterRs1, ID/EX.RegisterRs2
- Data hazards when
  - 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
  - 1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
  - 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
  - 2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2



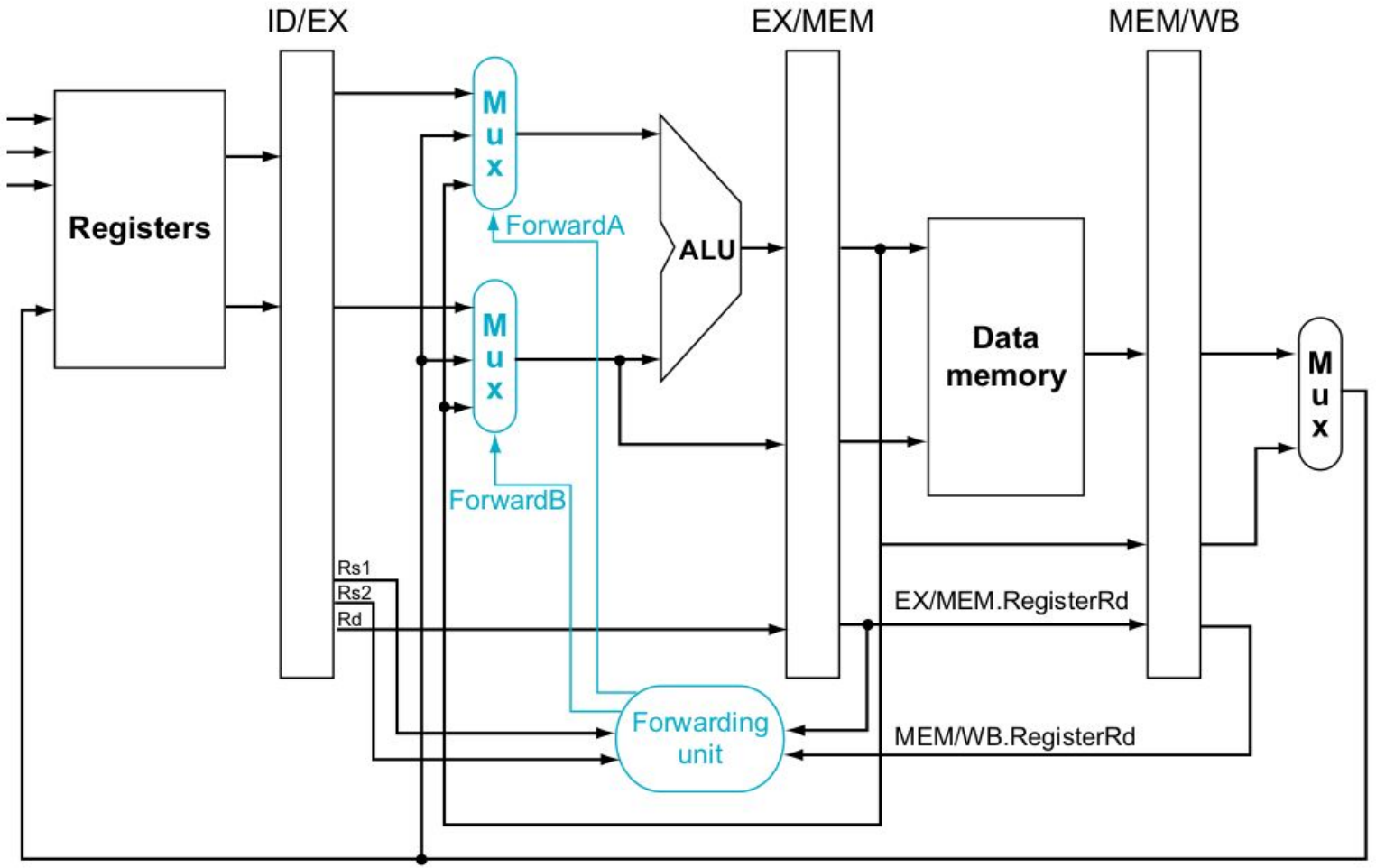
Forwarding Unit Design: Detection

- But only if forwarding instruction will write to a register!
  - EX/MEM.RegWrite
  - MEM/WB.RegWrite
- And only if Rd for that instruction is not x0
  - EX/MEM.RegisterRd ≠ 0
  - MEM/WB.RegisterRd ≠ 0

No Forwarding



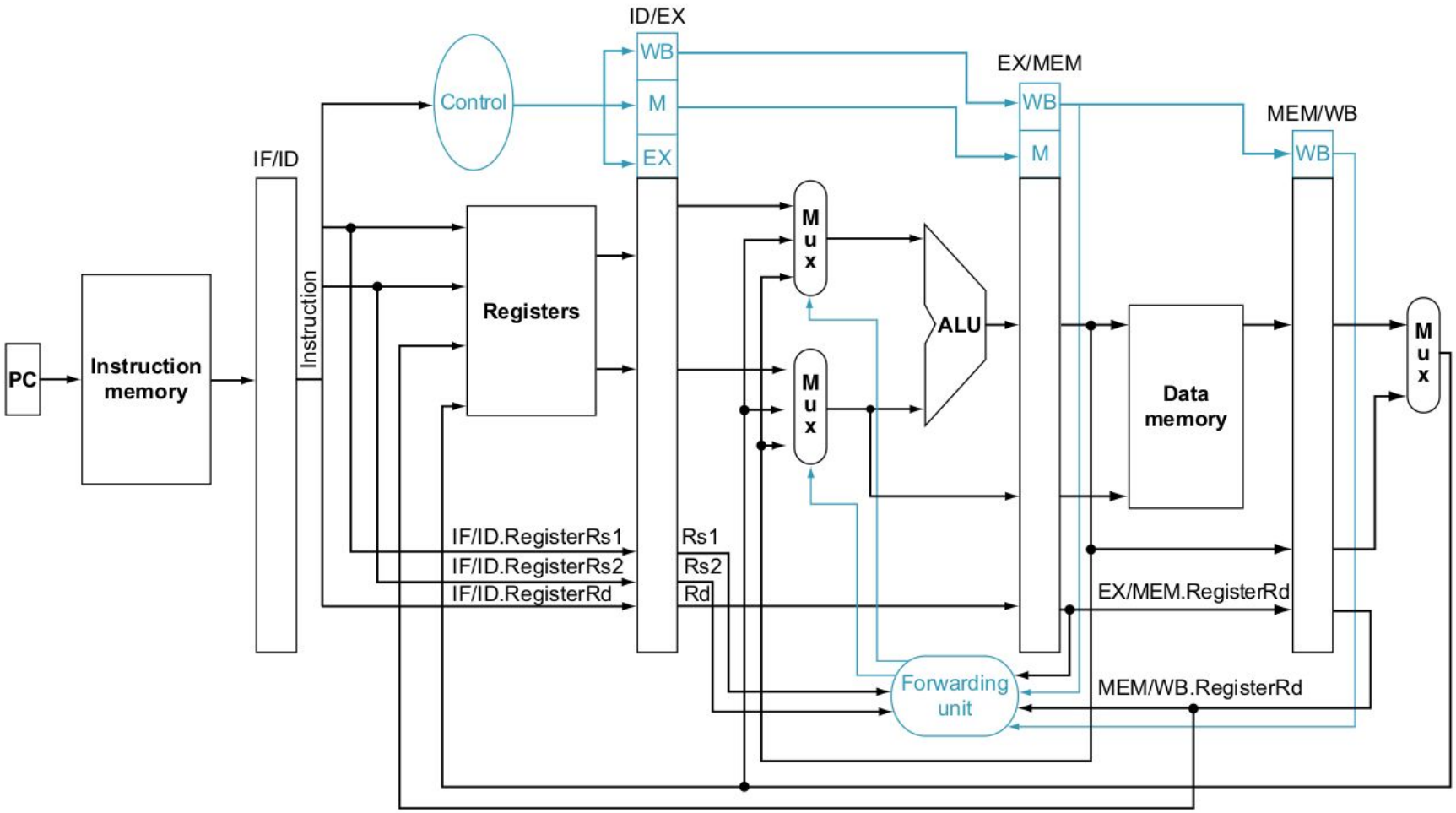
With Forwarding Unit



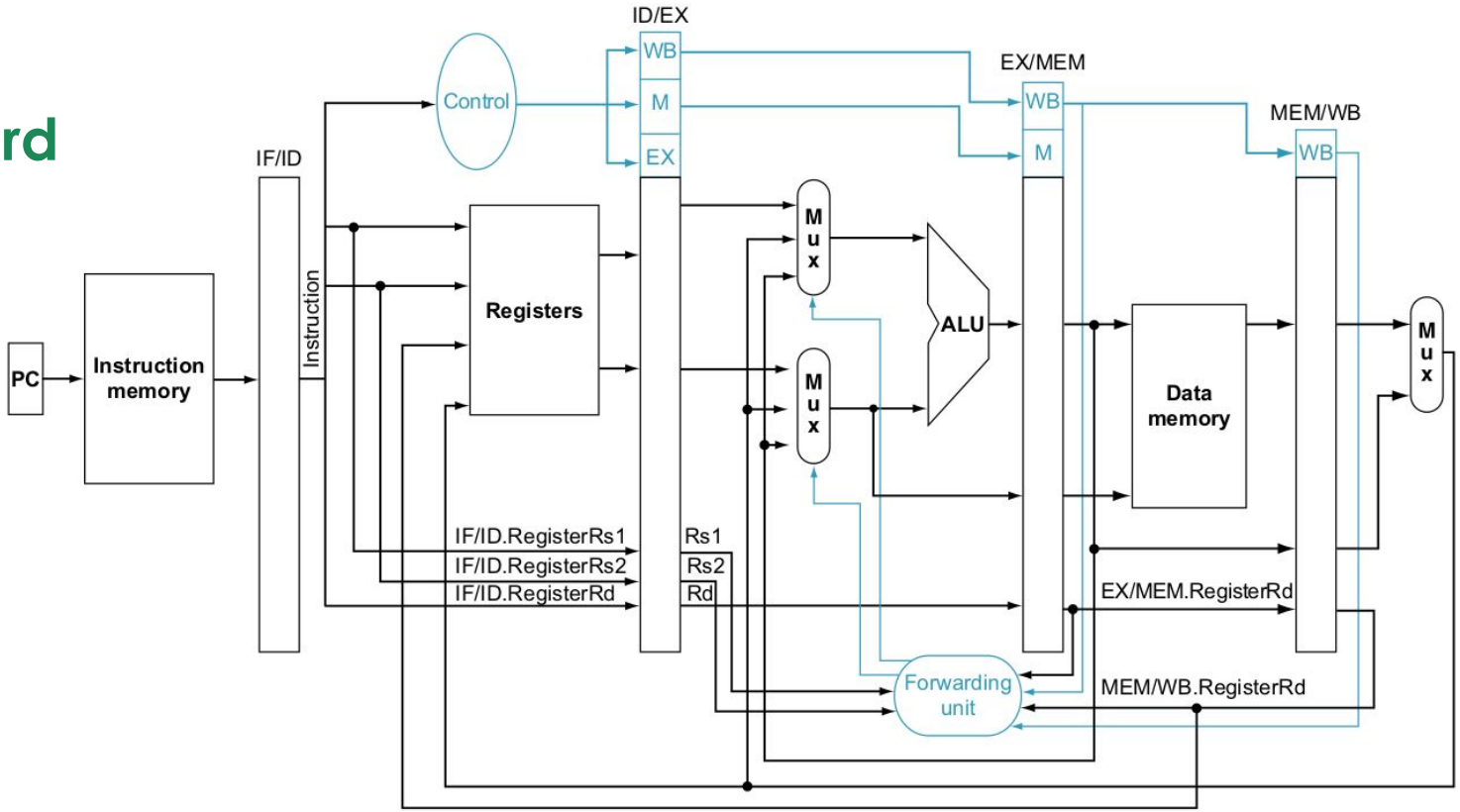
Forwarding Conditions

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Forwarding Unit and Control



EX Hazard



if (EX/MEM.RegWrite

and (EX/MEM.RegisterRd ≠ 0)

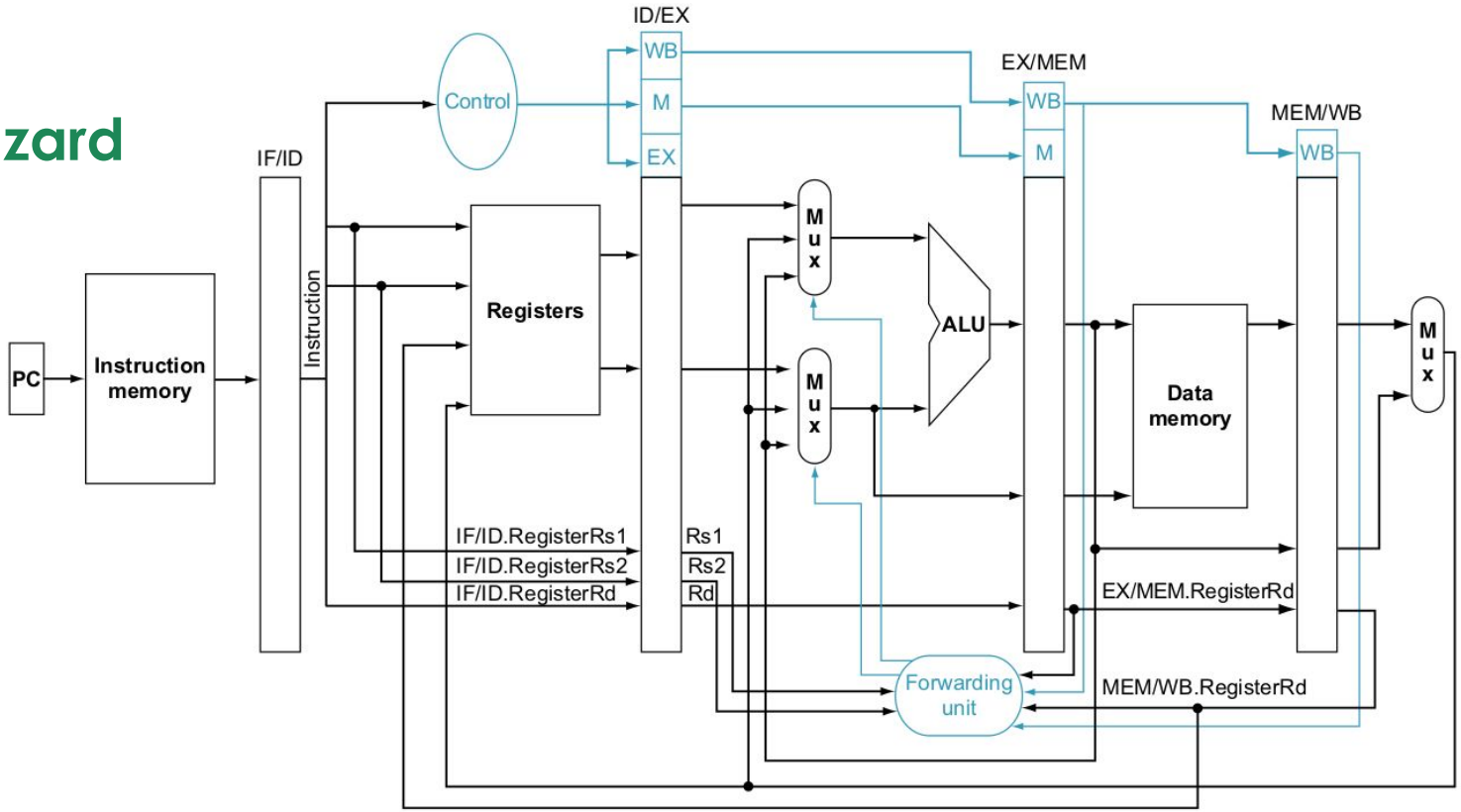
and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 10

if (EX/MEM.RegWrite

and (EX/MEM.RegisterRd ≠ 0)

and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 10

MEM Hazard



if (MEM/WB.RegWrite

and (MEM/WB.RegisterRd ≠ 0)

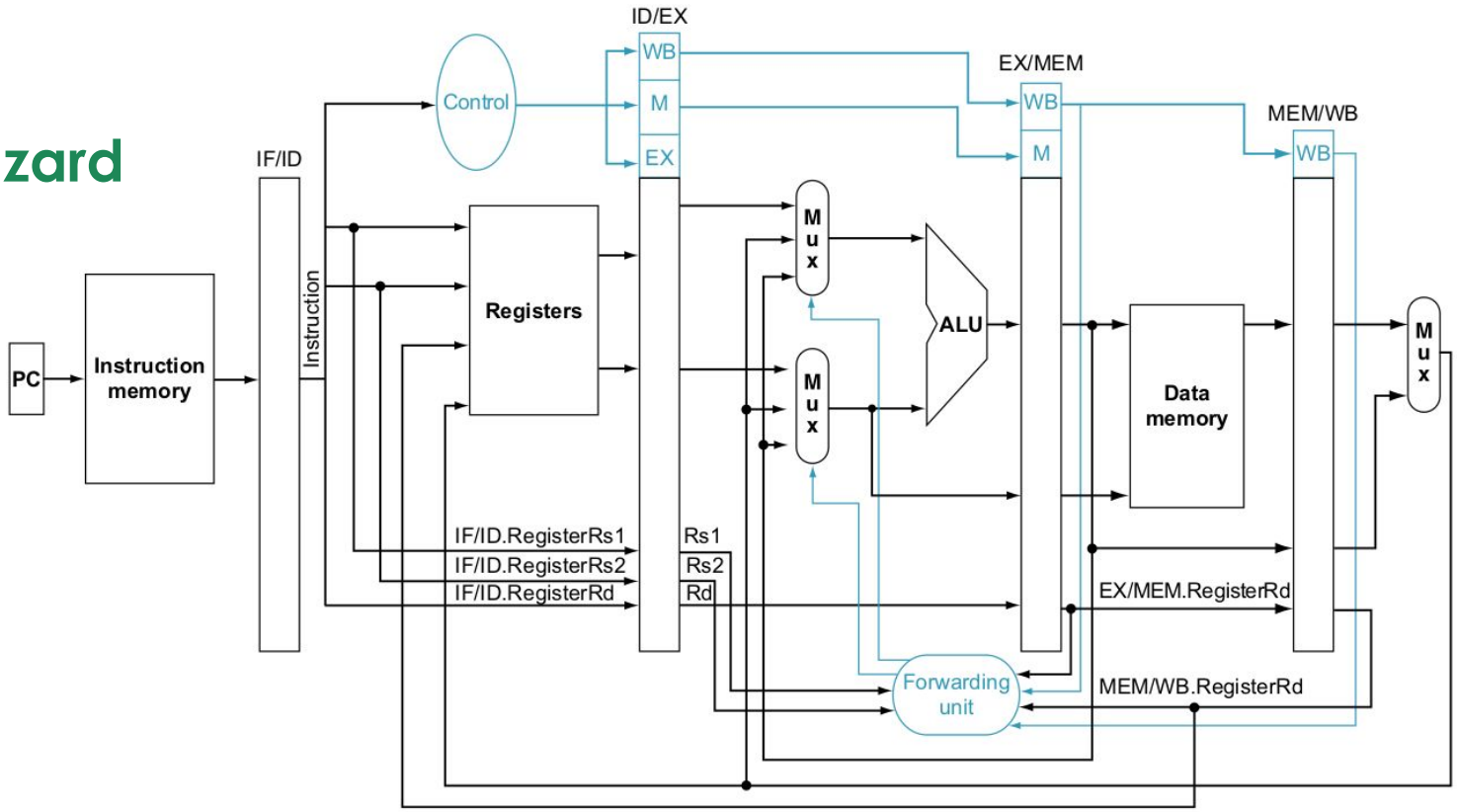
and (MEM/WB.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 01

if (MEM/WB.RegWrite

and (MEM/WB.RegisterRd ≠ 0)

and (MEM/WB.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 01

MEM Hazard



if (MEM/WB.RegWrite

and (MEM/WB.RegisterRd ≠ 0)

and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)

and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))

and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))

ForwardA = 01

if (MEM/WB.RegWrite

and (MEM/WB.RegisterRd ≠ 0)

and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)

and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))

and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))

ForwardB = 01

PIPELINED PROCESSOR IMPLEMENTATION:
Handling Data Hazards - Stalls

Load-Use Hazard Detection

- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
  - IF/ID.RegisterRs1, IF/ID.RegisterRs2
- Load-use hazard when

  ID/EX.MemRead and

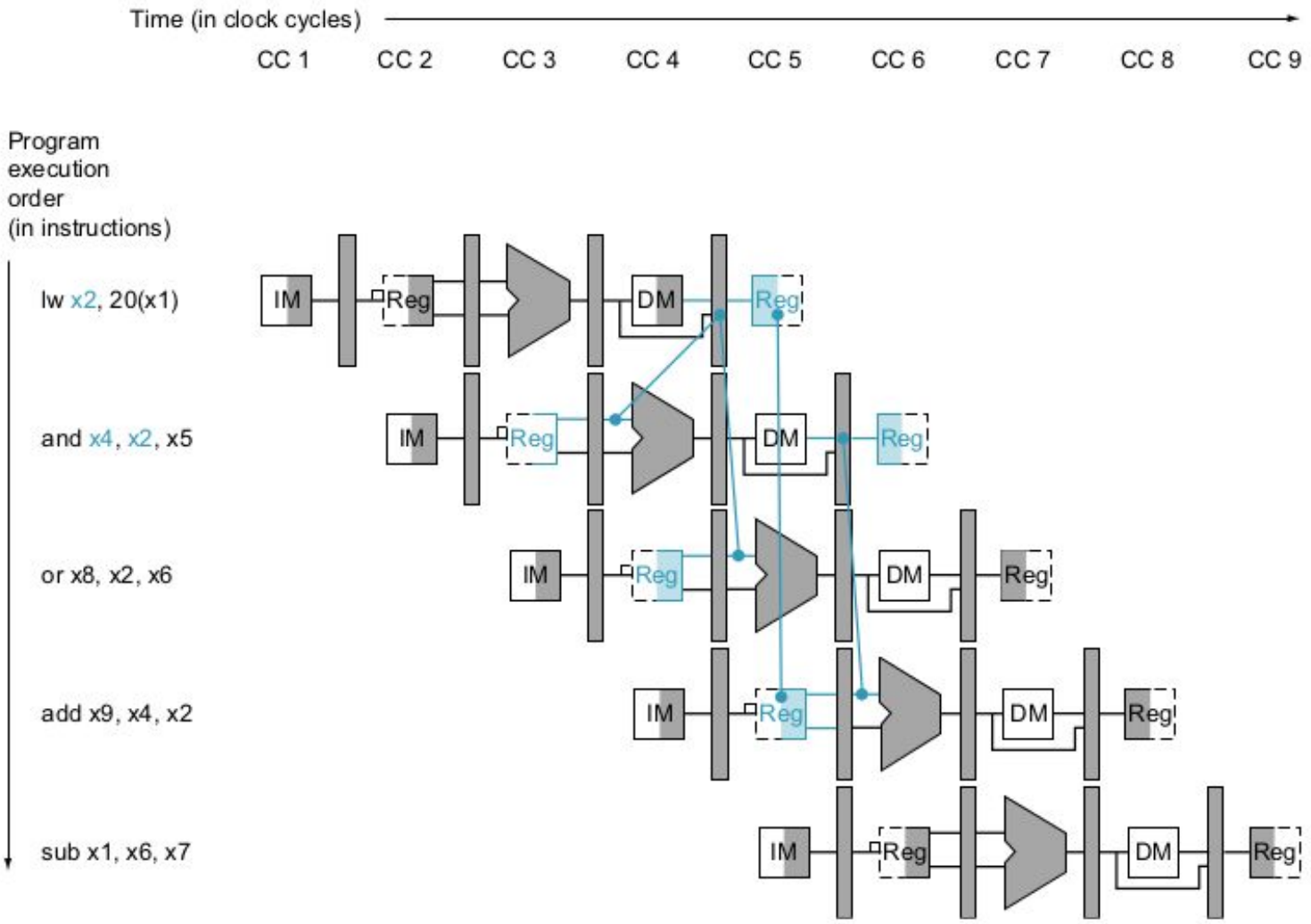
  ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or

  (ID/EX.RegisterRd = IF/ID.RegisterRs1))
- If detected, stall and insert bubble

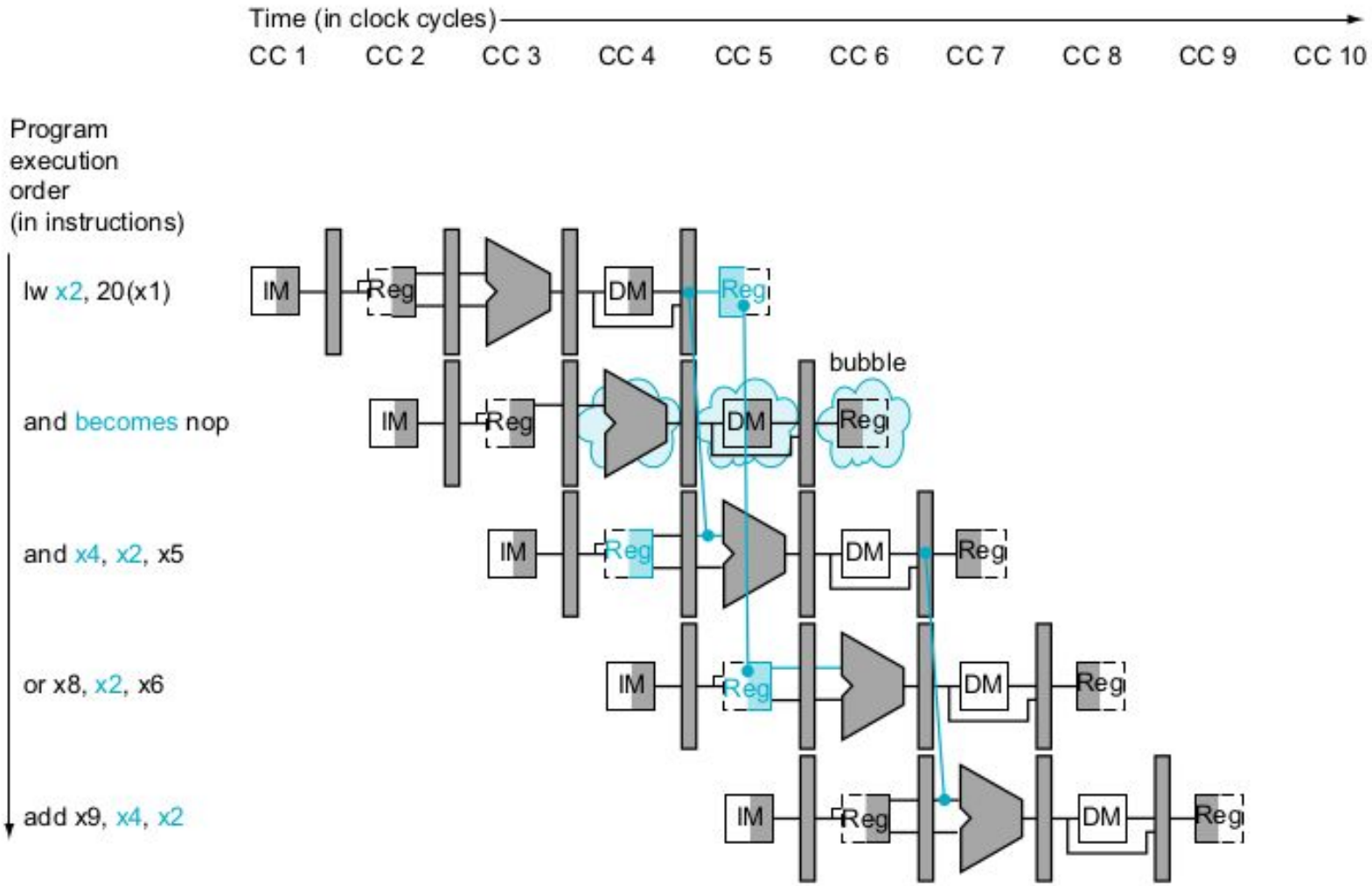
How to Stall the Pipeline

- Force control values in ID/EX register to 0
  - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for ld
    - Can subsequently forward to EX stage

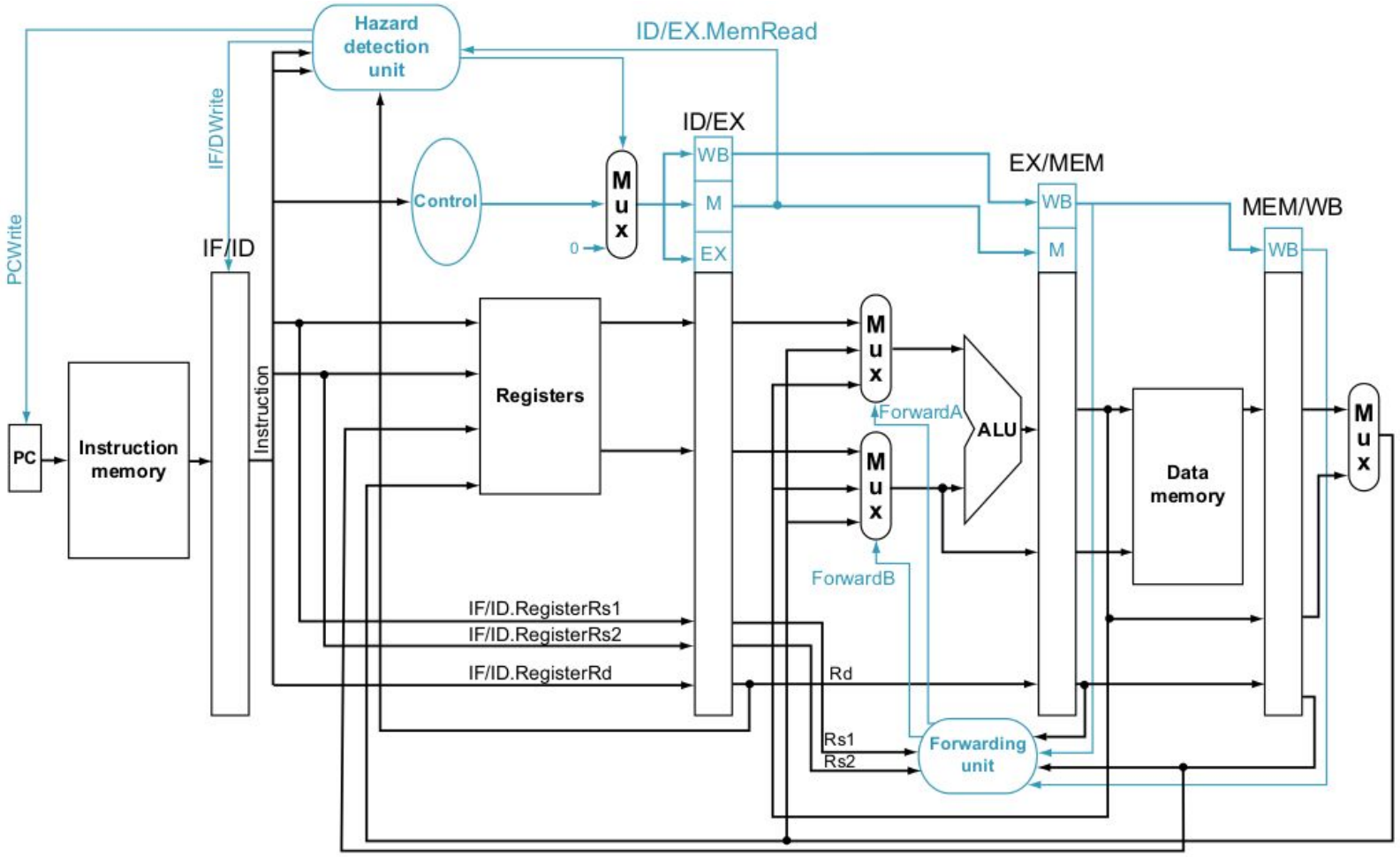
Data Hazards : Cannot be Resolved with Forwarding



Data Hazards : Resolved with Stalling



Pipelined Datapath: Forwarding Unit and Hazard Detection Unit

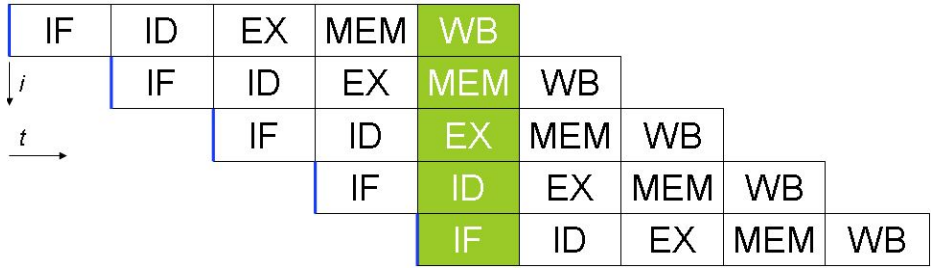


Exercise: Ideal Pipelined Processor CPI

Let us assume a classic RISC pipeline, with the five stages: IF, ID, EX, MEM and WB. Compare the processor's ideal CPI with and without pipelining.

CPI without pipelining: Each stage requires one clock cycle and an instruction passes through the stages sequentially. Without pipelining, in a multi-cycle processor, a new instruction is fetched in stage 1 only after the previous instruction finishes at stage 5, therefore the number of clock cycles it takes to execute an instruction is five (CPI = 5 > 1). In this case, the processor is said to be **subscalar.**

CPI with pipelining: With pipelining, a new instruction is fetched every clock cycle by exploiting instruction-level parallelism, therefore, since one could theoretically have five instructions in the five pipeline stages at once (one instruction per stage), a different instruction would complete stage 5 in every clock cycle and on average the number of clock cycles it takes to execute an instruction is 1 (CPI = 1). In this case, the processor is said to be **scalar.**



Exercise: Forwarding of Stalling?

Question: For each code sequence below, state whether it must stall, can avoid stalls using only forwarding, or can execute without stalling or forwarding.

Sequence 1	Sequence 2	Sequence 3
lw $t0,0($t0) add $t1,$t0,$t0	add $t1,$t0,$t0 addi $t2,$t0,#5 addi $t4,$t1,#5	addi $t1,$t0,#1 addi $t2,$t0,#2 addi $t3,$t0,#2 addi $t3,$t0,#4 addi $t5,$t0,#5

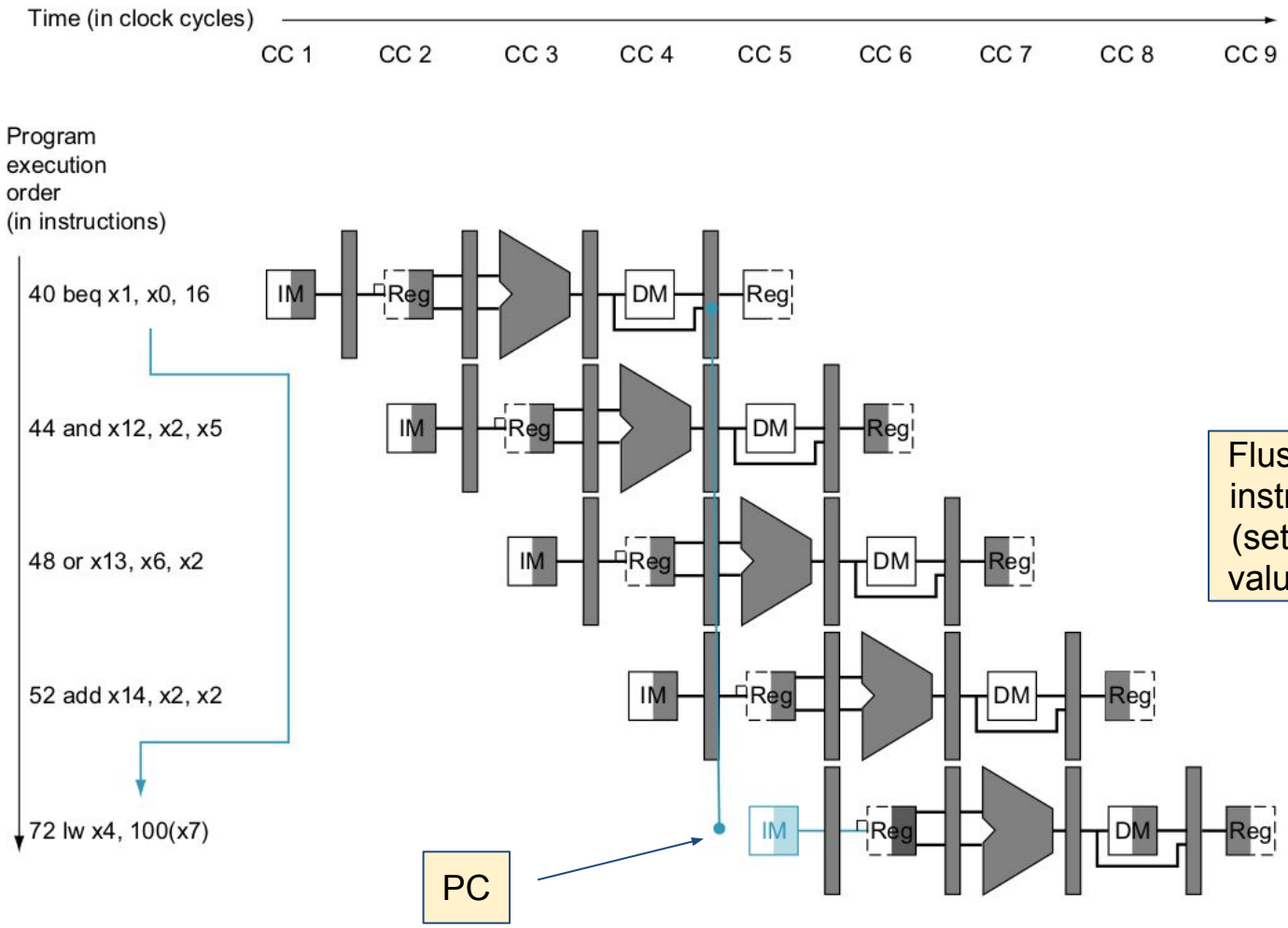
Stalls and Performance

- Stalls reduce performance
  - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
  - Requires knowledge of the pipeline structure

PIPELINED PROCESSOR IMPLEMENTATION:
Handling Control Hazards

Branch Hazard

- If branch outcome determined in MEM

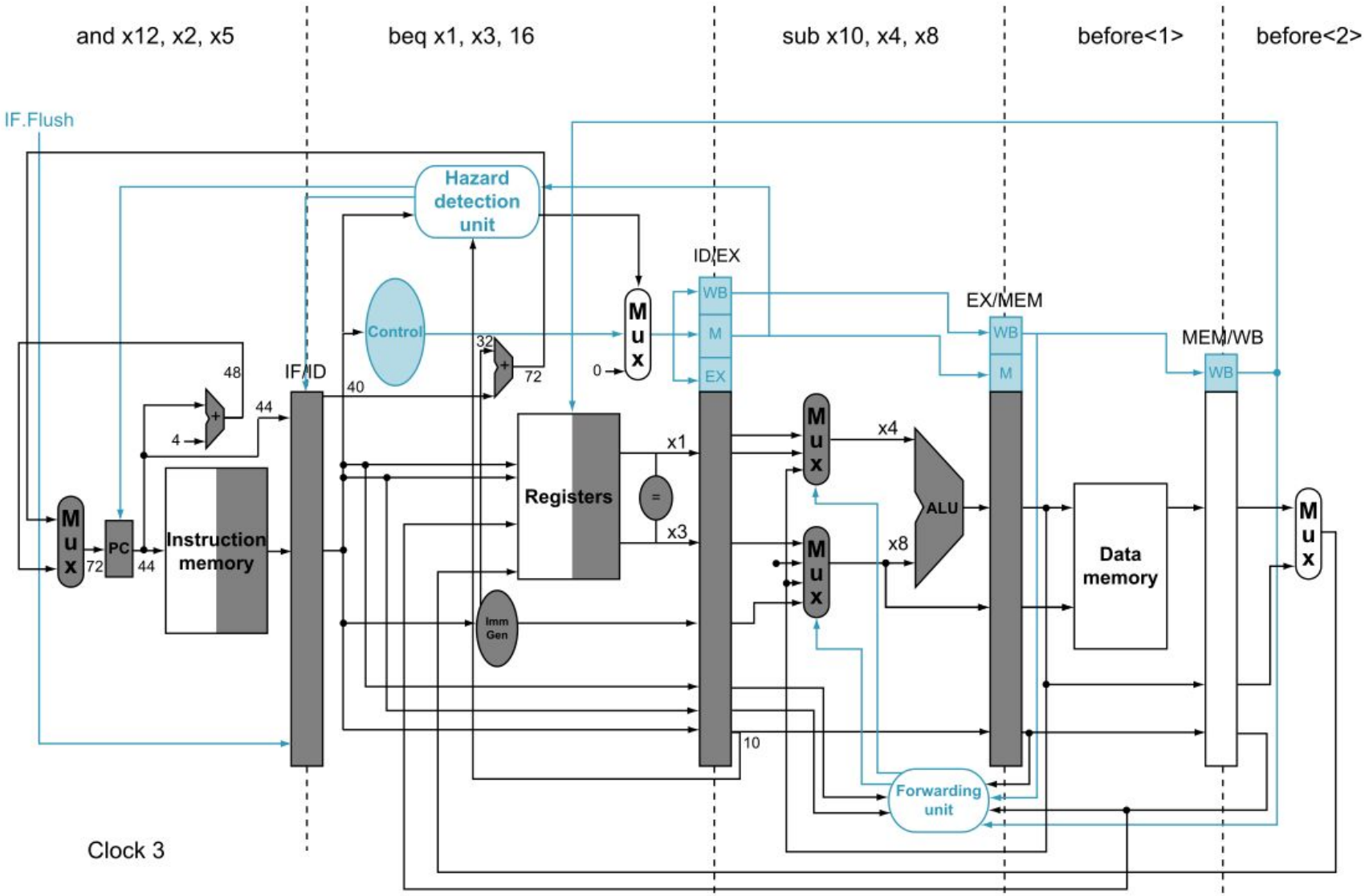


Reducing Branch Delay

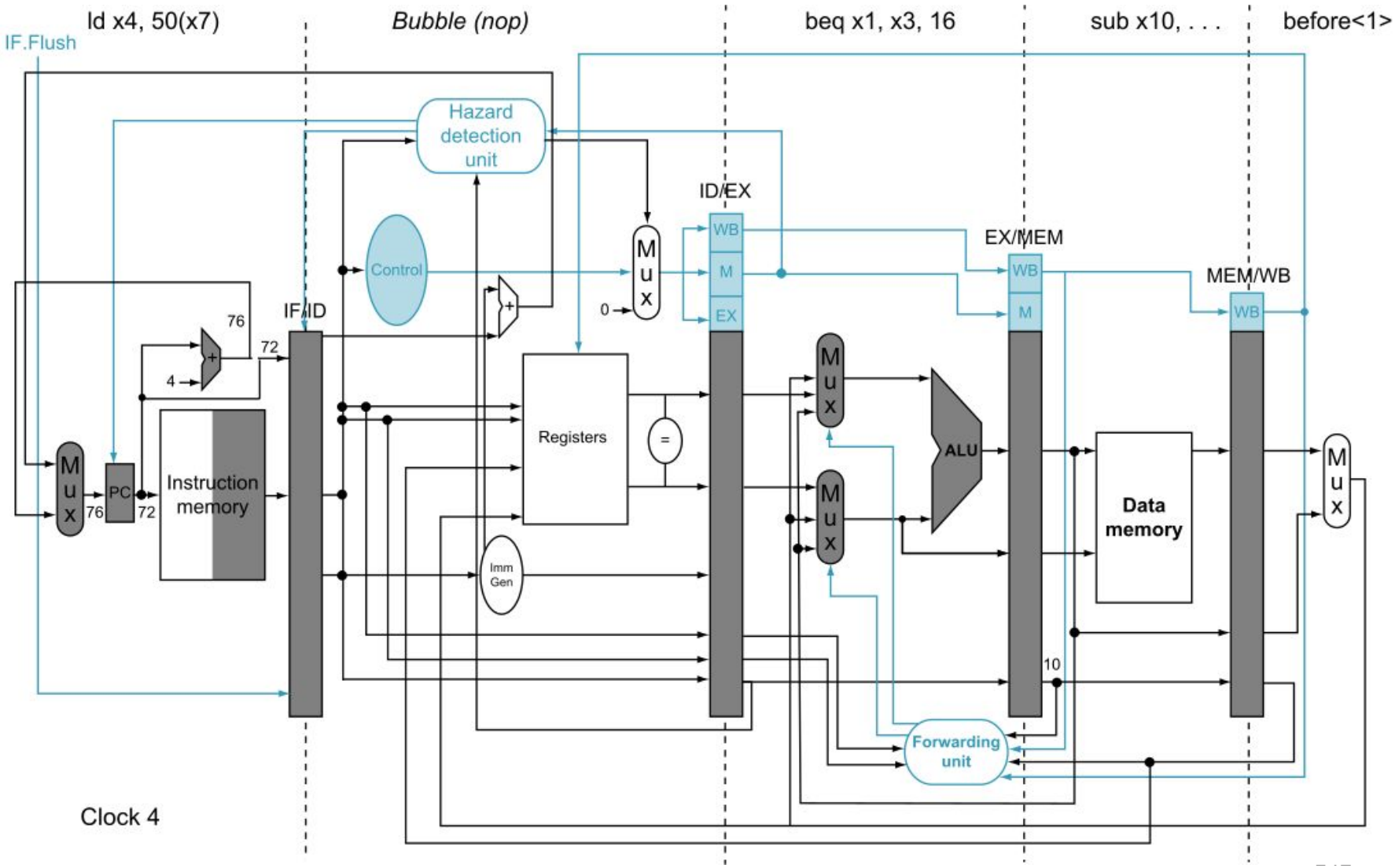
```
36   sub    x10, x4, x8
40   beq    x1, x3, 16      # PC-relative branch to 40 + 16×2 = 72
44   and    x12, x2, x5
48   or     x13, x2, x6
52   add    x14, x4, x2
56   sub    x15, x6, x7
...
72   lw     x4, 50(x7)
```

- Move hardware to determine outcome to ID stage
  - Target address adder
  - Register comparator

Example: Branch Taken



Example: Branch Taken

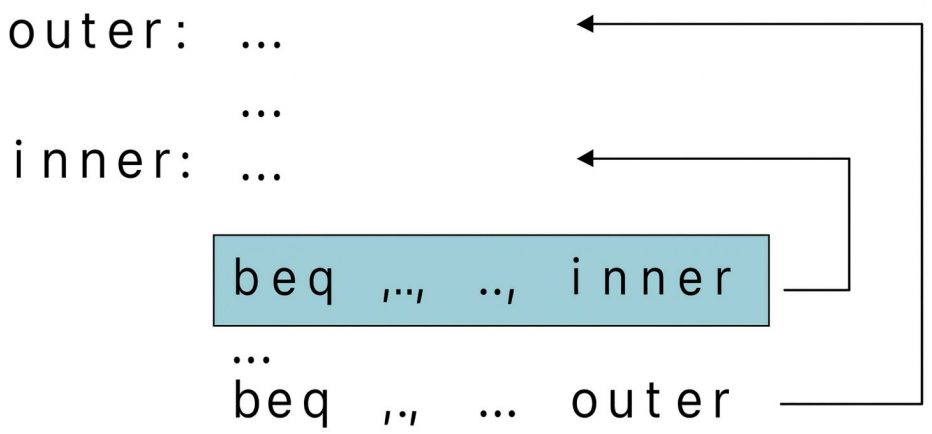


Dynamic Branch Prediction

- In deeper and superscalar pipelines, branch penalty is more significant
- Use dynamic prediction
  - Branch prediction buffer (aka branch history table)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
  - To execute a branch
    - Check table, expect the same outcome
    - Start fetching from fall-through or target
    - If wrong, flush pipeline and flip prediction

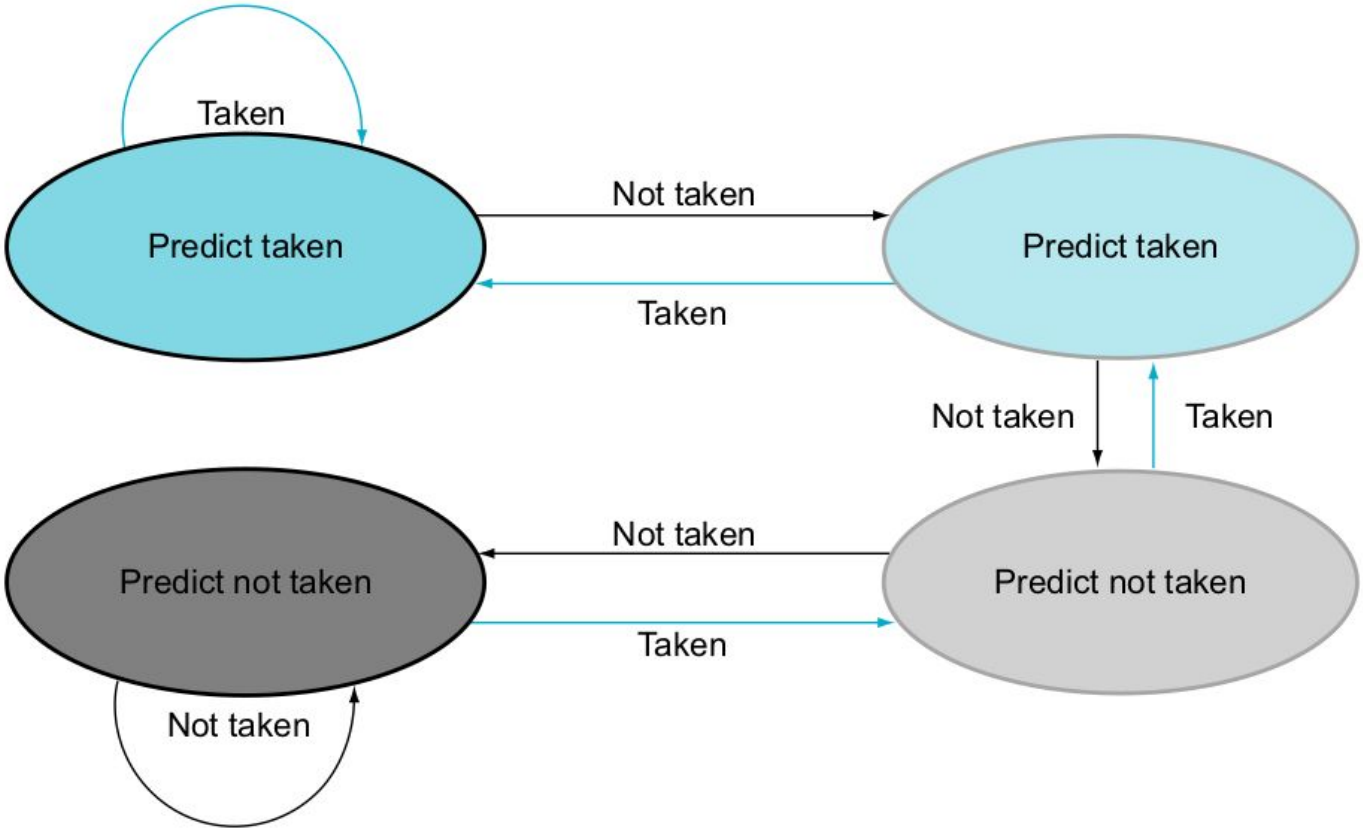
Dynamic 1-Bit Predictor: Shortcoming

- Inner loop branches mispredicted twice!
  - Mispredict as taken on last iteration of inner loop
  - Then mispredict as not taken on first iteration of inner loop next time around



2-Bit Predictor

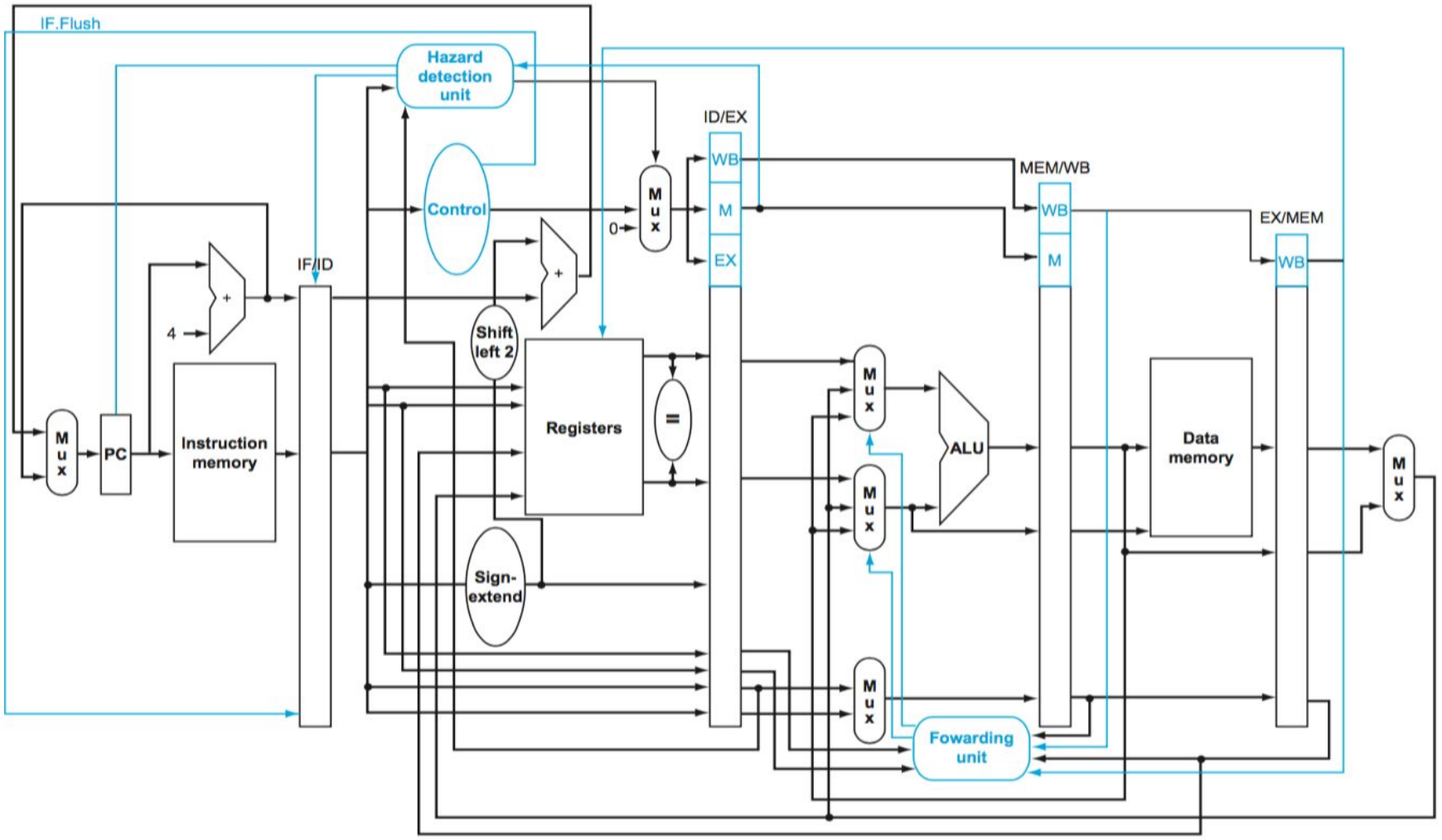
- Only change prediction on two successive mispredictions



Calculating the Branch Target

- Even with predictor, still need to calculate the target address
  - 1-cycle penalty for a taken branch
- Branch target buffer
  - Cache of target addresses
  - Indexed by PC when instruction fetched
    - If hit and instruction is branch predicted taken, can fetch target immediately

Control Hazard: Early Branch Decision Implementation



Exceptions and Interrupts

- "Unexpected" events requiring change in flow of control
  - Different ISAs use the terms differently
- Exception
  - Arises within the CPU
  - e.g., undefined opcode, syscall, …
- Interrupt
  - From an external I/O controller
  - Dealing with them without sacrificing performance is hard

Exceptions and Interrupts

Type of event	From where?	RISC-V terminology
System reset	External	Exception
I/O device request	External	Interrupt
Invoke the operating system from user program	Internal	Exception
Using an undefined instruction	Internal	Exception
Hardware malfunctions	Either	Either

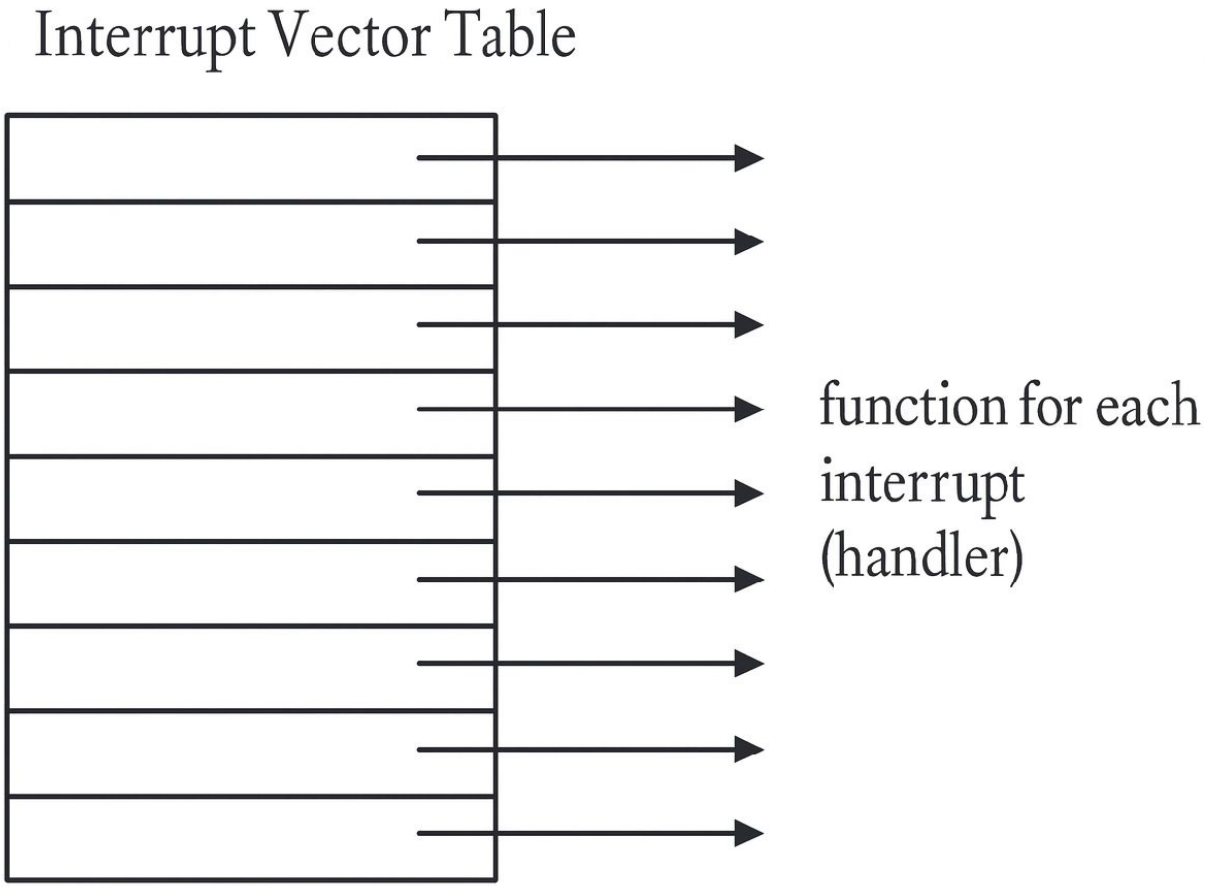
Handling Exceptions

- Save PC of offending (or interrupted) instruction
  - In RISC-V: Supervisor Exception Program Counter (SEPC)
- Save indication of the problem
  - In RISC-V: Supervisor Exception Cause Register (SCAUSE)
  - 64 bits, but most bits unused
    - Exception code field: 2 for undefined opcode, 12 for hardware malfunction, …
- Jump to handler
  - Assume at 0000 0000 1C09 0000hex

An Alternate Mechanism

- Vectored Interrupts
  - Handler address determined by the cause
- Exception vector address to be added to a vector table base register:
  - Undefined opcode 00 0100 0000two
  - Hardware malfunction: 01 1000 0000two
  - …: …
- Instructions either
  - Deal with the interrupt, or
  - Jump to real handler

An Alternate Mechanism



Handler Actions

- Read cause, and transfer to relevant handler
- Determine action required
- If restartable
  - Take corrective action
  - use SEPC to return to program
- Otherwise
  - Terminate program
  - Report error using SEPC, SCAUSE, …

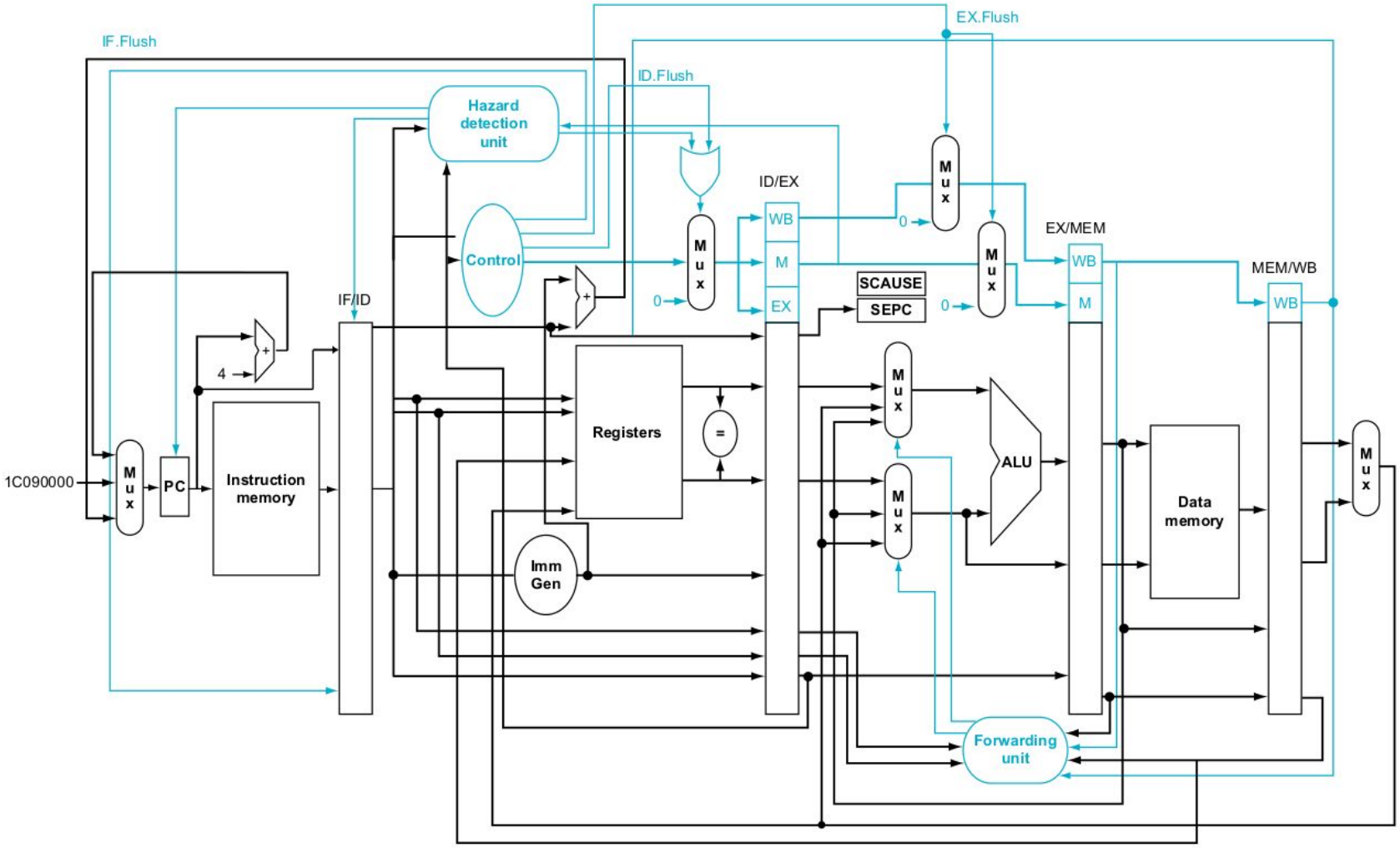
Exceptions in a Pipeline

- Another form of control hazard
- Consider malfunction on add in EX stage

  add x1, x2, x1

  - Prevent x1 from being clobbered
  - Complete previous instructions
  - Flush add and subsequent instructions
  - Set SEPC and SCAUSE register values
  - Transfer control to handler
- Similar to mispredicted branch
  - Use much of the same hardware

Pipeline with Exceptions



Exception Properties

- Restartable exceptions
  - Pipeline can flush the instruction
  - Handler executes, then returns to the instruction
    - Refetched and executed from scratch
- PC saved in SEPC register
  - Identifies causing instruction

Exception Properties

- Restartable exceptions
  - Pipeline can flush the instruction
  - Handler executes, then returns to the instruction
    - Refetched and executed from scratch
- PC saved in SEPC register
  - Identifies causing instruction

Exception Properties

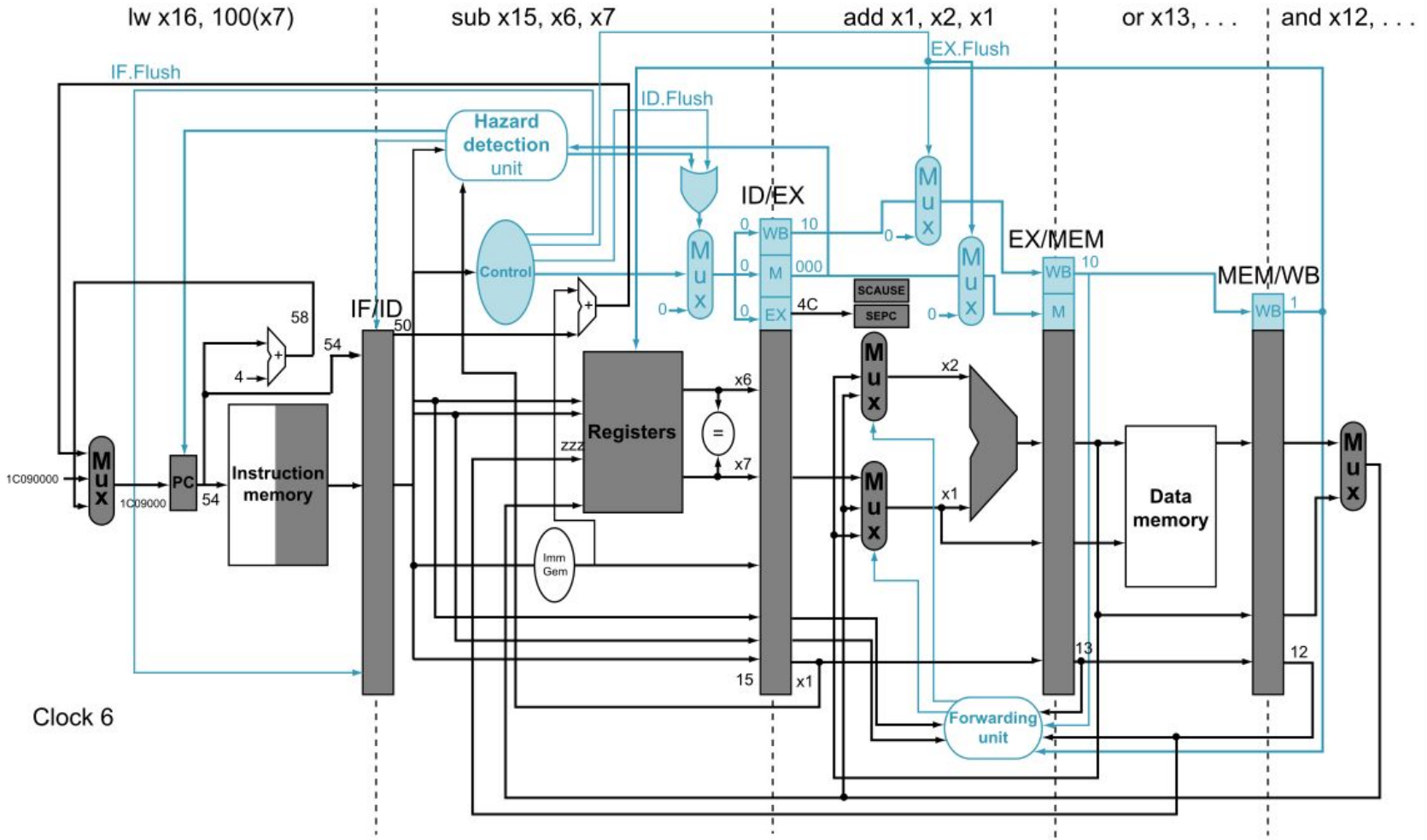
- Instruction sequence:

```
0x40    sub     x11, x2, x4
0x44    and     x12, x2, x5
0x48    or      x13, x2, x6
0x4C    add     x1,  x2, x1
0x50    sub     x15, x6, x7
0x54    lw      x16, 100(x7)
...
```

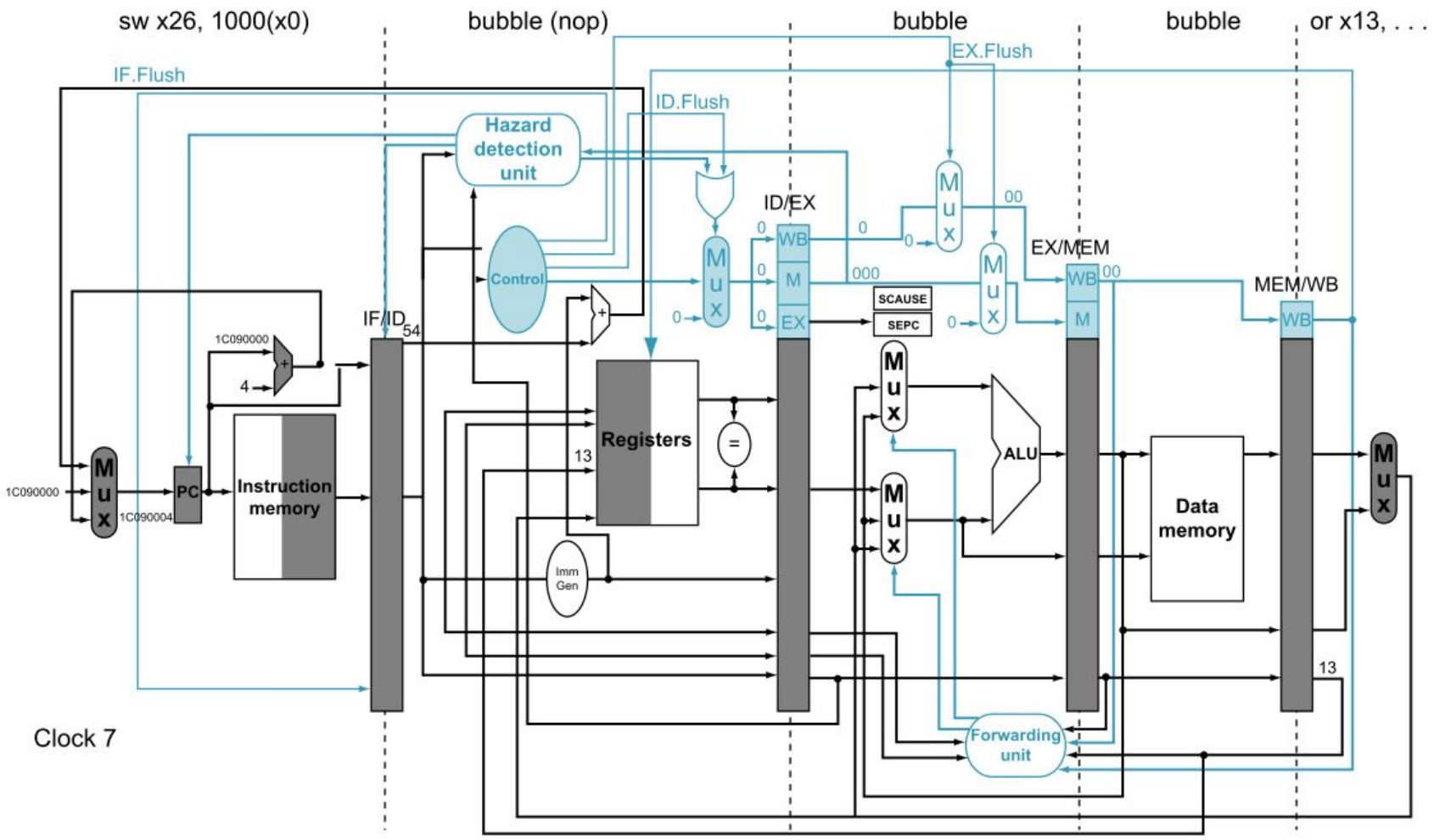
- Handler:

```
0x1C090000    sw     x26, 1000(x10)
0x1C090004    sw     x27, 1008(x10)
...
```

Exception Example



Exception Example



Fallacies

- Pipelining is easy (!)
  - The basic idea is easy
  - The devil is in the details
    - e.g., detecting data hazards
- Pipelining is independent of technology
  - So why haven't we always done pipelining?
  - More transistors make more advanced techniques feasible
  - Pipeline-related ISA design needs to take account of technology trends
    - e.g., predicated instructions

Conclusions

- ISA influences design of datapath and control
- Datapath and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
  - More instructions completed per second
  - Latency for each instruction not reduced
- Hazards: structural, data, control